

Phenix Ligand and Restraint Tools

16 documentation pages

Generated from local Phenix documentation

Contents

1. Making geometry restraints
2. Ligands
3. electronic Ligand Builder and Optimisation Workbench (eLBOW)
4. Generating ligand structures and restraints in the eLBOW GUI
5. Calculating Fragment Antigen-Binding (Fab) elbow angle
6. Finding all ligands in a map with `phenix.find_all_ligands`
7. Guided Ligand Replacement
8. Automated ligand identification
9. Automated protein-ligand structure determination with `phenix.ligand_pipeline`
10. Automated ligand fitting with LigandFit
11. Using the LigandFit GUI
12. Generating a polder map
13. Generating a polder map with the `compute_box` option
14. Quantum Mechanical Restraints (QMR) usage
15. ReadySet!
16. `phenix.secondary_structure_restraints`

Making geometry restraints

[making_geometry_restraints.html](#)

- Secondary structure restraints
- Restraints for ligands

Ligands

ligands.html

Why

The goal of a crystallographic study is often to visualize the interaction of a small molecule or ligand with a macromolecule. This is a core process in the development of new therapeutics. It is also common to discover density for an unanticipated small molecule in complex with a protein while solving a structure. In both cases, you need to fit the small molecule into the electron density to complete the atomic model. As part of this process, you frequently need to generate appropriate stereochemical restraints for new ligands for use in structure refinement.

How

In Phenix, the principal tool for fitting ligands into electron density is `phenix.ligand_fit`. This program identifies unfilled electron density in a map and attempts to place the user-defined ligand in the density by rotating around any torsional degrees of freedom. The process requires an electron density map, coordinates for the macromolecule, and information about the ligand (either as coordinates or as a chemical description that can be used to generate coordinates). The program outputs the coordinates of the ligand fitted into the density.

After ligand(s) are fit, subsequent refinement steps will require stereochemical restraints. In some cases, these may be present in the default restraint libraries. However, you often need to generate new restraints. The `phenix.elbow` (eLBOW) program automatically generates geometry restraints for novel ligands or improves restraints for standard ligands, starting from a variety of different formats.

How to use the ligand fitting GUI: [Click here](#)

Phenix reference manual for `phenix.ligand_fit`

How to use the `phenix.elbow` GUI: [Click here](#)

Phenix reference manual for `phenix.elbow`

Common issues

- [Frequently asked questions about ligands](#)

Related programs

- `phenix.guided_ligand_replacement`: This program uses prior knowledge from a previously fit ligand to expedite the fitting of a similar ligand into the same or similar protein. This is very useful when studying a series of compounds for the same or related macromolecular targets.
- `phenix.ligand_identification`: This program determines the identity of an unknown ligand by analyzing difference density peaks to reveal which ligand is likely to be present. Using a library of the 180 most frequently observed ligands in the PDB, the tool ranks each molecule by density fit and chemical interaction with the macromolecule.
- `phenix.ready_set`: This program prepares a model for refinement by generating all necessary ligand restraints with eLBOW and then updating the model file to reflect atom name changes from the new restraints.
- `phenix.reel`: This Restraints Editor Especially Ligands (REEL) tool includes a 3D view of the ligand and a tabular view of the restraints, allowing target values and standard deviations to be easily edited.

electronic Ligand Builder and Optimisation Workbench (eLBOW)

elbow.html

Contents

- Author
- Purpose
- Video Tutorial
- Examples PDB input SMILES input Other input Geometry optimisation Hydrogen addition
- PDB input
- SMILES input
- Other input
- Geometry optimisation
- Hydrogen addition
- Output
- Notes
- Advanced options Chiral Cis/Trans Pucker
- Chiral
- Cis/Trans
- Pucker
- Additional programs Literature Additional information Novice options Expert options
- Literature
- Additional information
- Novice options
- Expert options

Author

Nigel W. Moriarty

Purpose

Automate the generation of geometry restraint information for refinement of novel ligands and improved geometry restraint information for standard ligands. A protein crystal can contain more than just protein and other simple molecules that most refinement programs can interpret. An unusual molecule can be included in the refinement via eLBOW from a number of chemical inputs. The geometry can be optimised using various levels of chemical knowledge including a semi-empirical quantum mechanical method known as AM1.

Input formats include

- SMILES string
- PDB (Protein Data Bank)

- MolFiles (V2000, V3000 and SDFfiles)
- TRIPOS MOL2
- XYZ
- certain CIF files
- GAMESS input and output files

Output formats include

- PDB (Protein Data Bank)
- CIF restraint file
- eLBOW contains a number of programs. All programs have been written to allow command-line control and script access to the objects and algorithm. The main program is run thus:

```
phenix.elbow [options] input_file.ext
```

or in a Python script

```
from elbow.command_line import builder

molecule = builder.run("input_file.ext", **kwargs)
```

where the options are passed as a dictionary. The return object can be interrogated for information via the class methods. Output files from both techniques include a PDB file of the final geometry and a CIF file that contains the geometry restraint information for refinement. Other files are output as appropriate, such as edits and CIF files for linking the ligand to the protein. A final file contains the serialised data of the molecule in the Python pickle format.

Video Tutorial

The tutorial video is available on the Phenix YouTube channel and covers the following topics:

- Basic overview
- How to run eLBOW in the Phenix GUI

Examples

PDB input

To run eLBOW on a PDB file (containing one molecule)

```
phenix.elbow input_file.pdb
```

To run eLBOW on a PDB file containing protein and ligands. This will only process the ligands that are unknown to phenix.refine.

```
phenix.elbow input_file.pdb --do-all
```

To run eLBOW on a PDB file specifying a residue

```
phenix.elbow input_file.pdb --residue LIG
```

To use the atom names from a PDB file

```
phenix.elbow --smiles 0 --template input_file.pdb
```

SMILES input

To run eLBOW on a SMILES string

```
phenix.elbow --smiles="CCO"  
phenix.elbow --smiles=input_file.smi
```

Other input

To run eLBOW on other supported input formats

```
phenix.elbow input_file.ext
```

Geometry optimisation

eLBOW performs a simple force-field geometry optimisation by default, however an AM1 geometry optimisation can be performed as follows.

```
phenix.elbow input_file.pdb --opt
```

To start from a specific geometry for the optimisation

```
phenix.elbow --initial_geometry input_file.pdb --opt
```

To use a separately installed GAMESS and do a HF/3-21G geometry optimisation

```
phenix.elbow input_file.pdb --gamess --basis="3-21G"
```

To not optimise, but use the input geometry as the final geometry

```
phenix.elbow --final_geometry input_file.pdb
```

Hydrogen addition

eLBOW automatically adds hydrogens to the input molecules if there are less than a quarter of the possible hydrogens. This can be controlled using

```
phenix.elbow input_file.pdb --add-hydrogens=True
```

A common requirement is to add hydrogens to a ligand but retain the geometry and position relative to a protein. To do so use

```
phenix.elbow --final-geometry=input_file.pdb
```

Output

To choose the base name of the output files

```
phenix.elbow input_file.pdb --output="output"
```

To change the three letter ID

```
phenix.elbow input_file.pdb --id=NEW
```

To change other attributes

```
phenix.elbow input_file.pdb --pdb-assign "resSeq=3 b=100"
```

Some of the attributes.

- Residue name : resname
- Chain ID : chain, chainid
- Residue sequence ID : resseq, resid

- Alternative location ID : altid, altloc
- Insert code : icode
- Occupancy : occ, occupancy
- Temperature factor : b, tempfactor
- Segment ID : segid, segID

To output MOL2 format

```
phenix.elbow input_file.pdb --tripos
```

To output PDB Ligand output

```
phenix.elbow input_file.pdb --pdb-ligand
```

Notes

This program is used to generate restraints for either X-ray, CryoEM or MicroED. Next steps generally involve using the ligand geometry and restraints to fit a ligand into the model and experimental data. Then proceeding to refinement in either phenix.refine or phenix.real_space_refine.

Advanced options

Chiral

The options for chirality are "retain" (default), "both" and "enumerate". The default uses the chiral centres specified in the molecule input file or string as the guide for specifying the chirality in the restraints. The "both" will set all chiral centres in the input to "both". This means that each chiral centre can be either one or the other.

The "enumerate" option will write a restraints (CIF) and geometry (PDB) file for each permutation of the chiral centres. The number of permutation is 2^n (n = number of chiral centres). The files are labeled using the R/S notation. Each file has a string listing the configuration of the chiral centres based on the input order, e.g. _RSR, _RSS. The unappended file is the input configuration.

Cis/Trans

Cis/Trans moieties can occur centre on double bonds. SMILES can specify the exact configuration. Otherwise, eLBOW chooses the lowest energy configuration. The "retain" choice does exactly that.

The "enumerate" option will write a restraints (CIF) and geometry (PDB) file for each permutation of the cis/trans moiety. The number of permutation is 2^n (n = number of cis/trans moieties). The files are labeled using the E/Z notation. Each file has a string listing the configuration of the cis/trans moiety based on the input order, e.g. _EEZ, _ZEE. The unappended file is the input configuration.

Pucker

The pucker options are similar to the Chiral and Cis/Trans options. However, because it is a more complicated situation, the inputs are also complicated. Similarly, the default is to use input and if not specified, the position of the groups on the ring are placed relative to the other groups on the ring atom randomly and in the axial (A) or equatorial (Q) position based on the simple force field optimisation.

The "enumerate" option will write a restraints (CIF) and geometry (PDB) file for each permutation of the pucker group moiety. The number of permutation is 2^n (n = number of pucker group moieties). The files are labeled using the A/Q notation invented by the author. Each file has a string listing the configuration of the pucker group moiety based on the input order, e.g. _AAA, _QAA. The unappended file is the input configuration.

The definition of the A and Q needs more clarification. Consider a ring in the chair configuration. At any atom in the ring, a group can be attached in the axial or equatorial position. Whether it is an "up equatorial" or "down equatorial" depends on the ring pucker and orientation.

To force a ring into a specific pucker, use REEL. The define what configuration of the pucker group use the "user_defined" option.

Additional programs

- phenix.get_smiles
- phenix.get_pdb
- phenix.metal_coordination : Generate edits for metal coordination
- phenix.link_edits : Generate edits from PDB LINK records
- phenix.print_sequence
- elbow.become_expert
- elbow.become_novice
- elbow.compare_two_molecules
- elbow.join_cif_files
- elbow.join_pdb_files
- elbow.join_mol2_files
- elbow.check_residues_against_monomer_lib
- elbow.defaults : Generate a eLBOW defaults file

Literature

Moriarty NW, Grosse-Kunstleve RW, Adams PD: electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. *Acta Cryst.* 2009, **D65**:1074-1080.

Additional information

Novice options

Expert options

Generating ligand structures and restraints in the eLBOW GUI

elbow_gui.html

Overview

The electronic Ligand Builder and Optimization Workbench (eLBOW) is the primary tool for generating non-standard ligand restraints in Phenix. In addition to existing as a standalone program, it is also used internally by the LigandFit wizard and phenix.ready_set (integrated with the phenix.refine GUI). In addition to eLBOW, a separate standalone graphical restraint editor is available for advanced customization of restraints and structures.

There are actually two eLBOW GUI available in the Ligands menu. The first is a wizard-like interface that asks questions about the available inputs and the desired outcomes. The first screen asks for the basic inputs. These are explained in the Input section.

Above: The first screen of the wizard-like eLBOW GUI with the Chemical component parameter selected.

Once the basic inputs are provided the possible geometry optimisations are presented for selection.

Above: The second screen of the wizard-like eLBOW GUI with the AM1 optimisation parameter selected.

The final screen is customised to show you the parameters that apply to your choices. You can make some more changes and "Run".

Above: Final screen of the wizard-like eLBOW GUI.

An advance GUI for expert or those who wish to use some of the more complicated parameters is available.

Above: data entry window for eLBOW in the Phenix GUI. Input parameters are grouped separately at top.

Input files

eLBOW has extremely flexible options for describing ligand information. The only mandatory data are some description of the molecular topology, i.e. atomic composition and (implicit or explicit) bond connectivity. Four different sources of information may be used:

SMILES string: this is a simplified syntax for uniquely representing molecules as text strings, invented by Daylight CIS. It is often found as part of various internal or public databases. This is usually the most reliable form of information to use, if available. (You can also provide it as a file, in the "Chemical file" input.) Chemical file: this is either a file containing a single SMILES string, or one of a number of different formats, such as .mol, .sdf, or TRIPOS MOL2 files, or some CIF files. Most of these are at least as good as SMILES strings, although some CIFs are incomplete and less suitable for use in eLBOW. Chemical Components code: this is a three-letter residue code taken from the PDB Chemical Component Dictionary. We distribute this database (in CIF format) with Phenix; for all but a handful of molecules, the available data include a SMILES string, and often the geometry as well. If you are using a ligand already present in the PDB and know the code, this is probably the fastest way to run eLBOW. Geometry file: this is usually a PDB file, although other formats are possible. Although the geometry file may be useful to tell eLBOW the expected output geometry, it is often less ideal as a description of topology. We recommend using one of the other sources of information as the primary input if possible. The PDB file does not necessarily need to be limited to the ligand of interest; use the input field labeled "Residue selection in PDB file" to select the three-letter code to use. (The drop-down menu will automatically be populated with unrecognized residues.)

- **SMILES string:** this is a simplified syntax for uniquely representing molecules as text strings, invented by Daylight CIS. It is often found as part of various internal or public databases. This is usually the most reliable form of information to use, if available. (You can also provide it as a file, in the "Chemical file" input.)
- **Chemical file:** this is either a file containing a single SMILES string, or one of a number of different formats, such as .mol, .sdf, or TRIPOS MOL2 files, or some CIF files. Most of these are at least as good as SMILES strings, although some CIFs are incomplete and less suitable for use in eLBOW.
- **Chemical Components code:** this is a three-letter residue code taken from the PDB Chemical Component Dictionary. We distribute this database (in CIF format) with Phenix; for all but a handful of molecules, the available data include a SMILES string, and often the geometry as well. If you are using a ligand already present in the PDB and know the code, this is probably the fastest way to run eLBOW.
- **Geometry file:** this is usually a PDB file, although other formats are possible. Although the geometry file may be useful to tell eLBOW the expected output geometry, it is often less ideal as a description of topology. We recommend using one of the other sources of information as the primary input if possible. The PDB file does not necessarily need to be limited to the ligand of interest; use the input field labeled "Residue selection in PDB file" to select the three-letter code to use. (The drop-down menu will automatically be populated with unrecognized residues.)

Only one source of topology information may be provided, but you may also provide the geometry file in addition to one of the first three inputs, to guide eLBOW in constructing the resulting model. You will need to change the drop-down menu labeled "Use geometry for:" to one of the other three options if you do this. The geometry is especially useful if you already have placed the ligand in your structure and now need to refine it.

A final source of information is a PDB file containing the template atom names; the geometry and topology information will be discarded. If this is not provided, Phenix will generate atom names for the output model and CIF file automatically.

Output files

By default, eLBOW will generate a PDB file and a CIF file, suitable for use in any other Phenix app (or third-party software). Additional output formats are available and may be selected by clicking the "Output. . ." button. You may also manipulate other details such as chirality, Cis/Trans, ring pucker and PDB file attributes (including ligand code if you are starting from a SMILES string).

Above: the "Output. . ." dialog.

Above: the "Output options. . ." dialog.

What to do next

We recommend opening the output CIF file in REEL to make sure that the results are what you expect. The CIF may be used in any applications requiring restraints as input (primarily phenix.refine, the validation tools, and the AutoBuild and LigandFit wizards). If you have not already placed your ligand in the model, the PDB file is suitable for this, but we recommend trying LigandFit first to save time.

References

electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. Acta Crystallogr D Biol Crystallogr 65, 1074-80 (2009).

- electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 65, 1074-80 (2009).

Calculating Fragment Antigen-Binding (Fab) elbow angle

fab_elbow_angle.html

Overview

Each FAB is made of two chains, Heavy (H) and Light (L), and two domains, Variable (V) and Constant (C or C1, The heavy domain has three parts). The angle between the variable and constant domains is the FAB elbow angle. It is the angle between rotation axes, the pseudo-dyad axes, defined by aligning the light portion of each domain, on-to the heavy one.

Command line options:

- light: chain ID of light domain, by default set to L
- heavy: chain ID of heavy domain, by default set to H
- limit_l: the number of the residue separating between variable and constant domains in the light chain, by default 107
- limit_h: the number of the residue separating between variable and constant domains in the heavy chain, by default 113
- h: help

Usage:

```
>>>phenix.fab_elbow_angle fab_name.pdb [light=L] [heavy=H] [limit_l=107] [limit_h=113]
```

Examples:

```
>>>phenix.fab_elbow_angle 7fab.pdb light=L heavy=H limit_l=104 limit_h=117
```

```
123.00
```

```
>>>phenix.fab_elbow_angle 1bbd.pdb
```

```
126.34
```

Finding all ligands in a map with phenix.find_all_ligands

find_all_ligands.html

Author(s)

- phenix.find_all_ligands: Tom Terwilliger

Purpose

phenix.find_all_ligands is a command line tool for finding all the ligands in a map by repetitively running phenix.ligandfit with a series of ligands and choosing the best-fitting one at each cycle.

Usage

How phenix.find_all_ligands works:

The basic procedure for phenix.find_all_ligands has three steps. The first is to identify the largest contiguous region of density in your map that is not already occupied by your model or previously-fitted ligands. The second is to fit each ligand (you identify the candidate ligands in advance) into this density. The third is to choose the one that fits the density the best. Then the best-fitting ligand is added to the structure and the process is repeated until the number of ligands you request is found or the correlation of ligand to the map drops below the value you specify (default=0.5).

Output files from phenix.find_all_ligands

The output ligand files from phenix.find_all_ligands are normally in the temporary directory (default='temp_dir'). They will be files with names such as "SITE_1_ATP.pdb" for the placement of ATP in the first site fitted.

Examples

Standard run of phenix.find_all_ligands:

Running phenix.find_all_ligands is easy. Usually you will want to edit a small parameter file (find_all_ligands.eff) to contain your commands like this, where the ligandfit commands are sent to phenix.ligandfit: for the actual fitting and the find_all_ligands commands determine what searches are done: type:

```
# commands for running phenix.find_all_ligands
find_all_ligands {
  number_of_ligands = 5
  cc_min = 0.5
  ligand_list = ATP.pdb NAD.pdb
  nproc = 2
}
ligandfit {
  data = "nsf-d2.mtz"
  model = "nsf-d2_noligand.pdb"
  lig_map_type = fo-fc_difference_map
}
```

You might also want to add to this some additional commands for phenix.ligandfit. Any commands for ligandfit are allowed, except that the commands "ligand" and "input_lig_file" are ignored as the input ligand comes from the find_all_ligands command "ligand_list":

```
# find_all_ligands.eff more commands for ligandfit
ligandfit {
  data = "nsf-d2.mtz"
  model = "nsf-d2_noligand.pdb"
  lig_map_type = fo-fc_difference_map
  ligand_cc_min = 0.75
  verbose = Yes
}
```

where you can put any phenix.ligandfit commands in the braces.

Then you can run this with the command:

```
phenix.find_all_ligands find_all_ligands.eff
```

Possible Problems

Specific limitations and problems:

- This method uses phenix.ligandfit to do the ligand fitting, so all the commands, features, and limitations of phenix.ligandfit apply to phenix.find_all_ligands.

Literature

Additional information

NOTE: in addition to the find_all_ligands keywords shown here, all phenix.ligandfit commands are also allowed, except that the commands "ligand" and "input_lig_file" are ignored as the input ligand comes from the find_all_ligands.

List of all available keywords

find_all_ligands
number_of_ligands = None Total number of ligand sites. Ignored if "None". find_all_ligands will keep looking until the correlation coefficient for the fit of the best ligand is less than cc_min or the number of ligands placed is number_of_ligands, whichever comes first
cc_min = 0.50 Ignored if "None". find_all_ligands will keep looking until the correlation coefficient for the fit of the best ligand is less than cc_min or the number of ligands placed is number_of_ligands, whichever comes first
ligand_list = None List of files with ligands to find
nproc = 1 number of processors to use
background = True run jobs in background or not (if nproc is greater than 1)
run_command = "sh " Command for running jobs (e.g., sh or qsub)
verbose = False verbose output
raise_sorry = False Raise sorry if problems
debug = False debugging output
temp_dir = Auto Optional temporary work directory
output_dir = "" Output directory where files are to be written
dry_run = False Just read in and check parameter names
check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end
wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....
output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
ligandfitdata = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The

Wizard will guess the column identification. If you want to specify it you can say `input_labels="FP"`, or `input_labels="FP PHIB FOM"`. `ligand = None` Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)
`model = None` PDB file with model for everything but the ligand
`quick = False` Set to True for running as quickly as possible.
`crystal_infounit_cell = None` Enter cell parameter a b c alpha beta gamma
`resolution = 0` High-resolution limit. Zero means keep everything. If `map_in` is specified, resolution must be given.
`space_group = None` Space Group symbol (i.e., C2221 or C 2 2 21)
`file_info` file_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands
`input_labels = None` Labels for input data columns
`lig_map_type = fo-fc_difference_map`
`fobs_map *pre_calculated_map_coeffs` Enter the type of map to use in ligand fitting
`fo-fc_difference_map`: Fo-Fc difference map phased on partial model (requires FOBS in your input file)
`fobs_map`: Fo map phased on partial model (requires FOBS in your input file)
`pre_calculated_map_coeffs`: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at `pre_calculated_map_coeffs`. If your labels choice is not available and '---' is present, you can choose '---'.
`ligand_format = *PDB SMILES` Enter whether the files contain SMILES strings or PDB formatted information
`output_file` target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
`input_file` existing_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration.
`ligand_start = None` LigandFit will attempt to put your ligand superimposing on `ligand_start` if supplied. This must have some of the same atoms...

- `find_all_ligands` number_of_ligands = None Total number of ligand sites. Ignored if "None". `find_all_ligands` will keep looking until the correlation coefficient for the fit of the best ligand is less than `cc_min` or the number of ligands placed is `number_of_ligands`, whichever comes first
`cc_min = 0.50` Ignored if "None". `find_all_ligands` will keep looking until the correlation coefficient for the fit of the best ligand is less than `cc_min` or the number of ligands placed is `number_of_ligands`, whichever comes first
`ligand_list = None` List of files with ligands to find
`nproc = 1` number of processors to use
`background = True` run jobs in background or not (if `nproc` is greater than 1)
`run_command = "sh "` Command for running jobs (e.g., sh or qsub)
`verbose = False` verbose output
`raise_sorry = False` Raise sorry if problems
`debug = False` debugging output
`temp_dir = Auto` Optional temporary work directory
`output_dir = ""` Output directory where files are to be written
`dry_run = False` Just read in and check parameter names
`check_wait_time = 1.0` You can specify the length of time (seconds) to wait between checking for subprocesses to end
`wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....
`output_file` target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `number_of_ligands = None` Total number of ligand sites. Ignored if "None". `find_all_ligands` will keep looking until the correlation coefficient for the fit of the best ligand is less than `cc_min` or the number of ligands placed is `number_of_ligands`, whichever comes first
- `cc_min = 0.50` Ignored if "None". `find_all_ligands` will keep looking until the correlation coefficient for the fit of the best ligand is less than `cc_min` or the number of ligands placed is `number_of_ligands`, whichever comes first
- `ligand_list = None` List of files with ligands to find
- `nproc = 1` number of processors to use

- background = True run jobs in background or not (if nproc is greater than 1)
- run_command = "sh " Command for running jobs (e.g., sh or qsub)
- verbose = False verbose output
- raise_sorry = False Raise sorry if problems
- debug = False debugging output
- temp_dir = Auto Optional temporary work directory
- output_dir = "" Output directory where files are to be written
- dry_run = False Just read in and check parameter names
- check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end
- wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- ligandfitdata = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" , or input_labels="FP PHIB FOM". ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)model = None PDB file with model for everything but the ligand quick = False Set to True for running as quickly as possible. crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.space_group = None Space Group symbol (i.e., C2221 or C 2 2 21) file_infofile_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting
fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file)
fobs_map: Fo map phased on partial model (requires FOBS in your input file)
pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.input_filesexisting_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match

exactly)ncs_in = None You can supply a file with NCS information for use with
 ligands_from_ncsinput_ligand_compare_file = None If you enter a PDB file with a ligand in it, the
 coordinates of the newly-built ligand will be compared with the coordinates in this file. cif_def_file_list =
 None You can supply cif files for real-space refinement after fittingrefinement_file = None You can supply
 a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space
 refinement a round of full refinement will be carried out with phenix.refinefobs_labels = None Labels for
 Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner_free_label = None
 Label for FreeR_flag in refinement_file...same format as for phenix.refinemap_in = None Map file
 (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before
 use.search_parametersfixed_ligand = False Use fixed ligand (no rotations of any bonds) if setconformers
 = 1 Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate
 them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify
 allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW
 takes longer but is more accurate. group_search = 0 Enter the ID number of the group from the ligand to
 use to seed the search for conformations ligand_cc_min = 0.75 Ente...

- data = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" , or input_labels="FP PHIB FOM";.
- ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)
- model = None PDB file with model for everything but the ligand
- quick = False Set to True for running as quickly as possible.
- crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)
- unit_cell = None Enter cell parameter a b c alpha beta gamma
- resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.
- space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)
- file_infofile_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting
 fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file)
 fobs_map: Fo map phased on partial model (requires FOBS in your input file)
 pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- file_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands
- input_labels = None Labels for input data columns
- lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS

in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file)
pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.

- ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- input_fileexisting_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match exactly)ncs_in = None You can supply a file with NCS information for use with ligands_from_ncsinput_ligand_compare_file = None If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file. cif_def_file_list = None You can supply CIF files for real-space refinement after fittingrefinement_file = None You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refinefobs_labels = None Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner_free_label = None Label for FreeR_flag in refinement_file...same format as for phenix.refinemap_in = None Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.
- existing_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration.
- ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match exactly)
- ncs_in = None You can supply a file with NCS information for use with ligands_from_ncs
- input_ligand_compare_file = None If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file.
- cif_def_file_list = None You can supply CIF files for real-space refinement after fitting
- refinement_file = None You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refine
- fobs_labels = None Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refine
- r_free_label = None Label for FreeR_flag in refinement_file...same format as for phenix.refine
- map_in = None Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.

- `search_parameters.fixed_ligand = False` Use fixed ligand (no rotations of any bonds) if `setconformers = 1`. Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate.
- `group_search = 0` Enter the ID number of the group from the ligand to use to seed the search for conformations
- `ligand_cc_min = 0.75` Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations
- `ligand_completeness_min = 1` Enter the minimum completeness of the ligand to the map to quit searching for more conformations
- `local_search = True` If `local_search` is True then, only the region within `search_dist` of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- `search_dist = 10` If `local_search` is True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- `use_cc_local = False` You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify `use_cc_local`, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$.
- `ligands_from_ncs = False` You can try to use ncs (from your partial model file or from your `ncs_in` file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand.
- `max_ligands_from_ncs = 1` You can specify how many of the ligands already found to consider using NCS (usually 1)
- `n_group_search = 3` Enter the number of different fragments of the ligand that will be looked for in FFT search of the map
- `n_indiv_tries_max = 10` If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified
- `n_indiv_tries_min = 5` If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified
- `number_of_ligands = 1` Number of copies of the ligand expected in the asymmetric unit
- `offsets_list = 7 53 29` You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models.
- `refine_ligand = True` You can carry out real-space refinement on the ligand after fitting
- `ligand_occupancy = 1.0` You can set the initial occupancy of the ligand
- `real_space_target_weight = 10` You can carry change the weight on the real-space term in real-space refinement on the ligand after fitting
- `fittingParameters.tracing_ligand.delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments
- `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds
- `fit_phi_range = -180 180` Range of bond rotation angles to search

- `fixed_ligand = False` Use fixed ligand (no rotations of any bonds) if set

- `conformers = 1` Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate.

- `group_search = 0` Enter the ID number of the group from the ligand to use to seed the search for conformations

- `ligand_cc_min = 0.75` Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations

- `ligand_completeness_min = 1` Enter the minimum completeness of the ligand to the map to quit searching for more conformations

- `local_search = True` If `local_search` is True then, only the region within `search_dist` of the point in the map with the highest local rmsd will be searched in the FFT search for fragments

- `search_dist = 10` If `local_search` is True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- `use_cc_local = False` You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify `use_cc_local`, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$.
- `ligands_from_ncs = False` You can try to use ncs (from your partial model file or from your `ncs_in` file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand.
- `max_ligands_from_ncs = 1` You can specify how many of the ligands already found to consider using NCS (usually 1)
- `n_group_search = 3` Enter the number of different fragments of the ligand that will be looked for in FFT search of the map
- `n_indiv_tries_max = 10` If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified
- `n_indiv_tries_min = 5` If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified
- `number_of_ligands = 1` Number of copies of the ligand expected in the asymmetric unit
- `offsets_list = 7 53 29` You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models.
- `refine_ligand = True` You can carry out real-space refinement on the ligand after fitting
- `ligand_occupancy = 1.0` You can set the initial occupancy of the ligand
- `real_space_target_weight = 10`. You can carry change the weight on the real-space term in real-space refinement on the ligand after fitting
- `fittingParameters` for tracing ligand `delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds `fit_phi_range = -180 180` Range of bond rotation angles to search
- `delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments
- `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds
- `fit_phi_range = -180 180` Range of bond rotation angles to search
- `search_targetligand_near_chain = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. If you set `ligand_near_chain="None"` or leave it blank or do not set it, then all chains will be included. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line). `ligand_near_res = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain` The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line). `ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`. `name_of_ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name

with name_of_ligand_near_pdb. search_center = 0.0 0.0 0.0 Enter coordinates for center of search region (ignored if [0.,0.,0.])

- ligand_near_chain = None You can specify where to search for the ligand either with search_center or with ligand_near_res and ligand_near_chain. If you set ligand_near_chain="None" or leave it blank or do not set it, then all chains will be included. The keywords ligand_near_res and ligand_near_chain refer to residue/chain in the file defined by input_partial_model_file (or model if running from command line).
- ligand_near_res = None You can specify where to search for the ligand either with search_center or with ligand_near_res and ligand_near_chain The keywords ligand_near_res and ligand_near_chain refer to residue/chain in the file defined by input_partial_model_file (or model if running from command line).
- ligand_near_pdb = None You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with name_of_ligand_near_pdb.
- name_of_ligand_near_pdb = None You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with name_of_ligand_near_pdb.
- search_center = 0.0 0.0 0.0 Enter coordinates for center of search region (ignored if [0.,0.,0.])
- generalextend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch. ligand_id = None You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in input_lig_file has. The keyword is only valid if a single copy of the ligand is to be found. nbatch = 5 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch. nproc = 1 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch. If you set nproc=Auto and your machine has n processors, then it will use n-1 processors, or 1 if only 1 is available. resolve_command_list = None Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: coarse_grid resolution 200 2.0 hkl in test.mtz . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes. coot_name = "coot" If your version of coot is called something else, then you can specify that here. i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screen. background = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end. max_wait_time = 1.0 You can

specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'. `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....`cache_resolve_libs = True` Use caching of resolve libraries to speed up resolve. `resolve_size = 12` Size for solve/resolve ('<small>giant</small>', '<small>huge</small>', '<small>extra_huge</small>' or a number where 12=<small>giant</small> 18=<small>huge</small> `check_run_command = False` You can have the wizard check your run command at startup. `run_command = "sh "` When you specify `nproc=nn`, you can run the subprocesses as jobs in background with sh (default...

- `extend_try_list = True` You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with `nbatch`.
- `ligand_id = None` You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in `input_lig_file` has. The keyword is only valid if a single copy of the ligand is to be found.
- `nbatch = 5` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`.
- `nproc = 1` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`. If you set `nproc=Auto` and your machine has `n` processors, then it will use `n-1` processors, or 1 if only 1 is available
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: `coarse_grid resolution 200 2.0 hklin test.mtz` . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes.
- `coot_name = "coot"` If your version of coot is called something else, then you can specify that here.
- `i_ran_seed = 72432` Random seed (positive integer) for model-building and simulated annealing refinement
- `raise_sorry = False` You can have any failure end with a Sorry instead of simply printout to the screen
- `background = True` When you specify `nproc=nn`, you can run the jobs in background (default if `nproc` is greater than 1) or foreground (default if `nproc=1`). If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If you use `run_command='sh '` (or similar, sh is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `check_wait_time = 1.0` You can specify the length of time (seconds) to wait between checking for subprocesses to end

- `max_wait_time = 1.0` You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....
- `cache_resolve_libs = True` Use caching of resolve libraries to speed up resolve
- `resolve_size = 12` Size for solve/resolve ('giant', 'huge', 'extra_huge' or a number where 12=giant 18=huge)
- `check_run_command = False` You can have the wizard check your run command at startup
- `run_command = "sh "` When you specify `nproc=nn`, you can run the subprocesses as jobs in background with `sh` (default) or submit them to a queue with the command of your choice (i.e., `qsub`). If you have a multi-processor machine, use `sh`. If you have a cluster, use `qsub` or the equivalent command for your system. NOTE: If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If `nproc` is greater than 1 and you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `queue_commands = None` You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: `queue_commands='#PBS -N mr_rosetta'`
`queue_commands='#PBS -j oe'` `queue_commands='#PBS -l walltime=03:00:00'`
`queue_commands='#PBS -l nodes=1:ppn=1'` NOTE: you can put in the characters '<path>' in any `queue_commands` line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_', up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1'
- `condor_universe = vanilla` The universe for condor is usually vanilla. However you might need to set it to local for your cluster
- `add_double_quotes_in_condor = True` You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.
- `condor = None` Specifies if the `group_run_command` is submitting a job to a condor cluster. Set by default to True if `group_run_command=condor_submit`, otherwise False. For condor job submission `mr_rosetta` uses a customized script with condor commands. Also uses `one_subprocess_level=True`
- `last_process_is_local = True` If true, run the last process in a group in background with `sh` as part of the job that is submitting jobs. This prevents having the job that is submitting jobs sit and wait for all the others while doing nothing
- `skip_r_factor = False` You can skip R-factor calculation if refinement is not done and `maps_only=True`
- `test_flag_value = Auto` Normally leave this at Auto (default). This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with `test_flag_value`.

- skip_xtriage = False You can bypass xtriage if you want. This will prevent you from applying anisotropy corrections, however.
- base_path = None You can specify the base path for files (default is current working directory)
- temp_dir = None Define a temporary directory
- local_temp_directory = None Write all temporary files to this directory and copy files specified by keep_files back to TEMP directory in working directory at the end of run. Used to speed up read/write operations. NOTE: Deletes all files except those specified by keep_files just like clean_up=True.
- clean_up = None At the end of the entire run the TEMP directories will be removed if clean_up is True. Also all files except for overall_best_xxx (final) files and .eff files In the AutoSol_run_xx_AutoBuild_run_xx_ etc directory will be removed Files listed in keep_files will not be deleted. If you want to remove files after your run is finished use a command like `"phenix.autobuild run=1 clean_up=True"`;
- print_citations = True Print citations at end of run
- solution_output_pickle_file = None At end of run, write solutions to this file in output directory if defined
- job_title = None Job title in PHENIX GUI, not used on command line
- top_output_dir = None This is used in subprocess calls of wizards and to tell the Wizard where to look for the STOPWIZARD file.
- wizard_directory_number = None This is used by the GUI to define the run number for Wizards. It is the same as desired_run_number NOTE: this value can only be specified on the command line, as the directory number is set before parameters files are read.
- verbose = False Command files and other verbose output will be printed
- extra_verbose = False Facts and possible commands will be printed every cycle if True
- debug = False You can have the wizard stop with error messages about the code if you use debug. Additionally the output goes to the terminal if you specify `"debug=True"`;
- require_nonzero = True Require non-zero values in data columns to consider reading in.
- remove_path_word_list = None List of words identifying paths to remove from PATH These can be used to shorten your PATH. For example... `cns ccp4 coot` would remove all paths containing these words except those also containing `phenix`. Capitalization is ignored.
- fill = False Fill in all missing reflections to resolution `res_fill`. Applies to density modified maps. See also `filled_2fofc_maps` in autobuild.
- res_fill = None Resolution for filling in missing data (default = highest resolution of any datafile). Only applies to density modified maps. Default is fill to high resolution of data. Ignored if `fill=False`
- check_only = False Just read in and check initial parameters. Not for general use
- keep_files = `ligandfit*.pdb` List of files that are not to be cleaned up. wildcards permitted
- displaynumber_of_solutions_to_display = None Number of solutions to put on screen and to write out
 `solution_to_display = 1` Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- number_of_solutions_to_display = None Number of solutions to put on screen and to write out
- solution_to_display = 1 Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- run_controlignore_blanks = None `ignore_blanks` allows you to have a command-line keyword with a blank value like `"input_lig_file_list="`; `stop = None` You can stop the current wizard with

"stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild. display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be shown. display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown. carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off. run = None Set run to n to continue with run n where you left off. copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off. display_runs = None List all runs for this wizard. delete_runs = None List runs to delete: 1 2 3-5 9:12 display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-" # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol and AutoBuild like this: input_refinement_labels="FP SIGFP FreeR_flags" # each individual label counts dry_run = False Just read in and check parameter names params_only = False Just read in and return parameter defaults display_all = False Just read in and display parameter defaults coot = None Not presently applicable for ligandfit

- ignore_blanks = None ignore_blanks allows you to have a command-line keyword with a blank value like "input_lig_file_list="
- stop = None You can stop the current wizard with "stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild.
- display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be shown.
- display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown.
- carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off.
- run = None Set run to n to continue with run n where you left off.
- copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off.
- display_runs = None List all runs for this wizard.
- delete_runs = None List runs to delete: 1 2 3-5 9:12
- display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-" # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol and AutoBuild like this: input_refinement_labels="FP SIGFP FreeR_flags" # each individual label counts
- dry_run = False Just read in and check parameter names
- params_only = False Just read in and return parameter defaults
- display_all = False Just read in and display parameter defaults
- coot = None Not presently applicable for ligandfit
- special_keywords write_run_directory_to_file = None Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.

- `write_run_directory_to_file = None` Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.
- `non_user_parameters` These are obsolete parameters and parameters that the wizards use to communicate among themselves. Not normally for general use.
- `gui_output_dir = None` Used only by the GUI
- `sg = None` Obsolete. Use `space_group` instead
- `get_lig_volume = False` You can ask to get the volume of the ligand and to then stop
- `input_data_file = None` Not normally used. Use `"data"` instead
- `input_lig_file = None` Not normally used. Use `"ligand"` instead.
- `ligand_code = None` Not normally used. Use `"ligand"` instead.
- `input_partial_model_file = None` Not normally used. Use `"model"` instead
- `cif_already_generated = False` You can specify that the ligand cif file is already generated

Guided Ligand Replacement

guided_ligand_replacement.html

Contents

- Overview
- Usage
- Input files
- Output files
- Examples Example 1 Example 2
- Example 1
- Example 2
- References
- List of all available keywords

Authors: Nigel W. Moriarty and Herbert E. Klei

Overview

The Guided Ligand Replacement (GLR) module is intended to facilitate ligand placement when the X-ray crystal structure of at least one similar protein:ligand complex is known. GLR is often employed in structure-assisted drug design where the structures of many related protein:inhibitor complexes are determined. It is especially useful for complicated ligands and when there are multiple independent copies in the asymmetric unit.

The Guided Ligand Replacement (GLR) tool is designed to use prior knowledge from a previously fit ligand to expedite the fitting of a "similar" ligand into the same or "similar" protein. The similarity required between the pair of ligands can be as little as a few atoms.

The proteins are overlaid using `phenix.superpose_pdbs`. The ligands are matched using the electronic Ligand Builder and Optimization Workbench (eLBOW) via another tool called `phenix.superpose_ligands`. Once the ligand has been fit, a Real-Space Refinement (RSR) is performed and the final model written to disk.

Usage

Usage

```
phenix.guided_ligand_replacement [options] \  
  target_pdb_file_name=structure-in-which-target-ligand-is-to-be-placed.pdb \  
  ligand_cif_file_name=parameterization-of-target-ligand.cif \  
  map_coeffs_file_name=map-coefficients-for-target-electron-density.mtz \  
  guide_pdb_file_name=structure-of-reference-protein-ligand-complex.pdb \  
  ligand_selection_in_guide_model=selection-to-identify-reference-ligand \  
  replace_ligand_in_guide_pdb_file_name=[true|false]
```

Input files

GLR requires a number of inputs and also has some options, all of which are provided using the phil formalism. The list of inputs:

guide_pdb_file_name: Name of the PDB file for the structure of the reference protein:ligand complex. While the protein used in the target and reference structures is usually the same, the protein sequences need only be sufficiently similar to reliably superimpose. protein_pdb_file_name: Name of PDB file in which the target (i.e. new) ligand is to be placed. This structure is typically the apo structure from molecular replacement after one round of refinement. map_coeffs_file_name: Name of the map coefficient file for the electron density in which the target ligand is to be placed. This file is typically the MTZ file from latest run of phenix.refine. If this file contains more than one set of possible map coefficients, the first recognized set is used. If structure factors are provided (e.g. MTZ file with only FOBS and SIGFOBS), the 2Fo-Fc map is calculated and used. Map files (e.g. .map) are used directly.

- guide_pdb_file_name: Name of the PDB file for the structure of the reference protein:ligand complex. While the protein used in the target and reference structures is usually the same, the protein sequences need only be sufficiently similar to reliably superimpose.
- protein_pdb_file_name: Name of PDB file in which the target (i.e. new) ligand is to be placed. This structure is typically the apo structure from molecular replacement after one round of refinement.
- map_coeffs_file_name: Name of the map coefficient file for the electron density in which the target ligand is to be placed. This file is typically the MTZ file from latest run of phenix.refine. If this file contains more than one set of possible map coefficients, the first recognized set is used. If structure factors are provided (e.g. MTZ file with only FOBS and SIGFOBS), the 2Fo-Fc map is calculated and used. Map files (e.g. .map) are used directly.

The input for the ligand can be any input file that is readable by eLBOW. Either or both of the following phil parameters can be used.

ligand_input_file_name: this is a file containing chemical information about the ligand. ligand_cif_file_name: this is the of a ligand restraints file for the RSR step. If not supplied, the restraints are generated from ligand_input_file_name. ligand_code: the ligand code from the PDB Chemical Components so that restraints and geometry can be generated.

- ligand_input_file_name: this is a file containing chemical information about the ligand.
- ligand_cif_file_name: this is the of a ligand restraints file for the RSR step. If not supplied, the restraints are generated from ligand_input_file_name.
- ligand_code: the ligand code from the PDB Chemical Components so that restraints and geometry can be generated.

Optional inputs include:

ligand_selection_in_guide_model: Standard PHENIX selection expression to identify the reference ligand in the reference structure. A typical selection might look like "resname LG1 and resid 1". While GLR tries to make a sensible selection based on ligand code or uniqueness, it may be necessary to make an explicit selection. replace_ligand_in_guide_pdb_file_name=[true|false]: By default, this parameter is set to false and can be omitted. If this parameter is set to true, the reference structure is also used as the target structure (protein_pdb_file_name parameter is ignored). This functionality is useful in two circumstances: 1) the first copy of the ligand is placed manually and it needs to be propagated throughout the asymmetric unit 2) the current ligand is incorrect (e.g. GTP instead of ATP) or needs to be updated (e.g. remove non-standard atom names). phenix.superpose_pdbs: this allows selection of the same parameters as in the standalone program.

- ligand_selection_in_guide_model: Standard PHENIX selection expression to identify the reference ligand in the reference structure. A typical selection might look like "resname LG1 and resid 1". While GLR tries to make a sensible selection based on ligand code or uniqueness, it may be necessary to make an explicit selection.

ligand_selection_in_guide_model: Standard PHENIX selection expression to identify the reference ligand in the reference structure. A typical selection might look like "resname LG1 and resid 1". While GLR tries to make a sensible selection based on ligand code or uniqueness, it may be necessary to make an explicit selection.

- replace_ligand_in_guide_pdb_file_name=[true|false]: By default, this parameter is set to false and can be omitted. If this parameter is set to true, the reference structure is also used as the target structure (protein_pdb_file_name parameter is ignored). This functionality is useful in two circumstances: 1) the first copy of the ligand is placed manually and it needs to be propagated throughout the asymmetric unit 2) the current ligand is incorrect (e.g. GTP instead of ATP) or needs to be updated (e.g. remove non-standard atom names).

replace_ligand_in_guide_pdb_file_name=[true|false]: By default, this parameter is set to false and can be omitted. If this parameter is set to true, the reference structure is also used as the target structure (protein_pdb_file_name parameter is ignored). This functionality is useful in two circumstances:

1) the first copy of the ligand is placed manually and it needs to be propagated throughout the asymmetric unit

2) the current ligand is incorrect (e.g. GTP instead of ATP) or needs to be updated (e.g. remove non-standard atom names).

- phenix.superpose_pdbs: this allows selection of the same parameters as in the standalone program.

phenix.superpose_pdbs: this allows selection of the same parameters as in the standalone program.

Output files

By default, GLR will generate a PDB file with the fit ligand.

Examples

Example 1

The reference FXa structure, 3KQE, and its reference ligand, LGM, is used to place the target ligand, LGK, in the target FXa structure 3KQC

```
phenix.guided_ligand_replacement \
target_pdb_file_name=3KQC-apo.pdb \
ligand_cif_file_name=LGK.cif \
map_coeffs_file_name=refine-apo.mtz \
guide_pdb_file_name=3KQE.pdb \
ligand_selection_in_guide_model="resname LGM"
```

Example 2

In all of the above examples, the input can be provided through a PHIL file.

```
phenix.guided_ligand_replacement < glr.phil > glr.log
```

References

Ligand placement based on prior structures: the guided ligand-replacement method. H.E. Klei, N.W. Moriarty, N. Echols, T.C. Terwilliger, E.T. Baldwin, M. Pokross, S. Posy, and P.D. Adams. Acta Crystallogr D Biol Crystallogr 70, 134-43 (2014).

- Ligand placement based on prior structures: the guided ligand-replacement method. H.E. Klei, N.W. Moriarty, N. Echols, T.C. Terwilliger, E.T. Baldwin, M. Pokross, S. Posy, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 70, 134-43 (2014).

electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 65, 1074-80 (2009).

- electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 65, 1074-80 (2009).

List of all available keywords

guided_ligand_replacementjob_title = None Job title in PHENIX GUI, not used on command
 lineinputguide_pdb_file_name = None PDB filename containing protein model and docked ligand that can
 guide the placement of new ligand into the an apo modelprotein_pdb_file_name = None PDB file containing
 protein into which the ligand will be fit.ligand_input_file_name = None File containing chemical input of a
 ligandligand_code = None Ligand code from Chemical Componentsdata_file_name = None Amplitudes or
 intensitiessmap_coeffs_file_name = None Map coefficients from a refinement of the apo
 modelligand_cif_file_name = None This restraints file will be used for the real space refinement. If not supplied,
 a set of restraints are generated from the ligand input.ligand_selection_in_guide_model = None The specific
 ligand to use from the guide structure, using the standard Phenix atom selection
 syntax.real_space_refinement = *ideal off Perform a real space refinement after each ligand is
 addedreal_space_refinement_selection_radius = 5. replace_ligand_in_guide_pdb_file_name = False Use this
 option to replace the ligand in the Guide protein. This option saves creating a apo model of the same
 protein.map_labels = None outputfile_name = None Defaults to current directoryjob_title = None Job title in
 PHENIX GUI, not used on command linesuperpose_pdbsinputpdb_file_name_protein = None Name of PDB
 file with model to fit topdb_file_name_guide = None Name of PDB file with model that will be fit to
 pdb_file_name_proteinpdb_file_name_guide_other = None Name of another PDB file to transform with the
 same matrix as pdb_file_name_guidealignmentSet of parameters for sequence alignment. Defaults are good
 for most of casesalignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1
 similarity_matrix = blosum50 dayhoff *identity validationrmsd_cutoff = 2. similarity_cutoff = 80.
 automatic_guidanceuse_web_services = False Use the homology search in RCSB web services to identify
 and download candidate guide models.use_local_directory = None A directory containing models to search
 for similar proteins to use as a guide for fitting a similar ligand, similarly
 bound.minimum_ligand_similarity_tanimoto = 0.7 Minimum Tanimoto score to filter possible guide
 models.process_controlnproc = 1 superpose_ligandsquick = True

- guided_ligand_replacementjob_title = None Job title in PHENIX GUI, not used on command
 lineinputguide_pdb_file_name = None PDB filename containing protein model and docked ligand that can
 guide the placement of new ligand into the an apo modelprotein_pdb_file_name = None PDB file
 containing protein into which the ligand will be fit.ligand_input_file_name = None File containing chemical
 input of a ligandligand_code = None Ligand code from Chemical Componentsdata_file_name = None
 Amplitudes or intensitiessmap_coeffs_file_name = None Map coefficients from a refinement of the apo
 modelligand_cif_file_name = None This restraints file will be used for the real space refinement. If not
 supplied, a set of restraints are generated from the ligand input.ligand_selection_in_guide_model = None
 The specific ligand to use from the guide structure, using the standard Phenix atom selection
 syntax.real_space_refinement = *ideal off Perform a real space refinement after each ligand is
 addedreal_space_refinement_selection_radius = 5. replace_ligand_in_guide_pdb_file_name = False

Use this option to replace the ligand in the Guide protein. This option saves creating a apo model of the same protein.
`map_labels = None` Defaults to current directory
`job_title = None` Job title in PHENIX GUI, not used on command line
`superpose_pdbsinputpdb_file_name_protein = None` Name of PDB file with model to fit to
`superpose_pdbsininputpdb_file_name_guide = None` Name of PDB file with model that will be fit to
`superpose_pdbsininputpdb_file_name_guide_other = None` Name of another PDB file to transform with the same matrix as `superpose_pdbsininputpdb_file_name_guide`
`alignmentSet` of parameters for sequence alignment. Defaults are good for most of cases
`alignment_style = local` *`global`
`gap_opening_penalty = 1`
`gap_extension_penalty = 1`
`similarity_matrix = blosum50` dayhoff *identity validation
`rmsd_cutoff = 2`.
`similarity_cutoff = 80`.
`automatic_guidance`
`use_web_services = False` Use the homology search in RCSB web services to identify and download candidate guide models.
`use_local_directory = None` A directory containing models to search for similar proteins to use as a guide for fitting a similar ligand, similarly bound.
`minimum_ligand_similarity_tanimoto = 0.7` Minimum Tanimoto score to filter possible guide models.
`process_controlnproc = 1`
`superpose_ligandsquick = True`

- `job_title = None` Job title in PHENIX GUI, not used on command line
- `inputguide_pdb_file_name = None` PDB filename containing protein model and docked ligand that can guide the placement of new ligand into the an apo model
`protein_pdb_file_name = None` PDB file containing protein into which the ligand will be fit.
`ligand_input_file_name = None` File containing chemical input of a ligand
`ligand_code = None` Ligand code from Chemical Components
`data_file_name = None` Amplitudes or intensities
`map_coeffs_file_name = None` Map coefficients from a refinement of the apo model
`ligand_cif_file_name = None` This restraints file will be used for the real space refinement. If not supplied, a set of restraints are generated from the ligand input.
`ligand_selection_in_guide_model = None` The specific ligand to use from the guide structure, using the standard Phenix atom selection syntax.
`real_space_refinement = *ideal` off Perform a real space refinement after each ligand is added
`real_space_refinement_selection_radius = 5`.
`replace_ligand_in_guide_pdb_file_name = False` Use this option to replace the ligand in the Guide protein. This option saves creating a apo model of the same protein.
`map_labels = None`
- `guide_pdb_file_name = None` PDB filename containing protein model and docked ligand that can guide the placement of new ligand into the an apo model
- `protein_pdb_file_name = None` PDB file containing protein into which the ligand will be fit.
- `ligand_input_file_name = None` File containing chemical input of a ligand
- `ligand_code = None` Ligand code from Chemical Components
- `data_file_name = None` Amplitudes or intensities
- `map_coeffs_file_name = None` Map coefficients from a refinement of the apo model
- `ligand_cif_file_name = None` This restraints file will be used for the real space refinement. If not supplied, a set of restraints are generated from the ligand input.
- `ligand_selection_in_guide_model = None` The specific ligand to use from the guide structure, using the standard Phenix atom selection syntax.
- `real_space_refinement = *ideal` off Perform a real space refinement after each ligand is added
- `real_space_refinement_selection_radius = 5`.
- `replace_ligand_in_guide_pdb_file_name = False` Use this option to replace the ligand in the Guide protein. This option saves creating a apo model of the same protein.
- `map_labels = None`
- `outputfile_name = None` Defaults to current directory
`job_title = None` Job title in PHENIX GUI, not used on command line

- file_name = None Defaults to current directory
- job_title = None Job title in PHENIX GUI, not used on command line
- superpose_pdbsinputpdb_file_name_protein = None Name of PDB file with model to fit
topdb_file_name_guide = None Name of PDB file with model that will be fit to
pdb_file_name_proteinpdb_file_name_guide_other = None Name of another PDB file to transform with the same matrix as pdb_file_name_guidealignmentSet of parameters for sequence alignment. Defaults are good for most of casesalignment_style = local *global gap_opening_penalty = 1
gap_extension_penalty = 1 similarity_matrix = blosum50 dayhoff *identity validationrmsd_cutoff = 2.
similarity_cutoff = 80.
- inputpdb_file_name_protein = None Name of PDB file with model to fit topdb_file_name_guide = None
Name of PDB file with model that will be fit to pdb_file_name_proteinpdb_file_name_guide_other = None
Name of another PDB file to transform with the same matrix as pdb_file_name_guide
- pdb_file_name_protein = None Name of PDB file with model to fit to
- pdb_file_name_guide = None Name of PDB file with model that will be fit to pdb_file_name_protein
- pdb_file_name_guide_other = None Name of another PDB file to transform with the same matrix as
pdb_file_name_guide
- alignmentSet of parameters for sequence alignment. Defaults are good for most of
casesalignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1
similarity_matrix = blosum50 dayhoff *identity
- alignment_style = local *global
- gap_opening_penalty = 1
- gap_extension_penalty = 1
- similarity_matrix = blosum50 dayhoff *identity
- validationrmsd_cutoff = 2. similarity_cutoff = 80.
- rmsd_cutoff = 2.
- similarity_cutoff = 80.
- automatic_guidanceuse_web_services = False Use the homology search in RCSB web services to
identify and download candidate guide models.use_local_directory = None A directory containing models
to search for similar proteins to use as a guide for fitting a similar ligand, similarly
bound.minimum_ligand_similarity_tanimoto = 0.7 Minimum Tanimoto score to filter possible guide
models.
- use_web_services = False Use the homology search in RCSB web services to identify and download
candidate guide models.
- use_local_directory = None A directory containing models to search for similar proteins to use as a
guide for fitting a similar ligand, similarly bound.
- minimum_ligand_similarity_tanimoto = 0.7 Minimum Tanimoto score to filter possible guide models.
- process_controlnproc = 1
- nproc = 1
- superpose_ligandsquick = True
- quick = True

Automated ligand identification

ligand_identification.html

Contents

- Author(s)
- Purpose
- Purpose of the phenix.ligand_identification command
- Usage
- How the phenix.ligand_identification works:
- How to run the phenix.ligand_identification
- What the phenix.ligand_identification command needs to run:
- Choice of ligand library (keywords+examples)
- Output files from phenix.ligand_identification command
- Multiprocessing
- Running from a parameters file
- Examples
- Sample command_line inputs
- Possible Problems
- Specific limitations and problems:
- Literature
- Additional information
- List of all available keywords

Author(s)

- phenix.ligand_identification: Li-Wei Hung
- PHENIX GUI and PDS Server: Nigel W. Moriarty, Nathaniel Echols
- RESOLVE: Tom Terwilliger

Purpose

Purpose of the phenix.ligand_identification command

The phenix.ligand_identification command carries out fitting of a library of 180 most frequently observed ligands in the PDB to a given electron density map. The program also conducts the analysis and ranking of the ligand fitting results.

The phenix.ligand_identification command works with the ligand library provided with the Phenix program by default. It can also take a custom ligand library provided by the users.

Usage

The phenix.ligand_identification task can be run from command line, or from the PHENIX GUI.

How the phenix.ligand_identification works:

The phenix.ligand_identification command uses RESOLVE ligand fitting methods as described in the LigandFit documentation. The phenix.ligand_identification carries out this fitting process for a library of 180 most frequently observed ligands in the Protein Data Bank, or a custom library as described above, scores and ranks the overall fitting results. A real-space refinement is carried out on the ligand by default between RESOLVE fitting and Phenix scoring. The scoring algorithm takes into consideration of density correlation between ligand and density as well as non-bonded interactions between fitted ligand and the input model. The output consists of a list of the best fitted ligands from the library. The command provides options to view the top ranked ligand in coot with or without the electron density (use keyword "--open_in_coot=True").

How to run the phenix.ligand_identification

Example commands are provided below. The nsf-d2 files are in \$PHENIX/phenix_examples/nsf-d2-ligand. This program accepts all LigandFit keywords in addition to the build-in keywords listed at the end of this document.

What the phenix.ligand_identification command needs to run:

The phenix.ligand_identification command needs:

- a mtz file containing structure factors
- (optional), a PDB file with your protein model without ligand

Choice of ligand library (keywords+examples)

*default ligand library: no keyword needed

*to specify a list of ligands in 3-letter codes:

```
--ligand_list="ATP CTP TTP GTP"
```

*to use all small molecules with a .pdb extension in the 'ligand_dir' as the search library:

```
--ligand_dir=/my/compound/library/A1217321
```

*to use all ligands found in sequence and structural homologs of the input model as the search library. (Note, the compilation of ligand library is all done internally. No query is sent through the network.) :

```
--use_pdb_ligand=True
```

*to use all ligands associated with proteins of specific function found in the PDB as the search library:

```
--function="tyrosine kinase"
```

*to use all ligands found in proteins with a specific Enzyme Classification number in the PDB:

```
--EC=1.1.1.3
```

*to use all ligands found in proteins with a specific Pfam accession number in the PDB:

```
--pfam=PF00042
```

*to use all ligands found in proteins with specific SCOP and/or CATH terms in the PDB:

```
--scop='DNA/RNA-binding 3-helical bundle'
```

```
--cath='Trypsin-like serine proteases'
```

*to use all ligands found in proteins with a specific Gene Ontology (GO) accession number in the PDB:

```
--GO=0009253
```

*to use all ligands found in proteins with a specific InterPro ID in the PDB:

```
--ipro=014838
```

*to filter ligand sizes (# of non-H atoms) in ligand library to be used in ligand search:

(use either or both) `--natom_min=30 --natom_max=80`

Output files from phenix.ligand_identification command

When you run phenix.ligand_identification command, the output files will be in the directory you started Phenix:

- A summary file of the fitting results of all ligands:

overall_ligand_scores.log

- A summary table listing the results of the top ranked ligands:

topligand.txt

The last column "Sequence in library" contains numbers '####' indicating the sequence number of the corresponding ligands. The final fitted ligand coordinates and all the log files are in the corresponding '####' files described below.

- PDB files with the fitted ligands:

ligand_fit_pdbs/RSR_FITTED_[3-letter code]_####.pdb

- Resolve fitting Score files:

ligand_fit_scores/[3-letter code].scores

- Map coefficients for the map used for fitting:

resolve_map.mtz

- Command file to display results in coot:

display.com (uses ligid.scm, also in the same directory)

Multiprocessing

The phenix.ligand_identification program has build-in multi- processing capability. Use keyword

```
--nproc=[number of threads]
```

to run the command in multiprocessing mode. In general, the processing speed is proportional to the the number of CPU cores used, up to the maximum free cores the system can allocate at run time. For example, it takes about 25 minutes for the nsf-d2 example to run in 8 threaded mode, while single-process uses about 180 minutes for the same job on a dual-Xeon W5580 machine.

Running from a parameters file

You can run phenix.ligand_identification from a parameters file. This is often convenient because you can generate a default one with:

```
phenix.ligand_identification --show_defaults > my_ligand.eff
```

and then you can just edit this file to match your needs and run it with:

```
phenix.ligand_identification my_ligand.eff
```

Examples

Sample command_line inputs

1. Standard run of ligand_identification (input protein model and data file, default ligand library, 8 CPUs)

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb  
input_labels=F nproc=8
```

2. Search ligand from a difference map or pre-calculated map coefficients from phenix.refine

If you refine a model with a command such as,

```
phenix.refine data.mtz partial.pdb
```

then you will end up with the refined model,

```
partial_refine_001.pdb
```

and a map coefficients file:

```
partial_refine_001_map_coeffs.mtz
```

You can then run ligand_identification using the 2Fo-Fc map calculated from these map coefficients:

```
phenix.ligand_identification mtz_type=diffmap mtz_in=partial_refine_001_map_coeffs.mtz  
input_labels="2FOFCWT PH2FOFCWT" model=partial_refine_001.pdb nproc=8
```

For Fo-Fc map from the same file you can say:

```
phenix.ligand_identification mtz_type=diffmap mtz_in=partial_refine_001_map_coeffs.mtz  
input_labels="FOFCWT PHFOFCWT" model=partial_refine_001.pdb nproc=8
```

In the above two cases, "model" keyword is optional. If provided, non-bonded energy terms will be used in scoring.

The examples below show various ways of specifying custom ligand libraries based on the 'standard run' example above.

3. Identify ligand from a series of ligands in 3-letter codes

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb  
input_labels=F ligand_list="ATP GTP CTP TTP ADP GDP CDP TDP A3P GSP NAP AMP GMP CMP TMP"
```

4. Identify ligand from a given set of pdb files (could be your compound library) in a specific directory

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb  
input_labels=F ligand_dir=/my/compound/library/A1217321
```

This command will take all .pdb files in the ligand_dir and make a custom library to carry out the search

5. Identify ligand from a library of ligands founds in homologous structures (sequence or structural homologs) of the input pdb file.

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb  
input_labels=F use_pdb_ligand=True
```

This command will compile a library of ligands found in homologous pdbs of the 'model=' pdb file from Phenix's internal library. You can combine this keyword with other functional keywords (function, EC, pfam, ipro ...etc) and the program will compile a non-redundant combined ligand library. You may further filter the library to limit the sizes of ligands used in the search. See below. All these can be done in the GUI as well.

6. Identify ligand from a library composed of all ligands found in Tyrosine kinases in the PDB, and number of non-H atoms between 20 and 40

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb
input_labels=F function="Tyrosine kinase" natom_min=20 natom_max=40
```

This command could be useful when you want to use a function-specific ligand library. The 'function' should be in one of the EC terms. Note in the above example, if only "kinase" is specified, ligand found in all types of kinases will be searched.

7. Identify ligand from a library composed of all ligands found in proteins belonging to a specific Enzyme Classification

```
phenix.ligand_identification mtz_in=nsf-d2.mtz model=nsf-d2_noligand.pdb
input_labels=F EC=1.1.3
```

This command could be useful when you want to use a function-specific ligand library, and your protein belongs to a EC. Note the EC can be any parent set of the actual EC. (e.g. you can use 1.1.3 instead of 1.1.3.1 although you'll get a broader set of library with EC=1.1.3).

The nsf-d2 files in the above examples can be found in \$PHENIX/phenix_examples/nsf-d2-ligand

Possible Problems

Specific limitations and problems:

- When using a custom ligand library in PDB format, the ligand atoms in the user-provided PDBs should be under 'HETATM' records.
- Other ligand-fitting related limitations please refer to the document of the LigandFit wizard.
- Only crystallographic data are supported currently.

Literature

Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 63, 101-7 (2006).

- Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 63, 101-7 (2006).

Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 62, 915-22 (2006).

- Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 62, 915-22 (2006).

Additional information

List of ligands in the PHENIX ligand_identification default library:

```
PG4
CRY
```

CYS
DIO
DOX
GOL
M05
NBN
OXL
OXM
PUT
PYR
F3S
M06
PEG
COA
DTT
FS4
HED
HEZ
LI1
MPD
SF4
SIN
TMN
TRS
URA
BEN
BEZ
MET
NIO
PGA
POP
ASP
DAO
FS0
FUC
GLU
LYS
PEP
PGE
PHB
PHQ
PLM
TAR
XYS
ADE
AKG
7HP
BGC
CAM
HC4
ORO
DKA
GAL
GLC
MAN
MES
ARG
PHE
CIT
FLC
MMA
MPO
MYR
NHE
OLA
PG4
AMG

FER
NAA
NAG
NDG
SPM
EPE
NGA
PLP
TRP
BTN
F6P
FTT
G6P
LDA
UPL
1PE
BH4
H4B
THM
1PG
P6G
U10
ADN
BOG
EST
FBP
GSH
GTT
NVP
RET
UMP
C5P
C8E
DHT
TMP
UFP
CMP
NCN
PRP
BGC
MAN
GTS
IMP
LAT
MAL
SUC
GLC
AMP
FPP
PQQ
T44
2GP
3GP
5GP
TYD
UDP
GTX
SAH
TDP
TPP
IMO
SAM
A3P
ADP
DCP
GDP
NAG

XYS
2PE
CTP
DAD
FOK
TTP
DTP
DGA
FMN
SAP
ACP
ANP
APC
ATP
FOL
GNP
GSP
GTP
MTX
MAN
CB3
MA4
UPG
UD1
SP0
HEC
HEM
NAG
NAD
NAI
ACR
GLC
NAP
NDP
DHE
BPH
ACO
BCL
FAD
CAA
GLC
AP5
BPB
B12

List of all available keywords

Note: To use LigandFit parameters in `ligand_identification`, the parameter group (i.e. the bold text above the desired keyword) should be added to the keywords. For example,

*to specify search center:

```
search_target.search_center="10 10 10"
```

*to specify number of copies of the ligand in the asymmetric unit:

```
search_parameters.number_of_ligands=2
```

`ligand_identification` `mtz_in` = None Enter an MTZ file name `mtz_type` = *F diffmap If input a precalculated difference map, use "D" instead `input_labels` = None Provide a label of F if `mtz_type` is F, provide F, PHI if `mtz` if a difference map `ligand_list` = None enter ligands to be searched. Ligands should be in 3-letter codes separated by spaces. For example, `ligand_list`="ATP CTP APN FMN". If no input or uninterpretable, Default lib will be used. See `phenix.doc` for default ligand lib `ligand_dir` = None Directory of your ligand library. Files in this directory with `.pdb` or `.cif` extensions will be used in ligand search. If None, default library will be used.

work_dir = None Top level directory where jobs will be run. Default is the directory where phenix.ligand_identification is started. EC = None Provide an EC# of your protein. The program will screen all ligands found in PDBs with in this Enzyme class or subclass. function = None Provide function description of your protein. For example, 'kinase'. The program will then screen all ligands found in PDBs associated with your functional terms. Note: function should belong to partial EC terms. scop_fold = None Provide fold information of your protein in SCOP terms. For example, 'tim barrel'. The program will then screen all ligands found in PDBs with the partiular folds in SCOP terms. scath = None Provide structural information of protein in CATH terms. For example, 'Alanine racemase'. The program will then screen all ligands found in PDBs associated with this CATH term. pfam = None Provide PFam IDs for protein(s). For example 'PF01280'. The program will then screen all ligands found in PDBs of the PF01280 family. go = None Provide Gene Ontology(GO) information of the protein. For example, '01234'. The program will then screen all ligands found in PDBs associated with that GO number. ipro = None Provide Interpro ID of the protein. For example, '01740'. The program will then screen all ligands found in PDBs associated with the specified InterPro ID. use_pdb_ligand = False Use ligands found in homologous pdbs of the input model. The progeam will conduct a fast structure and sequence search within Phenix (no internet queries) to identify homologs of the input pdb. All ligands found in these homologous pdbs will be used to in ligand search. natom_min = 4 minimum number of non-H atoms in ligand. natom_max = 999 maximum number of non-H atoms in ligand. model = None Enter a PDB files containing the protein only. high_resolution = 2.5 specify the high resolution to use. default=2.5 low_resolution = 1000 Low resolution. restart_run = False Use (restart_run = True) to continue unfinished run. partial_analyze = False Analyze results before jobs complete. This function is currently not available with command-line version. ncpu = 1 Number of CPUs to use. n_indiv_tries_min = 30 usually 0 to 10, but set up to 300 to try harder to find solnn. n_indiv_tries_max = 300 usually 0 to 10, but set up to 300 to try harder to find solnn. group_search = 4 usually 3, but set up to 10 to try harder to find solnn. search_dist = 10 usually 10 A; always at least 5. smaller speeds up search. delta_phi_lig = 40 usually 40 degree increments. set lower to search more. fit_phi_inc = 20 usually 10 increments. set lower to search more. local_search = True Usually True; Use False to force complete search. search_center = " Search center (in A) for ligand search. Leave it empty or 0 0 0 to ignore. ligand_near_res = None Search ligand near this protein residue. Please enter a residue selection syntax, e.g. "chain A and resid 120". Leave it empty or None to ignore. nproc = 1 number of processors to use. This is used in resolve internal parallelization. verbose = False verbose output. debug = False debugging output. use_ligandfit = True use ligandfit to run fitting. search_mode = *default LigandFit Use default mode (1 trial, faster), or LigandFit mode (5 trials, takes longer, or 1 very short trial with 'quick=True) in ligand fitting. temp_dir = Auto Optional tempora...

- ligand_identification mtz_in = None Enter an MTZ file name mtz_type = *F diffmap If input a precalculated difference map, use "D" instead input_labels = None Provide a label of F if mtz type is F, provide F, PHI if mtz if a difference map ligand_list = None enter ligands to be searched. Ligands should be in 3-letter codes seperated by spaces. For example, ligand_list="ATP CTP APN FMN". If no input or uninterpretable, Default lib will be used. See phenix.doc for default ligand lib. ligand_dir = None Directory of your ligand library. Files in this directory with .pdb or .cif extentions will be used in ligand search. If None, default library will be used. work_dir = None Top level directory where jobs will be run. Default is the directory where phenix.ligand_identification is started. EC = None Provide an EC# of your protein. The program will screen all ligands found in PDBs with in this Enzyme class or subclass. function = None Provide function description of your protein. For example, 'kinase'. The program will then screen all ligands found in PDBs associated with your functional terms. Note: function should belong to partial EC terms. scop_fold = None Provide fold information of your protein in SCOP terms. For example, 'tim barrel'. The program will then screen all ligands found in PDBs with the partiular folds in SCOP terms. scath = None Provide structural information of protein in CATH terms. For example, 'Alanine racemase'. The program will then screen all ligands found in PDBs associated with this CATH term. pfam = None Provide PFam IDs for protein(s). For example 'PF01280'. The program will then screen all ligands found in PDBs

of the PF01280 family.go = None Provide Gene Ontology(GO) information of the protein. For example, '01234'. The program will then screen all ligands found in PDBs associated with that GO number.ipro = None Provide Interpro ID of the protein. For example, '01740'. The program will then screen all ligands found in PDBs associated with the specified InterPro ID.use_pdb_ligand = False Use ligands found in homologous pdbs of the input model. The program will conduct a fast structure and sequence search within Phenix (no internet queries) to identify homologs of the input pdb. All ligands found in these homologous pdbs will be used to in ligand search.natom_min = 4 minimum number of non-H atoms in ligand.natom_max = 999 maximum number of non-H atoms in ligand.model = None Enter a PDB files containing the protein only. high_resolution = 2.5 specify the high resolution to use. default=2.5 low_resolution = 1000 Low resolutionrestart_run = False Use (restart_run = True) to continue unfinished run partial_analyze = False Analyze results before jobs complete. This function is currently not available with command-line version. ncpu = 1 Number of CPUs to use.n_indiv_tries_min = 30 usually 0 to 10, but set up to 300 to try harder to find solnn_indiv_tries_max = 300 usually 0 to 10, but set up to 300 to try harder to find solnn_group_search = 4 usually 3, but set up to 10 to try harder to find solnsearch_dist = 10 usually 10 Å; always at least 5. smaller speeds up searchdelta_phi_lig = 40 usually 40 degree increments. set lower to search morefit_phi_inc = 20 usually 10 increments. set lower to search morelocal_search = True Usually True; Use False to force complete searchsearch_center = " Search center (in Å) for ligand search. Leave it empty or 0 0 0 to ignore.ligand_near_res = None Search ligand near this protein residue. Please enter a residue selection syntax, e.g. "chain A and resid 120". Leave it empty or None to ignore.nproc = 1 number of processors to use. This is used in resolve internal parallelizationverbose = False verbose outputdebug = False debugging outputuse_ligandfit = True use ligandfit to run fitting search_mode = *default LigandFit Use default mode (1 trial, faster), or LigandFit mode (5 trials, takes longer, or 1 very short trial with 'quick=True) in ligand fittingtemp_dir = Auto Optional tempora...

- mtz_in = None Enter an MTZ file name
- mtz_type = *F diffmap If input a precalculated difference map, use "D" instead
- input_labels = None Provide a label of F if mtz type is F, provide F, PHI if mtz if a difference map
- ligand_list = None enter ligands to be searched. Ligands should be in 3-letter codes separated by spaces. For example, ligand_list="ATP CTP APN FMN". If no input or uninterpretable, Default lib will be used. See phenix.doc for default ligand lib
- ligand_dir = None Directory of your ligand library. Files in this directory with .pdb or .cif extensions will be used in ligand search. If None, default library will be used.
- work_dir = None Top level directory where jobs will be run. Default is the directory where phenix.ligand_identification is started.
- EC = None Provide an EC# of your protein. The program will screen all ligands found in PDBs with in this Enzyme class or subclass.
- function = None Provide function description of your protein. For example, 'kinase'. The program will then screen all ligands found in PDBs associated with your functional terms. Note: function should belong to partial EC terms.
- scop_fold = None Provide fold information of your protein in SCOP terms. For example, 'tim barrel'. The program will then screen all ligands found in PDBs with the particular folds in SCOP terms
- cath = None Provide structural information of protein in CATH terms. For example, 'Alanine racemase'. The program will then screen all ligands found in PDBs associated with this CATH term.
- pfam = None Provide PFam IDs for protein(s). For example 'PF01280'. The program will then screen all ligands found in PDBs of the PF01280 family.

- go = None Provide Gene Ontology(GO) information of the protein. For example, '01234'. The program will then screen all ligands found in PDBs associated with that GO number.
- ipro = None Provide Interpro ID of the protein. For example, '01740'. The program will then screen all ligands found in PDBs associated with the specified InterPro ID.
- use_pdb_ligand = False Use ligands found in homologous pdbs of the input model. The progeam will conduct a fast structure and sequence search within Phenix (no internet queries) to identify homologs of the input pdb. All ligands found in these homologous pdbs will be used to in ligand search.
- natom_min = 4 minimum number of non-H atoms in ligand.
- natom_max = 999 maximum number of non-H atoms in ligand.
- model = None Enter a PDB files containing the protein only.
- high_resolution = 2.5 specify the high resolution to use. default=2.5
- low_resolution = 1000 Low resolution
- restart_run = False Use (restart_run = True) to continue unfinished run
- partial_analyze = False Analyze results before jobs complete. This function is currently not available with command-line version.
- ncpu = 1 Number of CPUs to use.
- n_indiv_tries_min = 30 usually 0 to 10, but set up to 300 to try harder to find soln
- n_indiv_tries_max = 300 usually 0 to 10, but set up to 300 to try harder to find soln
- n_group_search = 4 usually 3, but set up to 10 to try harder to find soln
- search_dist = 10 usually 10 Å; always at least 5. smaller speeds up search
- delta_phi_lig = 40 usually 40 degree increments. set lower to search more
- fit_phi_inc = 20 usually 10 increments. set lower to search more
- local_search = True Usually True; Use False to force complete search
- search_center = " Search center (in Å) for ligand search. Leave it empty or 0 0 0 to ignore.
- ligand_near_res = None Search ligand near this protein residue. Please enter a residue selection syntax, e.g. "chain A and resid 120". Leave it empty or None to ignore.
- nproc = 1 number of processors to use. This is used in resolve internal parallelization
- verbose = False verbose output
- debug = False debugging output
- use_ligandfit = True use ligandfit to run fitting
- search_mode = *default LigandFit Use default mode (1 trial, faster), or LigandFit mode (5 trials, takes longer, or 1 very short trial with 'quick=True) in ligand fitting
- temp_dir = Auto Optional temporary work directory
- output_dir = None Output directory where files are to be written
- dry_run = False Just read in and check parameter names
- number_of_ligands = None Total number of ligand sites. Ignored if "None". find_all_ligands will keep looking until the correlation coefficient for the fit of the best ligand is less than cc_min or the number of ligands placed is number_of_ligands, whichever comes first
- cc_min = 0.75 Ignored if "None". find_all_ligands will keep looking until the correlation coefficient for the fit of the best ligand is less than cc_min or the number of ligands placed is number_of_ligands, whichever

comes first

- `open_in_coot = False` If true, Phenix will automatically start Coot after the run is complete.
- `non_bonded = True` If true, `non_bonded` terms will be applied to the scores if RSR is used and model exists.
- `keep_all_files = False` If true, all intermediate pdb and log files will be kept.
- `cif_def_file_list = None` You can supply cif files for real-space refinement. example:
`cif_def_file_list='/my/cif/file1.cif /my/cif/file2.cif'`
- `real_space_target_weight = 10` You can carry change the weight on the real-space term in real-space refinement on the ligand after fitting.
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `ligandfitdata = None` Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say `input_labels='FP'`, or `input_labels='FP PHIB FOM'`.
`ligand = None` Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)
`model = None` PDB file with model for everything but the ligand
`quick = False` Set to True for running as quickly as possible.
`crystal_infounit_cell = None` Enter cell parameter a b c alpha beta gamma
`resolution = 0` High-resolution limit. Zero means keep everything. If `map_in` is specified, resolution must be given.
`space_group = None` Space Group symbol (i.e., C2221 or C 2 2 21)
`file_infofile_or_file_list = *single_file file_with_list_of_files` Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands
`input_labels = None` Labels for input data columns
`lig_map_type = fo-fc_difference_map`
`fobs_map *pre_calculated_map_coeffs` Enter the type of map to use in ligand fitting
`fo-fc_difference_map`: Fo-Fc difference map phased on partial model (requires FOBS in your input file)
`fobs_map`: Fo map phased on partial model (requires FOBS in your input file)
`pre_calculated_map_coeffs`: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at `pre_calculated_map_coeffs`. If your labels choice is not available and '---' is present, you can choose '---'.
`ligand_format = *PDB SMILES` Enter whether the files contain SMILES strings or PDB formatted information
`output_filetarget_output_format = *None pdb mmCIF` Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
`input_filesexisting_ligand_file_list = None` You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration.
`ligand_start = None` LigandFit will attempt to put your ligand superimposing on `ligand_start` if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the `ligand_start` ligand must be present in the `ligand_file` ligand (and must match exactly)
`ncs_in = None` You can supply a file with NCS information for use with
`ligands_from_ncsinput_ligand_compare_file = None` If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file.
`cif_def_file_list = None` You can supply cif files for real-space refinement after fitting
`refinement_file = None` You can supply a file for full refinement containing F/I SIGF/SIGI
`FreeR_flag` If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refine
`fobs_labels = None` Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner
`free_label = None` Label for FreeR_flag in refinement_file...same format as for phenix.refine
`map_in = None` Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.
`search_parametersfixed_ligand = False` Use fixed ligand (no rotations of any bonds) if set
`conformers = 1` Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate

them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate. group_search = 0 Enter the ID number of the group from the ligand to use to seed the search for conformations ligand_cc_min = 0.75 Enter...

- data = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" , or input_labels="FP PHIB FOM";.
- ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)
- model = None PDB file with model for everything but the ligand
- quick = False Set to True for running as quickly as possible.
- crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)
- unit_cell = None Enter cell parameter a b c alpha beta gamma
- resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.
- space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)
- file_infile_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file) pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- file_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands
- input_labels = None Labels for input data columns
- lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file) pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.
- ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.

- `target_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `input_filesexisting_ligand_file_list = None` You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. `ligand_start = None` LigandFit will attempt to put your ligand superimposing on `ligand_start` if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the `ligand_start` ligand must be present in the `ligand_file` ligand (and must match exactly)`ncs_in = None` You can supply a file with NCS information for use with `ligands_from_ncs``input_ligand_compare_file = None` If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file. `cif_def_file_list = None` You can supply CIF files for real-space refinement after fitting`refinement_file = None` You can supply a file for full refinement containing F/I SIGF/SIGI `FreeR_flag` If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refine`fobs_labels = None` Labels for Fobs SigFobs or lobs Siglobs for `refinement_file`... same format as for phenix.refine`r_free_label = None` Label for `FreeR_flag` in `refinement_file`...same format as for phenix.refine`map_in = None` Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.
- `existing_ligand_file_list = None` You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration.
- `ligand_start = None` LigandFit will attempt to put your ligand superimposing on `ligand_start` if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the `ligand_start` ligand must be present in the `ligand_file` ligand (and must match exactly)
- `ncs_in = None` You can supply a file with NCS information for use with `ligands_from_ncs`
- `input_ligand_compare_file = None` If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file.
- `cif_def_file_list = None` You can supply CIF files for real-space refinement after fitting
- `refinement_file = None` You can supply a file for full refinement containing F/I SIGF/SIGI `FreeR_flag` If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refine
- `fobs_labels = None` Labels for Fobs SigFobs or lobs Siglobs for `refinement_file`... same format as for phenix.refine
- `r_free_label = None` Label for `FreeR_flag` in `refinement_file`...same format as for phenix.refine
- `map_in = None` Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.
- `search_parametersfixed_ligand = False` Use fixed ligand (no rotations of any bonds) if `setconformers = 1` Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate. `group_search = 0` Enter the ID number of the group from the ligand to use to seed the search for conformations `ligand_cc_min = 0.75` Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations `ligand_completeness_min = 1` Enter the minimum completeness of the ligand to the map to quit searching for more conformations `local_search = True` If `local_search` is True then, only the region within `search_dist` of the point in the map with the highest local rmsd will be searched in the FFT search for fragments `search_dist = 10` If `local_search` is

True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments use_cc_local = False You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify use_cc_local, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$. ligands_from_ncs = False You can try to use ncs (from your partial model file or from your ncs_in file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand.max_ligands_from_ncs = 1 You can specify how many of the ligands already found to consider using NCS (usually 1)n_group_search = 3 Enter the number of different fragments of the ligand that will be looked for in FFT search of the map n_indiv_tries_max = 10 If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified n_indiv_tries_min = 5 If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified number_of_ligands = 1 Number of copies of the ligand expected in the asymmetric unit offsets_list = 7 53 29 You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models. refine_ligand = True You can carry out real-space refinement on the ligand after fittingligand_occupancy = 1.0 You can set the initial occupancy of the ligand real_space_target_weight = 10. You can carry change the weight on the real-space term in real-space refinement on the ligand after fittingfittingParameters for tracing liganddelta_phi_ligand = 40 Specify the angle (degrees) between successive tries in FFT search for fragments fit_phi_inc = 20 Specify the angle (degrees) between rotations around bonds fit_phi_range = -180 180 Range of bond rotation angles to search

- fixed_ligand = False Use fixed ligand (no rotations of any bonds) if set
- conformers = 1 Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate.
- group_search = 0 Enter the ID number of the group from the ligand to use to seed the search for conformations
- ligand_cc_min = 0.75 Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations
- ligand_completeness_min = 1 Enter the minimum completeness of the ligand to the map to quit searching for more conformations
- local_search = True If local_search is True then, only the region within search_dist of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- search_dist = 10 If local_search is True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- use_cc_local = False You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify use_cc_local, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$.
- ligands_from_ncs = False You can try to use ncs (from your partial model file or from your ncs_in file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand.

- `max_ligands_from_ncs = 1` You can specify how many of the ligands already found to consider using NCS (usually 1)
- `n_group_search = 3` Enter the number of different fragments of the ligand that will be looked for in FFT search of the map
- `n_indiv_tries_max = 10` If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified
- `n_indiv_tries_min = 5` If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified
- `number_of_ligands = 1` Number of copies of the ligand expected in the asymmetric unit
- `offsets_list = 7 53 29` You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models.
- `refine_ligand = True` You can carry out real-space refinement on the ligand after fitting
- `ligand_occupancy = 1.0` You can set the initial occupancy of the ligand
- `real_space_target_weight = 10.` You can carry change the weight on the real-space term in real-space refinement on the ligand after fitting
- `fittingParameters` for tracing ligand
`delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments
`fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds
`fit_phi_range = -180 180` Range of bond rotation angles to search
- `delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments
- `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds
- `fit_phi_range = -180 180` Range of bond rotation angles to search
- `search_targetligand_near_chain = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. If you set `ligand_near_chain="None"` or leave it blank or do not set it, then all chains will be included. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).
`ligand_near_res = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).
`ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
`name_of_ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
`search_center = 0.0 0.0 0.0` Enter coordinates for center of search region (ignored if [0.,0.,0.])
- `ligand_near_chain = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. If you set `ligand_near_chain="None"` or leave it blank or do not set it, then all chains will be included. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).
- `ligand_near_res = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).

- `ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
- `name_of_ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
- `search_center = 0.0 0.0 0.0` Enter coordinates for center of search region (ignored if [0.,0.,0.])
- `generalexten`
- `try_list = True` You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with `nbatch`.
- `ligand_id = None` You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in `input_lig_file` has. The keyword is only valid if a single copy of the ligand is to be found.
- `nbatch = 5` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`.
- `nproc = 1` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`.
- If you set `nproc=Auto` and your machine has `n` processors, then it will use `n-1` processors, or 1 if only 1 is available.
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: `coarse_grid resolution 200 2.0` `hkl in test.mtz` . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes.
- `coot_name = "coot"` If your version of coot is called something else, then you can specify that here.
- `i_ran_seed = 72432` Random seed (positive integer) for model-building and simulated annealing refinement
- `raise_sorry = False` You can have any failure end with a Sorry instead of simply printout to the screen
- `background = True` When you specify `nproc=nn`, you can run the jobs in background (default if `nproc` is greater than 1) or foreground (default if `nproc=1`). If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `check_wait_time = 1.0` You can specify the length of time (seconds) to wait between checking for subprocesses to end
- `max_wait_time = 1.0` You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait between submit time is File exists:....
- `cache_resolve_libs = True` Use caching of resolve libraries to speed up resolve
- `resolve_size = 12` Size for solve/resolve (""", "giant", "huge", "extra_huge" or a number where 12=giant 18=huge
- `check_run_command = False` You can have the wizard check your run command at startup
- `run_command = "sh "` When you specify `nproc=nn`, you can run the subprocesses as jobs in background with `sh` (default...

- `extend_try_list = True` You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with `nbatch`.
- `ligand_id = None` You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in `input_lig_file` has. The keyword is only valid if a single copy of the ligand is to be found.
- `nbatch = 5` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`.
- `nproc = 1` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of `nbatch`. If you set `nproc=Auto` and your machine has `n` processors, then it will use `n-1` processors, or 1 if only 1 is available
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: `coarse_grid resolution 200 2.0 hkl in test.mtz` . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes.
- `coot_name = "coot"` If your version of coot is called something else, then you can specify that here.
- `i_ran_seed = 72432` Random seed (positive integer) for model-building and simulated annealing refinement
- `raise_sorry = False` You can have any failure end with a Sorry instead of simply printout to the screen
- `background = True` When you specify `nproc=nn`, you can run the jobs in background (default if `nproc` is greater than 1) or foreground (default if `nproc=1`). If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `check_wait_time = 1.0` You can specify the length of time (seconds) to wait between checking for subprocesses to end
- `max_wait_time = 1.0` You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....
- `cache_resolve_libs = True` Use caching of resolve libraries to speed up resolve
- `resolve_size = 12` Size for solve/resolve ('<small>giant</small>', '<small>huge</small>', '<small>extra_huge</small>', or a number where 12=<small>giant</small> 18=<small>huge</small>

- `check_run_command = False` You can have the wizard check your run command at startup
- `run_command = "sh "` When you specify `nproc=nn`, you can run the subprocesses as jobs in background with `sh` (default) or submit them to a queue with the command of your choice (i.e., `qsub`). If you have a multi-processor machine, use `sh`. If you have a cluster, use `qsub` or the equivalent command for your system. NOTE: If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If `nproc` is greater than 1 and you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `queue_commands = None` You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: `queue_commands='#PBS -N mr_rosetta'`
`queue_commands='#PBS -j oe'` `queue_commands='#PBS -l walltime=03:00:00'`
`queue_commands='#PBS -l nodes=1:ppn=1'` NOTE: you can put in the characters '`<path>`' in any `queue_commands` line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_', up to 15 characters. For example `'test_autobuild/WORK_5/AutoBuild_run_1/TEMP0/RUN_1'` would be represented by: `'tld_W_5_A1__TP0_1'`
- `condor_universe = vanilla` The universe for condor is usually vanilla. However you might need to set it to local for your cluster
- `add_double_quotes_in_condor = True` You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.
- `condor = None` Specifies if the `group_run_command` is submitting a job to a condor cluster. Set by default to True if `group_run_command=condor_submit`, otherwise False. For condor job submission `mr_rosetta` uses a customized script with condor commands. Also uses `one_subprocess_level=True`
- `last_process_is_local = True` If true, run the last process in a group in background with `sh` as part of the job that is submitting jobs. This prevents having the job that is submitting jobs sit and wait for all the others while doing nothing
- `skip_r_factor = False` You can skip R-factor calculation if refinement is not done and `maps_only=True`
- `test_flag_value = Auto` Normally leave this at Auto (default). This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with `test_flag_value`.
- `skip_xtriage = False` You can bypass xtriage if you want. This will prevent you from applying anisotropy corrections, however.
- `base_path = None` You can specify the base path for files (default is current working directory)
- `temp_dir = None` Define a temporary directory
- `local_temp_directory = None` Write all temporary files to this directory and copy files specified by `keep_files` back to TEMP directory in working directory at the end of run. Used to speed up read/write operations. NOTE: Deletes all files except those specified by `keep_files` just like `clean_up=True`.
- `clean_up = None` At the end of the entire run the TEMP directories will be removed if `clean_up` is True. Also all files except for `overall_best_xxx` (final) files and `.eff` files in the `AutoSol_run_xx_` `AutoBuild_run_xx_` etc directory will be removed. Files listed in `keep_files` will not be deleted. If you want

to remove files after your run is finished use a command like `"phenix.autobuild run=1 clean_up=True"`;

- `print_citations = True` Print citations at end of run
- `solution_output_pickle_file = None` At end of run, write solutions to this file in output directory if defined
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `top_output_dir = None` This is used in subprocess calls of wizards and to tell the Wizard where to look for the STOPWIZARD file.
- `wizard_directory_number = None` This is used by the GUI to define the run number for Wizards. It is the same as `desired_run_number` NOTE: this value can only be specified on the command line, as the directory number is set before parameters files are read.
- `verbose = False` Command files and other verbose output will be printed
- `extra_verbose = False` Facts and possible commands will be printed every cycle if True
- `debug = False` You can have the wizard stop with error messages about the code if you use debug. Additionally the output goes to the terminal if you specify `"debug=True"`;
- `require_nonzero = True` Require non-zero values in data columns to consider reading in.
- `remove_path_word_list = None` List of words identifying paths to remove from PATH These can be used to shorten your PATH. For example... `cns ccp4 coot` would remove all paths containing these words except those also containing `phenix`. Capitalization is ignored.
- `fill = False` Fill in all missing reflections to resolution `res_fill`. Applies to density modified maps. See also `filled_2fofc_maps` in `autobuild`.
- `res_fill = None` Resolution for filling in missing data (default = highest resolution of any datafile). Only applies to density modified maps. Default is fill to high resolution of data. Ignored if `fill=False`
- `check_only = False` Just read in and check initial parameters. Not for general use
- `keep_files = ligandfit*.pdb` List of files that are not to be cleaned up. wildcards permitted
- `displaynumber_of_solutions_to_display = None` Number of solutions to put on screen and to write out
`solution_to_display = 1` Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- `number_of_solutions_to_display = None` Number of solutions to put on screen and to write out
- `solution_to_display = 1` Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- `run_controlignore_blanks = None` `ignore_blanks` allows you to have a command-line keyword with a blank value like `"input_lig_file_list="`; `stop = None` You can stop the current wizard with `"stopwizard"` or `"stop"`;. If you type `"phenix.autobuild run=3 stop"`; then this will stop run 3 of `autobuild`. `display_facts = None` Set `display_facts` to True and optionally `run=[run-number]` to display the facts for run `run-number`. If you just say `display_facts` then the facts for the highest-numbered existing run will be shown. `display_summary = None` Set `display_summary` to True and optionally `run=[run-number]` to show the summary for run `run-number`. If you just say `display_summary` then the summary for the highest-numbered existing run will be shown. `carry_on = None` Set `carry_on` to True to carry on with highest-numbered run from where you left off. `run = None` Set `run` to `n` to continue with run `n` where you left off. `copy_run = None` Set `copy_run` to `n` to copy run `n` to a new run and continue where you left off. `display_runs = None` List all runs for this wizard. `delete_runs = None` List runs to delete: `1 2 3-5 9:12` `display_labels = None` `display_labels=test.mtz` will list all the labels that identify data in `test.mtz`. You can use the label strings that are produced in AutoSol to identify which

data to use from a datafile like this: `peak.data="F+ SIGF+ F- SIGF-"`; # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol and AutoBuild like this: `input_refinement_labels="FP SIGFP FreeR_flags"`; # each individual label counts
`dry_run = False` Just read in and check parameter names
`params_only = False` Just read in and return parameter defaults
`display_all = False` Just read in and display parameter defaults
`coot = None` Not presently applicable for ligandfit

- `ignore_blanks = None` `ignore_blanks` allows you to have a command-line keyword with a blank value like `"input_lig_file_list="`;
- `stop = None` You can stop the current wizard with `"stopwizard"` or `"stop"`. If you type `"phenix.autobuild run=3 stop"`; then this will stop run 3 of autobuild.
- `display_facts = None` Set `display_facts` to True and optionally `run=[run-number]` to display the facts for run `run-number`. If you just say `display_facts` then the facts for the highest-numbered existing run will be shown.
- `display_summary = None` Set `display_summary` to True and optionally `run=[run-number]` to show the summary for run `run-number`. If you just say `display_summary` then the summary for the highest-numbered existing run will be shown.
- `carry_on = None` Set `carry_on` to True to carry on with highest-numbered run from where you left off.
- `run = None` Set `run` to `n` to continue with run `n` where you left off.
- `copy_run = None` Set `copy_run` to `n` to copy run `n` to a new run and continue where you left off.
- `display_runs = None` List all runs for this wizard.
- `delete_runs = None` List runs to delete: 1 2 3-5 9:12
- `display_labels = None` `display_labels=test.mtz` will list all the labels that identify data in `test.mtz`. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: `peak.data="F+ SIGF+ F- SIGF-"`; # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol and AutoBuild like this: `input_refinement_labels="FP SIGFP FreeR_flags"`; # each individual label counts
- `dry_run = False` Just read in and check parameter names
- `params_only = False` Just read in and return parameter defaults
- `display_all = False` Just read in and display parameter defaults
- `coot = None` Not presently applicable for ligandfit
- `special_keywordswrite_run_directory_to_file = None` Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.
- `write_run_directory_to_file = None` Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.
- `non_user_parameters` These are obsolete parameters and parameters that the wizards use to communicate among themselves. Not normally for general use.
`gui_output_dir = None` Used only by the GUI
`sg = None` Obsolete. Use `space_group` instead
`get_lig_volume = False` You can ask to get the volume of the ligand and to then stop
`input_data_file = None` Not normally used. Use `"data="` instead
`input_lig_file = None` Not normally used. Use `"ligand="` instead.
`ligand_code = None` Not normally used. Use `"ligand="` instead.
`input_partial_model_file = None` Not normally used. Use `"model="` instead
`cif_already_generated = False` You can specify that the ligand cif file is already generated
- `gui_output_dir = None` Used only by the GUI

- `sg = None` Obsolete. Use `space_group` instead
- `get_lig_volume = False` You can ask to get the volume of the ligand and to then stop
- `input_data_file = None` Not normally used. Use `"data="` instead
- `input_lig_file = None` Not normally used. Use `"ligand="` instead.
- `ligand_code = None` Not normally used. Use `"ligand="` instead.
- `input_partial_model_file = None` Not normally used. Use `"model="` instead
- `cif_already_generated = False` You can specify that the ligand cif file is already generated

Automated protein-ligand structure determination with phenix.ligand_pipeline

ligand_pipeline.html

Contents

- Overview
- Input files
- Program workflow
- Interactive mode
- Output files
- Next steps
- Limitations
- Troubleshooting
- Other notes about use
- References
- List of all available keywords

Overview

phenix.ligand_pipeline is an automation system combining Xtriage, Phaser, eLBOW, phenix.refine, AutoBuild, and LigandFit, with optional interaction with Coot. In favorable cases, it can produce a near-finished structure of a protein-ligand complex starting from minimal inputs, and can significantly reduce the manual effort required for more difficult structures.

Input files

The typical input for the pipeline consists of processed experimental data, a starting model for molecular replacement, a sequence file equivalent to the starting model (with the same number of chains), and a source of ligand information. File types will be automatically detected if explicit parameter names are not used, for instance:

```
phenix.ligand_pipeline model.pdb data.mtz seq.fa lig.cif
```

The data may be in any format; if R-free flags are not found they will be generated automatically, but you may also supply the flags in a separate file (`r_free_flags.file_name=flags.mtz`). This is especially recommended if you are working with multiple isomorphous structures. The sequence file is optional but highly recommended.

To specify the ligand geometry, several mutually exclusive options are available:

- Chemical file, such as MOL2, CIF (this can include existing restraints), or SMILES: `ligand=lig.cif`
- SMILES string: `ligand_smiles=c1ccccc1`
- For ligands already present in the PDB, if you know the three-letter code used, you may specify this instead: `ligand_code=ATP`

It is recommended that you use the most detailed and explicit information about the ligand chemistry and geometry that you have available: a pre-generated and manually validated CIF is ideal. You may also supply

a PDB file (template_pdb=lig.pdb) containing the atom names to use. However, PDB files should not be the primary source of ligand geometry information! Additionally, if you have additional ligands in your input model, you can supply CIF files for refinement with the extra_cif keyword. You should also specify keep_hetatms=True to prevent Phaser from resetting the occupancies to zero.

Finally, if all files are in the same directory and their uses can be unambiguously determined, you may supply the path name as a single argument when launching the program:

```
phenix.ligand_pipeline /path/to/data [options]
```

Program workflow

Data preparation. The program begins by importing the experimental data and converting it to amplitudes with standard MTZ column labels (this simplifies later steps). R-free flags will be generated if not available, otherwise the user-supplied flags will be used (extended to higher resolution if necessary). Xtriage will be run to assess data quality and guess an appropriate number of search copies for MR (which, by default, will be the same as the number of ligands to look for).

Ligand generation. eLBOW will be run on the input ligand definition to generate suitable geometry restraints and a starting structure for LigandFit. The most important option is the choice of whether to run the semi-empirical AM1 quantum-mechanical optimization or not (optimize=True). This is not done by default because it is more time-consuming, but in some cases it may lead to improved geometry. For troublesome cases, we recommend generating the ligand restraints and structure manually prior to running the pipeline, then supplying the output CIF as input to the pipeline with the additional argument keep_input_restraints=True. In this case, eBLOW will only be used to generate a PDB file, and the input restraints will be used for refinement.

Model preparation. For each input model, a series of cleanup steps is run to remove specific details such as alternate conformations, hydrogen atoms, or waters. The model will also be processed with Sculptor to ensure that all residues match the target sequence. (This will not extend truncated sidechains, but this is available as a later step after ligand placement.)

Model placement. Two options are available, rigid-body refinement or molecular replacement:

- **Molecular replacement.** This uses the full MR_AUTO mode in Phaser, which includes rigid-body and group B-factor refinement (although the B-factors are not output). The number of search copies will be determined automatically by Xtriage (copies=Auto), but since this will occasionally guess too few copies, you can explicitly set the search number instead (e.g. copies=2). If Phaser finds fewer copies than expected, the number of expected ligands will automatically be adjusted. Non-water heteroatoms will be preserved at their original occupancies. If you want to preserve a specific placement or orientation of the MR solution, you can specify a previously determined model by passing the reference_structure parameter.
- **Rigid-body refinement.** If the input model is already close enough to the target structure, rigid-body refinement alone will be suitable.

Currently MR is run by default, but this will be changed in the future; if you are confident that the model can be correctly placed without MR, add mr=False or skip_mr=True or --skip_mr to jump straight to refinement. (This will automatically turn on rigid-body refinement in the subsequent step.) A more useful (but slightly slower) option is mr=Auto, which will attempt rigid-body refinement and then revert to running Phaser if the R-free is greater than 0.4 or no models with appropriate symmetry are found.

In automatic or MR mode, you may specify multiple models as input; the best model will be selected either by Phaser or the rigid-body refinement step. The space group will be updated based on the Phaser solution or

selected model if necessary.

Initial refinement. The MR solution (or starting model, if MR was not performed) will be refined using a relatively conservative resolution-dependent strategy, by default rotamer fitting, real-space and reciprocal-space coordinate minimization, individual B-factor refinement, and TLS groups. Torsion NCS will be used if appropriate. Rigid-body refinement will be used if Phaser was not previously run. Water molecules will be added if the resolution is sufficient, but these will be removed prior to ligand fitting. Simulated annealing may be run if desired (`anneal=True`), but this will significantly increase the runtime. Because rapid convergence is more important at this point than ideal geometry, weight optimization will not be used by default. (Water picking is used because this improves the maps, but the solvent atoms will be removed prior to ligand fitting to avoid overlap.)

If the refined R-free is above 0.5, the program will immediately exit, since this indicates that the structure may be incorrect or at least in need of extensive manual rebuilding (or possibly additional search copies for MR). If R-free is above 0.3, the default behavior (`build=Auto`) is to rebuild the model in AutoBuild. Otherwise, the structure is considered suitable for ligand fitting.

Rebuilding. Rebuilding is optional but may be essential in some cases where large rearrangements are required. AutoBuild will be run with `rebuild_in_place=Auto`, which in most cases means that residues will not be added or removed. If run, AutoBuild is generally the most time-consuming stage in the pipeline, although it can take advantage of multiple processors.

An alternate protocol, run by default, is to simply delete those residues with poor fit to density, with the goal of moving them out of the way of the ligand binding site. Either the mainchain or sidechain atoms or both can be removed, but the default is to only prune sidechains. These may optionally be rebuilt after the ligand is placed.

As a final preparation step, waters are removed from the model and the difference map is recalculated.

Ligand placement. Currently only LigandFit is supported, although additional options will be available in the future. The target map will be the mFo-DFc map from phenix.refine. The parameters have been adjusted slightly for optimal performance at the expense of slightly longer runtime; you may specify `quick=True` or `aggressive=True` to override the defaults. Real-space refinement will be performed on ligands after placement to obtain an optimized fit and local geometry. Unlike most of the other steps, here the resolution will be truncated by default to 1.8Å (`ligandfit.d_min=1.8`), because higher-resolution data tend to lead to lower correlations even when the fit is excellent.

An essential feature of automatic ligand fitting is a cutoff model-to-map CC to prevent false positives (i.e. spurious placement). By default this is set to 0.7, which has been empirically determined to nearly always indicate a successful fit. Any ligands with a CC below this value will be omitted from the model. In some cases this may actually reject correct fits (see Troubleshooting section below), but we recommend that these solutions be viewed with suspicion. If no acceptable ligand fits are obtained, the pipeline will skip further refinement and exit early with an error message.

In many cases the one or more ligands will be placed, but fewer than the desired number; in these situations, a separate post-LigandFit routine will first attempt to use NCS operators to place additional copies of the ligand, pruning the model as necessary to avoid clashes. If there are still fewer copies than desired, the pipeline will continue with refinement but print a warning at the end of the run.

Final refinement and validation. If the ligand placement was at least partially successful, a final round of refinement will be performed on the combined model. In this run, waters will be added automatically if the resolution is at least 2.9Å, and the geometry weight will be optimized if the resolution is worse than 1.75Å (this is much slower but can run on multiple processors, and generally produces a superior model).

Interactive mode

For users desiring more control over the rebuilding process, or for difficult cases where more aggressive model modifications are required, the program can be run semi-interactively by adding the flag `interactive=True` (or alternately, `--interactive`). This will launch Coot after the first round of refinement, with model and maps loaded, allowing rebuilding to take place. The recommended procedure is to locate the putative ligand density in the mFo-DFc map, and adjust the protein structure as necessary to remove overlap with the ligand binding site (as well as any other modifications that will not be automatically fixed by refinement and/or AutoBuild). Once rebuilding is complete, click the button "Return to PHENIX" to save the edited model and resume the pipeline (the next step will be LigandFit, with a freshly calculated mFo-DFc map). At any point during rebuilding, you can obtain new maps for the edited model by clicking "Fetch new maps" on the toolbar, which will write out a temporary model and recalculate maps in the background, automatically loading these into Coot when finished.

Because the interactive mode overcomes many of the limitations present when significant rebuilding is required, it is the recommended method for running the pipeline, especially for new users.

Output files

All files will be written to a directory named `pipeline_X`, where X is an automatically incremented integer. Most of the important output files will be named after the ligand code, by default LIG:

- `LIG_data.mtz`: filtered data and R-free flags
- `LIG_final.pdb`: final output model
- `LIG_final.mtz`: final map coefficients (as well as data and flags)
- `LIG_ligand.cif`: ligand geometry restraints
- `pipeline.log`: log file for the entire run
- `visualize_coot.py`: script for loading results in Coot

The Coot script can be run like this:

```
coot --script pipeline_1/visualize_coot.py
```

This will open the final model and map coefficients, and if ligand fitting was successful, a window will be created containing buttons to zoom in on the ligand(s).

Inside the output directory, separate subdirectories will be created for each stage of the pipeline, e.g. `phaser_0`, `refine_0`, `refine_1`, `LigandFit_run_1_`. You do not normally need any of the files in these directories, but they may be useful for debugging purposes. In particular, if LigandFit failed to place one or more copies, the ligand PDB files will not be incorporated into the final model, but will still be available in the LigandFit output directory.

Next steps

Although the pipeline can produce nearly-complete models in favorable cases (typically at moderate-to-high resolution, approximately 2.2 to 1.8Å), the structures are unlikely to be truly complete. Several recommendations for finishing the refinement in Coot:

- Inspect the ligand binding closely. A number of structures in the PDB have been shown to exhibit ligand placements which are not supported by the data, leading to several retractions so far. Although the CC cutoff for ligand placement is designed to avoid this outcome, it is essential that you visually confirm the refined ligand position. Of particular concern: Does the difference map show residual positive or

(especially) negative peaks around the ligand binding site? Even if the overall position is approximately correct, this may indicate problems with the specific conformation and/or geometry; it is not uncommon, for example, for nearly-symmetric rings to be fit backwards. Does the overall shape of the 2mFo-DFc map approximate that of the ligand? If not, this could mean that the ligand has been mistakenly placed in density for missing protein atoms, or for another unrelated ligand (such as contaminants from the crystallization solution). Note that it is quite possible for flexible ligands to be partially disordered, but in such cases the majority of the ligand should still be clearly defined in the map. Does the binding make biological sense? Pay special attention to hydrogen bond donors and acceptors in both the ligand and the protein, and especially the interactions of charged groups. Chemically nonsensical binding usually indicates an incorrect fit (or, in extreme cases, an incorrect ligand).

Inspect the ligand binding closely. A number of structures in the PDB have been shown to exhibit ligand placements which are not supported by the data, leading to several retractions so far. Although the CC cutoff for ligand placement is designed to avoid this outcome, it is essential that you visually confirm the refined ligand position. Of particular concern:

Does the difference map show residual positive or (especially) negative peaks around the ligand binding site? Even if the overall position is approximately correct, this may indicate problems with the specific conformation and/or geometry; it is not uncommon, for example, for nearly-symmetric rings to be fit backwards. Does the overall shape of the 2mFo-DFc map approximate that of the ligand? If not, this could mean that the ligand has been mistakenly placed in density for missing protein atoms, or for another unrelated ligand (such as contaminants from the crystallization solution). Note that it is quite possible for flexible ligands to be partially disordered, but in such cases the majority of the ligand should still be clearly defined in the map. Does the binding make biological sense? Pay special attention to hydrogen bond donors and acceptors in both the ligand and the protein, and especially the interactions of charged groups. Chemically nonsensical binding usually indicates an incorrect fit (or, in extreme cases, an incorrect ligand).

- Does the difference map show residual positive or (especially) negative peaks around the ligand binding site? Even if the overall position is approximately correct, this may indicate problems with the specific conformation and/or geometry; it is not uncommon, for example, for nearly-symmetric rings to be fit backwards.
- Does the overall shape of the 2mFo-DFc map approximate that of the ligand? If not, this could mean that the ligand has been mistakenly placed in density for missing protein atoms, or for another unrelated ligand (such as contaminants from the crystallization solution). Note that it is quite possible for flexible ligands to be partially disordered, but in such cases the majority of the ligand should still be clearly defined in the map.
- Does the binding make biological sense? Pay special attention to hydrogen bond donors and acceptors in both the ligand and the protein, and especially the interactions of charged groups. Chemically nonsensical binding usually indicates an incorrect fit (or, in extreme cases, an incorrect ligand).
- Look for additional ligands. It is quite common for protein structures to co-crystallize with buffer molecules, ions, and endogenous contaminants. These can be placed by running LigandFit, or interactively in Coot.

Look for additional ligands. It is quite common for protein structures to co-crystallize with buffer molecules, ions, and endogenous contaminants. These can be placed by running LigandFit, or interactively in Coot.

- Check for errors or missing pieces in the protein model. Even after refinement and/or rebuilding, some local fitting problems may be present (especially at lower resolution). Additionally, it is very common for pieces of the start model to be missing from the refined structure, or vice-versa; this will usually be obvious from the maps. Alternate conformations will normally be stripped out before molecular

replacement, so at high resolution (typically 1.6Å or better), these may need to be added manually. You should also check the validation results to identify geometry problems and misfitting.

Check for errors or missing pieces in the protein model. Even after refinement and/or rebuilding, some local fitting problems may be present (especially at lower resolution). Additionally, it is very common for pieces of the start model to be missing from the refined structure, or vice-versa; this will usually be obvious from the maps. Alternate conformations will normally be stripped out before molecular replacement, so at high resolution (typically 1.6Å or better), these may need to be added manually. You should also check the validation results to identify geometry problems and misfitting.

- If none of the above steps requires further adjustments, your model is probably complete. Otherwise, if you make any changes, you should run a final round of refinement with weight optimization to get the best possible model, and a final set of maps which should also be inspected.

If none of the above steps requires further adjustments, your model is probably complete. Otherwise, if you make any changes, you should run a final round of refinement with weight optimization to get the best possible model, and a final set of maps which should also be inspected.

More generally, with any automation system, two rules apply: exercise appropriate skepticism and seek expert opinion if unsure of the results. (If a local expert is not available, email help@phenix-online.org for advice.)

Limitations

- For maximum flexibility, no restriction is placed on the quality of the starting model or its similarity to the target model; the Sculptor step will take care of any necessary sequence deletions. However, in practice it is vastly more effective when working with near-identical structures.
- When working with a set of related structures that differ from previously solved crystal forms, it will probably be most productive to solve the first one interactively, then use this as a starting point for the remaining structures.
- Only one type of ligand is supported at present. Essential cofactors or other molecules whose binding mode is already known may included in the input model; in these cases, you should add `keep_hetatms=True` to instruct Phaser to leave the occupancies unmodified. Any additional ligands will need to be placed after the final refinement.
- Multiple stereoisomers will need to be screened independently; the program does not currently have the ability to evaluate different chiral forms.
- Covalently bound ligands may be placed correctly, but the covalent linkage will not be handled, and nonbonded forces may distort binding. Additionally, the ligand parameterization may be based on the unbound molecule.
- Structures such as HIV protease which exhibit symmetric binding of inhibitors will not be handled properly - and more generally, alternate conformations for the ligand are not modeled. (The pipeline may be able to place a single conformation, however.)
- The pipeline is designed to work at resolutions of 3.0Å or better; while it is possible to solve complexes at lower resolution, the ligand fitting procedure is not expected to work as well.
- There is no high-resolution limit, but structures at atomic resolution will inevitably require more manual effort to complete due to the presence of alternate conformations and (often) additional small-molecule ligands, which are not presently handled by Phenix.
- Building is currently limited to either rebuild-in-place mode or building an entirely new model from scratch, and rebuilds the entire model. This means that for structures where local rebuilding or addition of

a few residues is required, interactive mode will probably be more successful.

- Most of the optimization of program behavior and parameters has been focused on structures bound to drug-like molecules; less attention has been given to fragments. These tend to have weaker binding and partial occupancy, and are therefore more difficult to distinguish in maps.

Troubleshooting

The pipeline may fail for a number of reasons, usually because LigandFit could not successfully place any copies of the ligand. If this happens, the program will skip the final refinement step and exit early with an error message. You can still run the Coot script to load the output of the previous refinement or AutoBuild, which may provide clues about the cause of failure. Several common problems are listed below. Note that these may also apply to cases where the pipeline runs to completion but was unable to place all copies of the ligand successfully (in which case a warning message will be printed).

- The protein blocks the ligand site. This is one of the most common problems, especially in structures which undergo extensive conformational changes or local rearrangements, such as kinases. Even a single stray sidechain may prevent LigandFit from placing the ligand correctly, both due to flattened difference density, and steric clashes. For structures with NCS, this can sometimes be overcome by the NCS application procedure described above, but this still requires that at least one copy was placed correctly. Running AutoBuild can sometimes help, but this also has the risk of moving the protein into ligand density. The interactive mode is strongly recommended for overcoming this problem.
- Spurious difference density for other atoms. If large regions of positive mFo-DFc density corresponding to missing protein residues are present, LigandFit may sometimes place the ligand in these sites instead of the correct binding site. Again, interactive mode is the recommended solution (and an optimal search model can help too). Note that unexpected ordered buffer components may also lead to false positives; this is especially problematic for fragments.
- Poor or incomplete difference density. In some cases, a particularly flexible ligand may not fit entirely into difference density; this is not uncommon or necessarily problematic, but it can reduce the CC below the cutoff required to continue. More common is poor density quality for the entire molecule; this can be caused by an incompletely refined model, low resolution, or partial binding. In some cases, it means that the ligand is not bound at all. While reducing the CC cutoff may allow some structures to proceed, we recommend inspecting the density interactively, both before and after ligand placement. Also note that maximum entropy treatment of the difference map (`maxent=True`) may improve the fitting in borderline cases.
- Poor ligand geometry. In some cases eLBOW is unable to automatically determine the optimal geometry from the inputs. Running AM1 optimization may help for structures where the initial geometry is partially correct. However, in extreme cases the geometry will be too distorted for AM1 to work, and additional information on geometry is required. For this reason, we recommend supplying as rich a source of information as possible; in particular, if geometry from a crystal structure of the target compound is available, this should be included in the inputs.

Other notes about use

Speeding up execution: the bulk of the runtime is spent refining the model; the default protocol is very robust but not especially fast, and may be unnecessarily intensive for easy cases. You can switch to a faster protocol by specifying `quick_refine=True`, which will shorten both refinement steps from 6 to 3 cycles, and disable weight optimization.

Hydrogen atoms: these will be added prior to the final refinement step. Usually this will improve geometry somewhat (and also R-factors at high resolution); however, it has the side effect of making refinement significantly slower. Specify `after_ligand.hydrogens=False` to leave them off.

Rebuilding sidechains: sidechains truncated by a previous step (Sculptor or the pruning step), or missing in the input model, may optionally be restored after ligand placement by specifying `extend_model=True`.

Standalone tools: although most of the pipeline is built around pre-existing programs in Phenix, several new tools have been developed for this protocol and are also available as separate command-line programs:

- `mmtbx.select_best_starting_model`: given a data file and a collection of PDB files, calculate R-factors and (optionally) run rigid-body refinement to pick the best one for further refinement
- `mmtbx.prune_model`: remove protein atoms with poor fit to density.
- `mmtbx.apply_ncs_to_ligand`: place additional copies of the desired ligand using NCS operators between protein chains.
- `mmtbx.extend_sidechains`: rebuild missing protein sidechain atoms and perform quick real-space refinement.
- `mmtbx.validate_ligand`: check for possible fitting errors in a specific ligand.
- `phenix.sort_hetatms`: group heteroatoms (ligands, water, etc.) with the closest macromolecule(s) in the PDB file, similar to the procedure used at PDB deposition.

References

Automating crystallographic structure solution and refinement of protein-ligand complexes. N. Echols, N.W. Moriarty, H.E. Klei, P.V. Afonine, G. Bunkóczi, J.J. Headd, A.J. McCoy, R.D. Oeffner, R.J. Read, T.C. Terwilliger, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 70, 144-54 (2014).

- Automating crystallographic structure solution and refinement of protein-ligand complexes. N. Echols, N.W. Moriarty, H.E. Klei, P.V. Afonine, G. Bunkóczi, J.J. Headd, A.J. McCoy, R.D. Oeffner, R.J. Read, T.C. Terwilliger, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 70, 144-54 (2014).

Xtriage and Fest: automatic assessment of X-ray data and substructure structure factor estimation. P.H. Zwart, R.W. Grosse-Kunstleve, and P.D. Adams. *CCP4 Newsletter Winter, Contribution 7* (2005).

- Xtriage and Fest: automatic assessment of X-ray data and substructure structure factor estimation. P.H. Zwart, R.W. Grosse-Kunstleve, and P.D. Adams. *CCP4 Newsletter Winter, Contribution 7* (2005).

Improvement of molecular-replacement models with Sculptor. G. Bunkóczi, and R.J. Read. *Acta Crystallogr D Biol Crystallogr* 67, 303-12 (2011).

- Improvement of molecular-replacement models with Sculptor. G. Bunkóczi, and R.J. Read. *Acta Crystallogr D Biol Crystallogr* 67, 303-12 (2011).

Phaser crystallographic software. A.J. McCoy, R.W. Grosse-Kunstleve, P.D. Adams, M.D. Winn, L.C. Storoni, and R.J. Read. *J Appl Crystallogr* 40, 658-674 (2007).

- Phaser crystallographic software. A.J. McCoy, R.W. Grosse-Kunstleve, P.D. Adams, M.D. Winn, L.C. Storoni, and R.J. Read. *J Appl Crystallogr* 40, 658-674 (2007).

electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 65, 1074-80 (2009).

- electronic Ligand Builder and Optimization Workbench (eLBOW): a tool for ligand coordinate and restraint generation. N.W. Moriarty, R.W. Grosse-Kunstleve, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 65, 1074-80 (2009).

Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

- Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. *Acta Cryst. D64*, 61-69 (2008).

- Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. *Acta Cryst. D64*, 61-69 (2008).

Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006).

- Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006).

MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. *Protein Science* 27, 293-315 (2018).

- MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. *Protein Science* 27, 293-315 (2018).

List of all available keywords

input_filesseq_file = None Sequence of the crystallized protein. This should correspond to the search model, i.e. if the model contains two copies of the protein, the sequence file should as well.model = None Search model for MR (or starting model for refinement). Water molecules will be removed. Additional ligands may be present but their occupancy will be set to zero by Phaser unless you specify keep_hetatms=True.ligand = None Ligand description in chemical format (e.g. SMILES)ligand_smiles = None SMILES string for ligandligand_pdb = None PDB file with ligand geometry and/or atom namesligand_code = None Chemical components code for ligand (if no file is provided), for molecules already present in the PDB.extra_cif = None Additional CIF file(s) for ligands already present in the structure.reference_structure = None If specified, phenix.find_alt_orig_sym_mate will be applied to map the solution to the reference structure. Primarily intended for testing, but also useful in cases where a common reference frame is desired.xray_datawavelength = None file_name = None labels = None high_resolution = None low_resolution = None twin_law = None outliers_rejection = True Remove basic wilson outliers , extreme wilson outliers , and beamstop shadow outliersfrench_wilson_scale = True sigma_fobs_rejection_criterion = None sigma_lob_rejection_criterion = None ignore_all_zeros = True force_anomalous_flag_to_be_equal_to = None convert_to_non_anomalous_if_ratio_pairs_lone_less_than_threshold = 0.5 phase_labels = None

space_group = None unit_cell = None unmerged_data = None unmerged_labels = None
 french_wilsonmax_bins = 60 Maximum number of resolution binsmin_bin_size = 40 Minimum number of
 reflections per binr_free_flagsfile_name = None This is normally the same as the file containing Fobs and is
 usually selected automatically.label = None required = True Specify if free-r flags are must be present (or else
 generated)test_flag_value = None This value is usually selected automatically - do not change unless you
 really know what you're doing!ignore_r_free_flags = False Use all reflections in refinement (work and
 test)disable_suitability_test = False ignore_pdb_hexdigest = False If True, disables safety check based on
 MD5 hexdigests stored in PDB files produced by previous runs.generate = False Generate R-free flags (if not
 available in input files)fraction = 0.1 max_free = 2000 lattice_symmetry_max_delta = 5 use_lattice_symmetry
 = True use_dataman_shells = False Used to avoid biasing of the test set by certain types of
 non-crystallographic symmetry.n_shells = 20 reference_datafile_name = None labels = None outputprefix =
 None directory = None make_subdir = True If True, the program will create a subdirectory for the current
 run.directory_number = None verbose = False job_title = None Job title in PHENIX GUI, not used on
 command linecoot_absolute_paths = True export_for_web = False runtimenproc = 1 Number of processors to
 use for parallelized steps (including LigandFit, AutoBuild, and phenix.refine with weight optimization). If set to
 Auto, the program will use as many CPUs as are available, minus the current load average.ligand_copies =
 None Number of copies of the ligand to place. By default, this is the same as the number of copies as the MR
 search model.skip_xtriage = False clean_models = True Run various cleanup on input model file(s) before
 MR or refinement.mr = True If True, molecular replacement will be run to place the model. If Auto, the
 program will try rigid-body refinement first, then run Phaser if the R-free is too high. Otherwise, rigid-body
 refinement will be used.skip_mr = False Alias for mr=False (deprecated)prune = Auto Prune the model after
 refinement to remove residues and sidechains in poor density.build = Auto Run AutoBuild after initial
 refinement. By default, this will be done if R-free is greater than the max_r_free cutoff.skip_ligand = False
 refine_after_fitting = True Run phenix.refine a final time after ligand placeme...

- input_filesseq_file = None Sequence of the crystallized protein. This should correspond to the search
 model, i.e. if the model contains two copies of the protein, the sequence file should as well.model = None
 Search model for MR (or starting model for refinement). Water molecules will be removed. Additional
 ligands may be present but their occupancy will be set to zero by Phaser unless you specify
 keep_hetatms=True.ligand = None Ligand description in chemical format (e.g. SMILES)ligand_smiles =
 None SMILES string for ligandligand_pdb = None PDB file with ligand geometry and/or atom
 namesligand_code = None Chemical components code for ligand (if no file is provided), for molecules
 already present in the PDB.extra_cif = None Additional CIF file(s) for ligands already present in the
 structure.reference_structure = None If specified, phenix.find_alt_orig_sym_mate will be applied to map
 the solution to the reference structure. Primarily intended for testing, but also useful in cases where a
 common reference frame is desired.xray_datawavelength = None file_name = None labels = None
 high_resolution = None low_resolution = None twin_law = None outliers_rejection = True Remove basic
 wilson outliers , extreme wilson outliers , and beamstop shadow outliersfrench_wilson_scale = True
 sigma_fobs_rejection_criterion = None sigma_iobs_rejection_criterion = None ignore_all_zeros = True
 force_anomalous_flag_to_be_equal_to = None
 convert_to_non_anomalous_if_ratio_pairs_lone_less_than_threshold = 0.5 phase_labels = None
 space_group = None unit_cell = None unmerged_data = None unmerged_labels = None
 french_wilsonmax_bins = 60 Maximum number of resolution binsmin_bin_size = 40 Minimum number of
 reflections per binr_free_flagsfile_name = None This is normally the same as the file containing Fobs and
 is usually selected automatically.label = None required = True Specify if free-r flags are must be present
 (or else generated)test_flag_value = None This value is usually selected automatically - do not change
 unless you really know what you're doing!ignore_r_free_flags = False Use all reflections in refinement
 (work and test)disable_suitability_test = False ignore_pdb_hexdigest = False If True, disables safety
 check based on MD5 hexdigests stored in PDB files produced by previous runs.generate = False

Generate R-free flags (if not available in input files)fraction = 0.1 max_free = 2000

lattice_symmetry_max_delta = 5 use_lattice_symmetry = True use_dataman_shells = False Used to avoid biasing of the test set by certain types of non-crystallographic symmetry.n_shells = 20

reference_datafile_name = None labels = None

- seq_file = None Sequence of the crystallized protein. This should correspond to the search model, i.e. if the model contains two copies of the protein, the sequence file should as well.

- model = None Search model for MR (or starting model for refinement). Water molecules will be removed. Additional ligands may be present but their occupancy will be set to zero by Phaser unless you specify keep_hetatms=True.

- ligand = None Ligand description in chemical format (e.g. SMILES)

- ligand_smiles = None SMILES string for ligand

- ligand_pdb = None PDB file with ligand geometry and/or atom names

- ligand_code = None Chemical components code for ligand (if no file is provided), for molecules already present in the PDB.

- extra_cif = None Additional CIF file(s) for ligands already present in the structure.

- reference_structure = None If specified, phenix.find_alt_orig_sym_mate will be applied to map the solution to the reference structure. Primarily intended for testing, but also useful in cases where a common reference frame is desired.

- xray_datawavelength = None file_name = None labels = None high_resolution = None low_resolution = None twin_law = None outliers_rejection = True Remove basic wilson outliers , extreme wilson outliers , and beamstop shadow outliersfrench_wilson_scale = True sigma_fobs_rejection_criterion = None sigma_iobs_rejection_criterion = None ignore_all_zeros = True force_anomalous_flag_to_be_equal_to = None convert_to_non_anomalous_if_ratio_pairs_lone_less_than_threshold = 0.5 phase_labels = None space_group = None unit_cell = None unmerged_data = None unmerged_labels = None french_wilsonmax_bins = 60 Maximum number of resolution binsmin_bin_size = 40 Minimum number of reflections per binr_free_flagsfile_name = None This is normally the same as the file containing Fobs and is usually selected automatically.label = None required = True Specify if free-r flags are must be present (or else generated)test_flag_value = None This value is usually selected automatically - do not change unless you really know what you're doing!ignore_r_free_flags = False Use all reflections in refinement (work and test)disable_suitability_test = False ignore_pdb_hexdigest = False If True, disables safety check based on MD5 hexdigests stored in PDB files produced by previous runs.generate = False

Generate R-free flags (if not available in input files)fraction = 0.1 max_free = 2000

lattice_symmetry_max_delta = 5 use_lattice_symmetry = True use_dataman_shells = False Used to avoid biasing of the test set by certain types of non-crystallographic symmetry.n_shells = 20

- wavelength = None

- file_name = None

- labels = None

- high_resolution = None

- low_resolution = None

- twin_law = None

- outliers_rejection = True Remove basic wilson outliers , extreme wilson outliers , and beamstop shadow outliers

- french_wilson_scale = True

- `sigma_fobs_rejection_criterion` = None
- `sigma_iobs_rejection_criterion` = None
- `ignore_all_zeros` = True
- `force_anomalous_flag_to_be_equal_to` = None
- `convert_to_non_anomalous_if_ratio_pairs_lone_less_than_threshold` = 0.5
- `phase_labels` = None
- `space_group` = None
- `unit_cell` = None
- `unmerged_data` = None
- `unmerged_labels` = None
- `french_wilson` `max_bins` = 60 Maximum number of resolution bins `min_bin_size` = 40 Minimum number of reflections per bin
- `max_bins` = 60 Maximum number of resolution bins
- `min_bin_size` = 40 Minimum number of reflections per bin
- `r_free_flagsfile_name` = None This is normally the same as the file containing Fobs and is usually selected automatically. `label` = None required = True Specify if free-r flags are must be present (or else generated) `test_flag_value` = None This value is usually selected automatically - do not change unless you really know what you're doing! `ignore_r_free_flags` = False Use all reflections in refinement (work and test) `disable_suitability_test` = False `ignore_pdb_hexdigest` = False If True, disables safety check based on MD5 hexdigests stored in PDB files produced by previous runs. `generate` = False Generate R-free flags (if not available in input files) `fraction` = 0.1 `max_free` = 2000 `lattice_symmetry_max_delta` = 5 `use_lattice_symmetry` = True `use_dataman_shells` = False Used to avoid biasing of the test set by certain types of non-crystallographic symmetry. `n_shells` = 20
- `file_name` = None This is normally the same as the file containing Fobs and is usually selected automatically.
- `label` = None
- `required` = True Specify if free-r flags are must be present (or else generated)
- `test_flag_value` = None This value is usually selected automatically - do not change unless you really know what you're doing!
- `ignore_r_free_flags` = False Use all reflections in refinement (work and test)
- `disable_suitability_test` = False
- `ignore_pdb_hexdigest` = False If True, disables safety check based on MD5 hexdigests stored in PDB files produced by previous runs.
- `generate` = False Generate R-free flags (if not available in input files)
- `fraction` = 0.1
- `max_free` = 2000
- `lattice_symmetry_max_delta` = 5
- `use_lattice_symmetry` = True
- `use_dataman_shells` = False Used to avoid biasing of the test set by certain types of non-crystallographic symmetry.

- `n_shells = 20`
- `reference_datafile_name = None` `labels = None`
- `file_name = None`
- `labels = None`
- `outputprefix = None` `directory = None` `make_subdir = True` If True, the program will create a subdirectory for the current run. `directory_number = None` `verbose = False` `job_title = None` Job title in PHENIX GUI, not used on command line. `coot_absolute_paths = True` `export_for_web = False`
- `prefix = None`
- `directory = None`
- `make_subdir = True` If True, the program will create a subdirectory for the current run.
- `directory_number = None`
- `verbose = False`
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `coot_absolute_paths = True`
- `export_for_web = False`
- `runtimemproc = 1` Number of processors to use for parallelized steps (including LigandFit, AutoBuild, and phenix.refine with weight optimization). If set to Auto, the program will use as many CPUs as are available, minus the current load average. `ligand_copies = None` Number of copies of the ligand to place. By default, this is the same as the number of copies as the MR search model. `skip_xtriage = False` `clean_models = True` Run various cleanup on input model file(s) before MR or refinement. `mr = True` If True, molecular replacement will be run to place the model. If Auto, the program will try rigid-body refinement first, then run Phaser if the R-free is too high. Otherwise, rigid-body refinement will be used. `skip_mr = False` Alias for `mr=False` (deprecated). `prune = Auto` Prune the model after refinement to remove residues and sidechains in poor density. `build = Auto` Run AutoBuild after initial refinement. By default, this will be done if R-free is greater than the `max_r_free` cutoff. `skip_ligand = False` `refine_after_fitting = True` Run phenix.refine a final time after ligand placement. If not True, new maps will be calculated instead. `refine_ligand_occupancy = False` Refine group occupancy for each placed ligand. Recommended for fragments. `mr_program = *Auto` phaser mrage Program to use for molecular replacement. Will use Phaser by default if a PDB file is included in input, otherwise will use phaser. `mrage` (sequence required). `method = *ligandfit` Program to use for ligand placement. Currently only LigandFit is supported, but other programs will be added in the future. `fill_maps = True` `extend_model = False` If enabled, incomplete sidechains will be rebuilt and fit to density if possible after ligand placement. `validate = True` `sort_hetatms = True` Rearrange non-polymer heteroatoms (waters and ligands) to have the same chain ID and asymmetric unit as the nearest polymer chain. `run_command = None` `ncs_for_ligands = True` Use NCS operators to place additional ligands if LigandFit was not able to find all copies with acceptable CC. `interactive = False` Run program interactively with manual editing in Coot between refinement (or autobuilding) and ligand-fitting steps. Requires that Coot be installed and present on the path. `stop_if_r_free_greater_than = 0.5` Cutoff for continuing the program after initial round of refinement - usually an R-free above 0.5 indicates that the model contains serious defects which cannot be automatically fixed (or is simply wrong altogether). Set to None to disable this check. `auto_resolution = False` If True, and unmerged intensities are included as input, the merging statistics will be used to determine appropriate resolution cutoffs for different steps in the pipeline (starting with a conservative cutoff for the initial refinement, and extending to higher resolution for later steps).

- `nproc = 1` Number of processors to use for parallelized steps (including LigandFit, AutoBuild, and phenix.refine with weight optimization). If set to Auto, the program will use as many CPUs as are available, minus the current load average.
- `ligand_copies = None` Number of copies of the ligand to place. By default, this is the same as the number of copies as the MR search model.
- `skip_xtriage = False`
- `clean_models = True` Run various cleanup on input model file(s) before MR or refinement.
- `mr = True` If True, molecular replacement will be run to place the model. If Auto, the program will try rigid-body refinement first, then run Phaser if the R-free is too high. Otherwise, rigid-body refinement will be used.
- `skip_mr = False` Alias for `mr=False` (deprecated)
- `prune = Auto` Prune the model after refinement to remove residues and sidechains in poor density.
- `build = Auto` Run AutoBuild after initial refinement. By default, this will be done if R-free is greater than the `max_r_free` cutoff.
- `skip_ligand = False`
- `refine_after_fitting = True` Run phenix.refine a final time after ligand placement. If not True, new maps will be calculated instead.
- `refine_ligand_occupancy = False` Refine group occupancy for each placed ligand. Recommended for fragments.
- `mr_program = *Auto` phaser mrange Program to use for molecular replacement. Will use Phaser by default if a PDB file is included in input, otherwise will use phaser.mrange (sequence required).
- `method = *ligandfit` Program to use for ligand placement. Currently only LigandFit is supported, but other programs will be added in the future.
- `fill_maps = True`
- `extend_model = False` If enabled, incomplete sidechains will be rebuilt and fit to density if possible after ligand placement.
- `validate = True`
- `sort_hetatms = True` Rearrange non-polymer heteroatoms (waters and ligands) to have the same chain ID and asymmetric unit as the nearest polymer chain.
- `run_command = None`
- `ncs_for_ligands = True` Use NCS operators to place additional ligands if LigandFit was not able to find all copies with acceptable CC.
- `interactive = False` Run program interactively with manual editing in Coot between refinement (or autobuilding) and ligand-fitting steps. Requires that Coot be installed and present on the path.
- `stop_if_r_free_greater_than = 0.5` Cutoff for continuing the program after initial round of refinement - usually an R-free above 0.5 indicates that the model contains serious defects which cannot be automatically fixed (or is simply wrong altogether). Set to None to disable this check.
- `auto_resolution = False` If True, and unmerged intensities are included as input, the merging statistics will be used to determine appropriate resolution cutoffs for different steps in the pipeline (starting with a conservative cutoff for the initial refinement, and extending to higher resolution for later steps).
- `xtriagemin_i_over_sigma = None` `min_cc_one_half = None` `max_r_merge = None` `max_r_meas = None` `min_completeness = None`

- min_i_over_sigma = None
- min_cc_one_half = None
- max_r_merge = None
- max_r_meas = None
- min_completeness = None
- prepare_modelOptions for preparing input model(s) for MR or refinement
 sculpt = True Run Sculptor to homogenize sidechains before MR or refinement
 remove_waters = True Remove all water molecules (HOH)
 remove_hydrogens = True Remove explicit hydrogen atoms
 remove_alt_confs = True Remove alternate conformations
 convert_somet_to_met = True Change MSE residues to MET
 convert_to_isotropic = True Convert atoms to anisotropic
 reset_occupancies = True Set occupancies to 1.0
 remove_ligands = False Remove all ligands
 reset_hetatm_flag = False Change HETATM records to ATOM
 sort_first = True
- sculpt = True Run Sculptor to homogenize sidechains before MR or refinement
- remove_waters = True Remove all water molecules (HOH)
- remove_hydrogens = True Remove explicit hydrogen atoms
- remove_alt_confs = True Remove alternate conformations
- convert_somet_to_met = True Change MSE residues to MET
- convert_to_isotropic = True Convert atoms to anisotropic
- reset_occupancies = True Set occupancies to 1.0
- remove_ligands = False Remove all ligands
- reset_hetatm_flag = False Change HETATM records to ATOM
- sort_first = True
- select_modelModel selection options
 d_min = None rigid_body_refine = True optimize_b_factors = False
 max_cell_angle_rmsd = 1.0 max_cell_edge_rmsd = 1.0 max_r_free = 0.4
- d_min = None
- rigid_body_refine = True
- optimize_b_factors = False
- max_cell_angle_rmsd = 1.0
- max_cell_edge_rmsd = 1.0
- max_r_free = 0.4
- phaserd_min = 2.0 Resolution limit for running Phaser.
 copies = Auto Number of copies to search for. If Auto, Xtriage will be used to guess based on the expected solvent content.
 identity = None Sequence identity of the search model to the target structure. If None and rmsd is also None, perfect sequence identity will be assumed.
 rmsd = None RMSD of the search model to the target structure. If not specified, the sequence identity will be used instead.
 sg_alt = none hand *all Choice of alternate space groups to try.
 ignore_packing = False pack_cutoff = 10 keep_hetatms = True If True, HETATMs will not have their occupancies reset to zero.
 force_accept_composition = False tncs = True
- d_min = 2.0 Resolution limit for running Phaser.
- copies = Auto Number of copies to search for. If Auto, Xtriage will be used to guess based on the expected solvent content.
- identity = None Sequence identity of the search model to the target structure. If None and rmsd is also None, perfect sequence identity will be assumed.

- rmsd = None RMSD of the search model to the target structure. If not specified, the sequence identity will be used instead.
- sg_alt = none hand *all Choice of alternate space groups to try.
- ignore_packing = False
- pack_cutoff = 10
- keep_hetatms = True If True, HETATMs will not have their occupancies reset to zero.
- force_accept_composition = False
- tnccs = True
- prune_modelresolution_factor = 1/4. Map grid spacing (multiplied times d_min).sidechains = True Remove poor sidechainsmainchain = False Remove entire residues in poor densitymin_backbone_2fofc = 0.8 Minimum 2mFo-DFc sigma level at C-alpha to keep. Residues with C-alpha in density below this cutoff will be deleted.min_backbone_fofc = -3.0 Maximum mFo-DFc sigma level at C-alpha to keep. Residues with C-alpha in difference density below this cutoff will be deleted.min_sidechain_2fofc = 0.6 Minimum mean 2mFo-DFc sigma level for sidechain atoms to keep. Residues with sidechains below this level will be truncated.max_sidechain_fofc = -2.8 Maximum mean 2mFo-DFc sigma level for sidechain atoms to keep. Residues with sidechains below this level will be truncated.min_cc = 0.7 Minimum overall CC for entire residue to keep.min_cc_sidechain = 0.6 Minimum overall CC for sidechains to keep.min_fragment_size = 3 Minimum fragment size to keep. Fragments smaller than this will be deleted in the final step (based on the assumption that the adjacent residues were already removed). Set this to None to prevent fragment filtering.check_cgammma = True Check for poor density at the C-gamma atom for long sidechains. Useful in cases where the terminal atoms may have been misfit into nearby density.
- resolution_factor = 1/4. Map grid spacing (multiplied times d_min).
- sidechains = True Remove poor sidechains
- mainchain = False Remove entire residues in poor density
- min_backbone_2fofc = 0.8 Minimum 2mFo-DFc sigma level at C-alpha to keep. Residues with C-alpha in density below this cutoff will be deleted.
- min_backbone_fofc = -3.0 Maximum mFo-DFc sigma level at C-alpha to keep. Residues with C-alpha in difference density below this cutoff will be deleted.
- min_sidechain_2fofc = 0.6 Minimum mean 2mFo-DFc sigma level for sidechain atoms to keep. Residues with sidechains below this level will be truncated.
- max_sidechain_fofc = -2.8 Maximum mean 2mFo-DFc sigma level for sidechain atoms to keep. Residues with sidechains below this level will be truncated.
- min_cc = 0.7 Minimum overall CC for entire residue to keep.
- min_cc_sidechain = 0.6 Minimum overall CC for sidechains to keep.
- min_fragment_size = 3 Minimum fragment size to keep. Fragments smaller than this will be deleted in the final step (based on the assumption that the adjacent residues were already removed). Set this to None to prevent fragment filtering.
- check_cgammma = True Check for poor density at the C-gamma atom for long sidechains. Useful in cases where the terminal atoms may have been misfit into nearby density.
- autobuildd_min = None Resolution for running AutoBuildmax_r_free = 0.3 R-free cutoff for deciding whether to run AutoBuild when build=Auto.max_frac_missing = 0.05 quick = False Run AutoBuild in quick mode. Inferior results, but a huge time-saver.rebuild_in_place = Auto Controls whether AutoBuild will only rebuild the existing model, without adding or removing atoms, or build an entirely new

`model.include_input_model = True` Specifies whether to incorporate parts of the input model when building a new model (i.e. `rebuild_in_place=False`).
`phil_file = None` Parameters file for AutoBuild.

- `d_min = None` Resolution for running AutoBuild
- `max_r_free = 0.3` R-free cutoff for deciding whether to run AutoBuild when `build=Auto`.
- `max_frac_missing = 0.05`
- `quick = False` Run AutoBuild in quick mode. Inferior results, but a huge time-saver.
- `rebuild_in_place = Auto` Controls whether AutoBuild will only rebuild the existing model, without adding or removing atoms, or build an entirely new model.
- `include_input_model = True` Specifies whether to incorporate parts of the input model when building a new model (i.e. `rebuild_in_place=False`).
- `phil_file = None` Parameters file for AutoBuild.
- `refinequick_refine = False` Use a shorter refinement protocol: fewer cycles and no weight optimization. Suitable for isomorphous structures.
`d_min = None` `ncs_type = *Auto` torsion cartesian `None` `twin_law = Auto` `phil_file = None` File containing additional parameters for phenix.refine.
`after_mrcycles = 6` Number of macrocycles of refinement to run.
`optimize_weights = False` Perform weight optimization (geometry and ADP restraints).
`sites = True` Refine with individual_sites strategy.
`real_space = Auto` `rigid_body = Auto` Run rigid-body refinement. By default this is only run if MR was not used to place the model.
`adp_type = *Auto` `aniso iso` `tls = Auto` `update_waters = Auto` Add/remove waters automatically. These will not interfere with ligand placement, since the fitting programs ignore them.
`anneal = False` Run simulated annealing. May increase radius of convergence, especially for rough initial models, but significantly adds to runtime.
`ready_set = False` Run `ready_set` to generate additional parameter files for phenix.refine. Hydrogens are not added at this stage.
`after_ligandoptimize_weights = Auto` Perform a grid search to optimize the XYZ and ADP restraint weights. Not usually necessary if `d_min <= 1.75`.
`hydrogens = True` Add hydrogens to model with phenix.ready_set. If Auto, the decision will be made based on resolution, the presence of hydrogens in the input model, or a mention of riding hydrogens in the PDB header.
`cycles = 6` Number of macrocycles.
`tls = Auto` Refine TLS groups.
`adp_type = *Auto` `aniso iso` B-factor parameterization
`waters = Auto` Run ordered solvent update - normally resolution-dependent.
`ions = None` List of element symbols for ion picking (separated by commas or spaces).
`outlier_rejection = True`
- `quick_refine = False` Use a shorter refinement protocol: fewer cycles and no weight optimization. Suitable for isomorphous structures.
- `d_min = None`
- `ncs_type = *Auto` torsion cartesian `None`
- `twin_law = Auto`
- `phil_file = None` File containing additional parameters for phenix.refine.
- `after_mrcycles = 6` Number of macrocycles of refinement to run.
`optimize_weights = False` Perform weight optimization (geometry and ADP restraints).
`sites = True` Refine with individual_sites strategy.
`real_space = Auto` `rigid_body = Auto` Run rigid-body refinement. By default this is only run if MR was not used to place the model.
`adp_type = *Auto` `aniso iso` `tls = Auto` `update_waters = Auto` Add/remove waters automatically. These will not interfere with ligand placement, since the fitting programs ignore them.
`anneal = False` Run simulated annealing. May increase radius of convergence, especially for rough initial models, but significantly adds to runtime.
`ready_set = False` Run `ready_set` to generate additional parameter files for phenix.refine. Hydrogens are not added at this stage.
- `cycles = 6` Number of macrocycles of refinement to run.

- `optimize_weights = False` Perform weight optimization (geometry and ADP restraints).
- `sites = True` Refine with individual_sites strategy.
- `real_space = Auto`
- `rigid_body = Auto` Run rigid-body refinement. By default this is only run if MR was not used to place the model.
- `adp_type = *Auto aniso iso`
- `tls = Auto`
- `update_waters = Auto` Add/remove waters automatically. These will not interfere with ligand placement, since the fitting programs ignore them.
- `anneal = False` Run simulated annealing. May increase radius of convergence, especially for rough initial models, but significantly adds to runtime.
- `ready_set = False` Run ready_set to generate additional parameter files for phenix.refine. Hydrogens are not added at this stage.
- `after_ligandoptimize_weights = Auto` Perform a grid search to optimize the XYZ and ADP restraint weights. Not usually necessary if `d_min <= 1.75`.
- `hydrogens = True` Add hydrogens to model with `phenix.ready_set`. If Auto, the decision will be made based on resolution, the presence of hydrogens in the input model, or a mention of riding hydrogens in the PDB header.
- `cycles = 6` Number of macrocycles.
- `tls = Auto` Refine TLS groups.
- `adp_type = *Auto aniso iso` B-factor parameterization
- `waters = Auto` Run ordered solvent update - normally resolution-dependent.
- `ions = None` List of element symbols for ion picking (separated by commas or spaces).
- `outlier_rejection = True`
- `elbowligand_id = Auto` Three-letter residue code to use in output PDB file. If not specified, will default to `LIG`.
- `optimize = False` Run AM1 quantum-mechanical optimization on ligand geometry.
- `initial_geometry = None` Starting geometry for AM1 optimization.
- `final_geometry = None` Final geometry for output ligand

- `keep_input_restraints = True` If True, and the input files include pre-calculated restraints for the target ligand, eLBOW will propagate these restraints instead of generating new ones.
- `template_pdb = None` PDB file containing atom names to use for ligand. If not specified (and not present in the input ligand file), the atom names will be generated automatically.
- `ligandfitd_min = 1.5` Resolution cutoff for ligand fitting. For atomic resolution datasets, LigandFit usually works better at slightly lower resolution.
- `map_type = *mFo-DFc fo_minus_fo` Type of map to use for ligand fitting. The default is to use the standard mFo-DFc difference map, but if a reference dataset is supplied, an isomorphous difference map may be used instead.
- `quick = False` Run LigandFit in quick (single-job) mode. Not recommended.
- `aggressive = False` Adjusts the LigandFit parameters to search for more conformations, at the expense of increased runtime.
- `min_cc = None`
- `conformers = 20`
- `n_group_search = 5`
- `ligand_near_residue = None`
- `ligand_near_chain = None`
- `fixed_ligand = False`
- `min_ligand_cc_keep = 0.7` Minimum CC to consider a ligand placement correct. Ligands with at least this CC will be incorporated into the current model for refinement.
- `min_overall_cc_stop = 0.75` Minimum overall CC (for all ligands) to consider the structure complete.
- `maxent = False`
- `first_only = False` If True, LigandFit will place a single ligand, and the remaining copies will be added based solely on NCS operators.
- `fobs_minus_fobs_multiscale = False` `d_min = None` `max_r_factor = 0.2` Maximum allowed R-factor for comparison of the target and reference datasets.
- `relative_length_tolerance = 0.01` Tolerance for deviation in unit cell lengths between target and reference datasets.
- `absolute_angle_tolerance = 1` Tolerance for deviation in unit cell angles between target and reference datasets.
- `multiscale = False`
- `d_min = None`
- `max_r_factor = 0.2` Maximum allowed R-factor for comparison of the target and reference datasets.

- `relative_length_tolerance = 0.01` Tolerance for deviation in unit cell lengths between target and reference datasets.
- `absolute_angle_tolerance = 1.` Tolerance for deviation in unit cell angles between target and reference datasets.
- `ligand_ncsParameters` for placing missing ligands using NCS operators between protein chains.
`d_min = 2.5`
`max_rmsd = 2.0`
`min_cc = 0.7`
`min_cc_reference = 0.85`
`min_2fofc = 1.0`
`min_dist_center = 5`
`remove_clashing_atoms = True`
`clash_cutoff = 2.0`
`write_sampled_pdb = False`
- `d_min = 2.5`
- `max_rmsd = 2.0`
- `min_cc = 0.7`
- `min_cc_reference = 0.85`
- `min_2fofc = 1.0`
- `min_dist_center = 5`
- `remove_clashing_atoms = True`
- `clash_cutoff = 2.0`
- `write_sampled_pdb = False`
- `extensionselection = None`
`build_hydrogens = Auto`
`max_atoms_missing = None`
`use_rotamers = True`
`anneal_residues = False`
`skip_rsr = False`
`output_model = None`
`output_map_coeffs = None`
- `selection = None`
- `build_hydrogens = Auto`
- `max_atoms_missing = None`
- `use_rotamers = True`
- `anneal_residues = False`
- `skip_rsr = False`
- `output_model = None`
- `output_map_coeffs = None`
- `sortingpreserve_chain_id = False` The default behavior is to group heteroatoms with the nearest macromolecule chain, whose ID is inherited. This parameter disables the change of chain ID, and preserves the original chain ID.
`waters_only = False` Rearrange waters, but leave all other ligands alone.
`sort_waters_by = None` *b_iso Ordering of waters - by default it will sort them by the isotropic B-factor.
`set_hetatom_record = True` Convert ATOM to HETATM where appropriate.
`ignore_selection = None` Selection of atoms to skip. Any residue group which overlaps with this selection will be preserved with the original chain ID and numbering.
`renumber = True` Renumber heteroatoms once they are in new chains.
`sequential_numbering = True` If True, the heteroatoms will be renumbered starting from the next available residue number after the end of the associated macromolecule chain. Otherwise, numbering will start from 1.
`distance_cutoff = 6.0` Cutoff for identifying nearby macromolecule chains. This should be kept relatively small for speed reasons, but it may miss waters that are far out in solvent channels.
`remove_waters_outside_radius = False` Remove waters more than the specified distance cutoff to the nearest polymer chain (to avoid PDB complaints).
`loose_chain_id = X` Chain ID assigned to heteroatoms that can't be mapped to a nearby macromolecule chain.
`model_skip_expand_with_mtrix = False` If set to false then expand the model into NCS copies using matrix (MTRIX) records present in PDB file

- `preserve_chain_id = False` The default behavior is to group heteroatoms with the nearest macromolecule chain, whose ID is inherited. This parameter disables the change of chain ID, and preserves the original chain ID.
- `waters_only = False` Rearrange waters, but leave all other ligands alone.
- `sort_waters_by = None` *b_iso Ordering of waters - by default it will sort them by the isotropic B-factor.
- `set_hetatm_record = True` Convert ATOM to HETATM where appropriate.
- `ignore_selection = None` Selection of atoms to skip. Any residue group which overlaps with this selection will be preserved with the original chain ID and numbering.
- `renumber = True` Renumber heteroatoms once they are in new chains.
- `sequential_numbering = True` If True, the heteroatoms will be renumbered starting from the next available residue number after the end of the associated macromolecule chain. Otherwise, numbering will start from 1.
- `distance_cutoff = 6.0` Cutoff for identifying nearby macromolecule chains. This should be kept relatively small for speed reasons, but it may miss waters that are far out in solvent channels.
- `remove_waters_outside_radius = False` Remove waters more than the specified distance cutoff to the nearest polymer chain (to avoid PDB complaints).
- `loose_chain_id = X` Chain ID assigned to heteroatoms that can't be mapped to a nearby macromolecule chain.
- `model_skip_expand_with_mtrix = False` If set to false then expand the model into NCS copies using matrix (MTRIX) records present in PDB file
- `testingParameters` for testing and benchmarking the program - not intended for general use. `pdb_id = None` If specified, the program will download all relevant data including search model and SMILES string from the PDB, and exit.
- `pdb_id = None` If specified, the program will download all relevant data including search model and SMILES string from the PDB, and exit.

Automated ligand fitting with LigandFit

ligandfit.html

Contents

- Author(s)
- Purpose
- Purpose of the LigandFit Wizard
- Usage
- How the LigandFit Wizard works
- How to run the LigandFit Wizard
- What the LigandFit wizard needs to run
- Specifying which columns of data to use from input data files
- Output files from LigandFit
- Running from a parameters file
- Examples
- Sample command_line inputs
- R-free flags and test set
- Using LigandFit with an mrc map as input
- What to do next after LigandFit
- Possible Problems
- Specific limitations and problems
- Literature
- List of all available keywords

Author(s)

- LigandFit Wizard: Tom Terwilliger
- PHENIX GUI and PDS Server: Nigel W. Moriarty
- RESOLVE: Tom Terwilliger

Purpose

Purpose of the LigandFit Wizard

The LigandFit Wizard carries out fitting of flexible ligands to electron density maps. It can be used with X-ray or cryo-EM data.

Usage

The LigandFit Wizard can be run from the PHENIX GUI, from the command-line, and from parameters files. All three versions are identical except in the way that they take commands from the user. See Using the

PHENIX Wizards for details of how to run a Wizard. The command-line version will be described here.

How the LigandFit Wizard works

The LigandFit wizard provides a command-line and graphical user interface allowing the user to identify a datafile containing crystallographic structure factor information, an optional PDB file with a partial model of the structure without the ligand, and a PDB file containing the ligand to be fit (in an allowed but arbitrary conformation).

The wizard checks the data files for consistency and then calls RESOLVE to carry out the fitting of the ligand into the electron-density map. The best map to use is usually a 2Fo-Fc map from phenix.refine. You can also have LigandFit calculate a difference map, with $F = F_P - F_C$. It can also be an Fobs map (calculated from F_P with phases PHIC from the input partial model), or an arbitrary map, calculated with F_P PHI and optional FOM. If you supply an input partial model, then the region occupied by the partial model is flattened in the map used to fit the ligand, so that the ligand will normally not get placed in this region.

The ligand fitting is done by RESOLVE in a three-stage process. First, the largest contiguous region of density in the map not already occupied by the model is identified. The ligand will be placed in this density. (If desired, the location of the ligand can instead be defined by the user as near a certain residue or near specified coordinates.) Next, many possible placements of the largest rigid sub-fragments of the ligand are found within this region of high density. Third, each of these placements is taken as a starting point for fitting the remainder of the ligand. All these ligand fits are scored based on the fit to the density, and the best-fitting placement is written out.

The output of the wizard consists of a fitted ligand in PDB format and a summary of the quality of the fit. Multiple copies of a ligand can be fit to a single map in an automated fashion using the LigandFit wizard as well.

How to run the LigandFit Wizard

Running the LigandFit Wizard is easy. For example, from the command-line you can type:

```
phenix.ligandfit data=datafile.mtz model=partial_model.pdb ligand=ligand.pdb
```

The LigandFit Wizard will carry out ligand fitting of the ligand in ligand.pdb based on the structure factor amplitudes in datafile.mtz, calculating phases based on partial-model.pdb. All rotatable bonds will be identified and allowed to take stereochemically reasonable orientations.

What the LigandFit wizard needs to run

The ligandfit wizard needs:

- a datafile (w1.sca or data=w1.sca); this can be any format
- a PDB file with your model without ligand (model=partial.pdb; optional if your datafile contains map coefficients)
- a file with information about your ligand (ligand=side.pdb), or a 3-letter code for your ligand (ligand=ATP)

The ligand file can be a PDB file with 1 stereochemically acceptable conformation of your ligand. It can alternatively be a file containing a SMILES string, in which case the starting ligand conformation will be generated with the PHENIX elbow routine. It can also be a 3-letter code that specifies a ligand in the Chemical Components Dictionary of the PDB, in which case the ligand is taken from that dictionary with idealized geometry.

The command_line ligandfit interpreter will guess which file is your data file but you have to tell it which file is the model and which is the ligand.

Specifying which columns of data to use from input data files

If one or more of your data files has column names that the Wizard cannot identify automatically, you can specify them yourself. You will need to provide one column "name" for each expected column of data, with "None" for anything that is missing.

For example, if your data file data.mtz has columns FP SIGFP then you might specify

```
data=data.mtz
input_labels="FP SIGFP"
```

You can find out all the possible label strings in a data file that you might use by typing:

```
phenix.autosol display_labels=data.mtz # display all labels for data.mtz
```

You can specify many more parameters as well. See the list of keywords, defaults and descriptions at the end of this page and also general information about running Wizards at Using the PHENIX Wizards for how to do this. Some of the most common parameters are:

```
data=w1.sca # data file
partial_model=coords.pdb # starting model without ligand
ligand=ligand.pdb # any stereochemically allowed conformation of your ligand
resolution=3 # dmin of 3 A
quick=False # specify if you want to look hard for a good conformation
ligand_cc_min=0.75 # quit if the CC of ligand to map is 0.75 or better
number_of_ligands=3 # find 3 copies of the ligand
n_group_search=3 # try 3 different fragments of the ligand in initial search
ligand_start=side.pdb # build ligand superimposing on side.pdb
```

Output files from LigandFit

When you run LigandFit the output files will be in a subdirectory with your run number:

```
LigandFit_run_1_/ # subdirectory with results
```

- A summary file listing the results of the run and the other files produced:

```
LigandFit_summary.dat # overall summary
```

- A file that lists all parameters and knowledge accumulated by the Wizard during the run (some parts are binary and are not printed)

```
LigandFit_Facts.dat # all Facts about the run
```

- A warnings file listing any warnings about the run

```
LigandFit_warnings.dat # any warnings
```

- A PDB file with the fitted ligand (in this case the first copy of ligand number 1):

```
ligand_fit_1_1.pdb
```

- A log file with the fitting of the ligand:

```
ligand_1_1.log
```

- A log file with the fit of the ligand to the map:

```
ligand_cc_1_1.log
```

- Map coefficients for the map used for fitting:

```
resolve_map.mtz
```

Running from a parameters file

You can run phenix.ligandfit from a parameters file. When you run ligandfit a parameters file is written in the output directory. Then you can edit this file and run it with:

```
phenix.ligandfit my_ligandfit.eff
```

Examples

Sample command_line inputs

- Standard run of ligandfit (generate map from model and data file)

```
phenix.ligandfit w1.sca model=partial.pdb ligand=ATP \  
lig_map_type=fo-fc_difference_map
```

- Build into a map from pre-determined coefficients

```
phenix.ligandfit data=perfect.mtz \  
lig_map_type=pre_calculated_map_coeffs \  
model=partial.pdb ligand=NAD
```

- Quick run of ligandfit

```
phenix.ligandfit w1.sca model=partial.pdb ligand=ATP quick=True
```

- Run ligandfit using pre-calculated map coefficients from phenix.refine

If you refine a model with a command such as,

```
phenix.refine data.mtz partial.pdb
```

then you will end up with the refined model,

```
partial_refine_001.pdb
```

and a map coefficients file:

```
partial_refine_001_map_coeffs.mtz
```

You can then run ligandfit using the 2Fo-Fc map calculated from these map coefficients:

```
phenix.ligandfit data=partial_refine_001_map_coeffs.mtz \  
model=partial_refine_001.pdb ligand=NAD quick=True
```

or if you want to specify the coefficients explicitly you can add the column labels:

```
phenix.ligandfit data=partial_refine_001_map_coeffs.mtz \  
model=partial_refine_001.pdb ligand=GOL quick=True \  
input_labels="2FOFCWT PH2FOFCWT"
```

For a difference map from the same file you can say:

```
phenix.ligandfit data=partial_refine_001_map_coeffs.mtz \  
model=partial_refine_001.pdb ligand=AMP quick=True \  
input_labels="FOFCWT PHFOFCWT"
```

- Run ligandfit on a series of ligands specified in ligand_list.dat

```
phenix.ligandfit w1.sca model=partial.pdb \  
ligand=ligand_list.dat file_or_file_list=file_with_list_of_files
```

Note that you have to specify

```
file_or_file_list=file_with_list_of_files
```

or else the Wizard will try to interpret the contents of ligand_list.dat as a SMILES string. Here the "file_with_list_of_files" is a flag, not something you substitute with an actual file name. You use it just as listed above.

- Place ligand near residue 94 of chain "A" from partial.pdb

```
phenix.ligandfit w1.sca model=partial.pdb ligand=ADP \
  ligand_near_chain="A" ligand_near_res=92
```

- Use start.pdb as a template for some of the atoms in the ligand; build the remainder of the ligand, fixing the coordinates of the corresponding atoms:

```
phenix.ligandfit w1.sca model=partial.pdb ligand=GTP \
  ligand_start=start.pdb
```

NOTE: the file start.pdb must contain an entire rigid group of atoms (so that ligandfit can identify the position and orientation of at least one rigid part of the ligand.)

- Use NCS from the model and the first ligand fitted to guess the positions of NCS-related ligands:

```
phenix.ligandfit w1.sca model=partial.pdb ligand=GTP \
  ligands_from_ncs=True
```

R-free flags and test set

In Phenix the parameter test_flag_value sets the value of the test set that is to be free. Normally Phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix and you do not need to do anything special. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

Using LigandFit with an mrc map as input

You can run ligand fitting using an mrc (or ccp4) map file as input instead of using map coefficients. If you supply a map, it will be converted into map coefficients and then ligand fitting will proceed as usual. If you supply a model file, it will be trimmed to remove all atoms outside of the original boxed map before using it.

If you supply a map that has an origin offset (a boxed map), then the map will be converted to map coefficients that keep the density in the same place as the original map. Then the ligand will be fitted in the usual way.

If your map has an origin offset that is specified by an external origin, the external origin is normally converted into an origin offset as in the above case. Note that not all display programs can properly display maps with external origins.

What to do next after LigandFit

Normally the next step after LigandFit is refinement (with phenix.refine for X-ray data or phenix.real_space_refine for cryo-EM data) and validation, iterating with manual rebuilding.

If LigandFit fails to find a ligand, have a close look at your map to see whether there is clear density for a ligand. If not, then you should not try to fit a ligand. If there is clear density but LigandFit does not find it, try increasing the number of conformers of the ligand that are tried, try varying the resolution, and check to make sure that you are looking for the right ligand.

Possible Problems

In some cases ligand fitting results in an incomplete ligand (some atoms are missing). Here is a procedure you can try to complete the ligand:

- Add the following commands to in the GUI in the field called "Resolve commands" (use the search function to find the input field). Include the quotes just as in the example below:
- `"dist_min_offset 1" "density_off_final 1000" "acceptable_offset 2"`
- You may wish to set: `nbatch=20 conformers=0` (these may not affect the results in many cases)
- Run `ligandfit` and make sure the ligand contains all the input atoms.
- If not, take the complete ligand and superimpose it on the partial ligand that was fitted.

Specific limitations and problems

- The ligand to be searched for must have at least 3 atoms.
- The partial-model file must not have any atoms (other than waters, which are automatically removed) in the position where the ligand is to be built. If this file contains atoms other than waters in the position where the ligand is to be built, then you may wish to remove them before building the ligand.
- If your ligand file has more than one ligand in it (or copies of the same ligand with different residue numbers), they will be treated as a single ligand. Typically this will result in part of the ligand (i.e., just one of the component ligands) being fitted. Normally supply instead a single ligand in your ligand file (or multiple copies of your ligand with different conformations and all the same residue number if you are trying to specify allowed conformations.)
- If a ring in the ligand can have more than one conformation (e.g., chair or boat conformation) then you need to do separate runs for each conformation of the ring (rings are taken as fixed units in `LigandFit`).
- If you want to treat your entire ligand as a single entity (no groups), then use the command: `resolve_command_list="bond_tol 100"` (both sets of quotes needed in command line, in GUI put in the inner set of quotes).
- `LigandFit` ignores insertion codes, so if you specify a residue with `ligand_near_res`, only the residue number is used.
- The size of the asymmetric unit in the `SOLVE/RESOLVE` portion of the `LigandFit` wizard is limited by the memory in your computer and the binaries used. The Wizard is supplied with regular-size (`"", size=6`), giant (`"_giant", size=12`), huge (`"_huge", size=18`) and extra_huge (`"_extra_huge", size=36`). Larger-size versions can be obtained on request.
- The `LigandFit` Wizard can take most settings of most space groups, however it can only use the hexagonal setting of rhombohedral space groups (eg., `#146 R3:H` or `#155 R32:H`), and it cannot use space groups 114-119 (not found in macromolecular crystallography) even in the standard setting due to difficulties with the use of `asuset` in the version of `ccp4` libraries used in `PHENIX` for these settings and space groups.

Literature

Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 63, 101-7 (2006). Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006).

- Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 63, 101-7 (2006).

- Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 62, 915-22 (2006).

List of all available keywords

ligandfitdata = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" , or input_labels="FP PHIB FOM". ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES) model = None PDB file with model for everything but the ligand quick = False Set to True for running as quickly as possible.

crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given. space_group = None Space Group symbol (i.e., C2221 or C 2 2 21) file_info file_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file) pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'. ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information output_file target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb. input_files existing_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match exactly) ncs_in = None You can supply a file with NCS information for use with ligands_from_ncs input_ligand_compare_file = None If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file. cif_def_file_list = None You can supply CIF files for real-space refinement after fitting refinement_file = None You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refine fobs_labels = None Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner_free_label = None Label for FreeR_flag in refinement_file... same format as for phenix.refine map_in = None Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use. search_parameters fixed_ligand = False Use fixed ligand (no rotations of any bonds) if set conformers = 1 Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate. group_search = 0 Enter the ID number of the group from the ligand to use to seed the search for conformations ligand_cc_min = 0.75 Enter...

- ligandfitdata = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" , or input_labels="FP PHIB FOM". ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES) model = None PDB file with model for everything but the ligand quick = False Set to True for running as quickly as

possible. crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.space_group = None Space Group symbol (i.e., C2221 or C 2 2 21) file_infofile_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file) pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.input_filesexisting_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match exactly)ncs_in = None You can supply a file with NCS information for use with ligands_from_ncsinput_ligand_compare_file = None If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file. cif_def_file_list = None You can supply CIF files for real-space refinement after fittingrefinement_file = None You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refinefobs_labels = None Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner_free_label = None Label for FreeR_flag in refinement_file...same format as for phenix.refinemap_in = None Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.search_parametersfixed_ligand = False Use fixed ligand (no rotations of any bonds) if setconformers = 1 Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate. group_search = 0 Enter the ID number of the group from the ligand to use to seed the search for conformations ligand_cc_min = 0.75 Ente...

- data = None Datafile. This can be any format if only FP is to be read in. If phases are to be read in then MTZ format is required. The Wizard will guess the column identification. If you want to specify it you can say input_labels="FP" ; , or input_labels="FP PHIB FOM";.

- ligand = None Three-letter code of ligand, or file containing information about the ligand (PDB or SMILES)

- model = None PDB file with model for everything but the ligand

- quick = False Set to True for running as quickly as possible.

- crystal_infounit_cell = None Enter cell parameter a b c alpha beta gamma resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)

- unit_cell = None Enter cell parameter a b c alpha beta gamma

- resolution = 0 High-resolution limit. Zero means keep everything. If map_in is specified, resolution must be given.
- space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)
- file_infile_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands input_labels = None Labels for input data columns lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting
fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file)
fobs_map: Fo map phased on partial model (requires FOBS in your input file)
pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- file_or_file_list = *single_file file_with_list_of_files Choose if you want to input a single file with PDB or other information about the ligand or if you want to input a file containing a list of files with this information for a list of ligands
- input_labels = None Labels for input data columns
- lig_map_type = fo-fc_difference_map fobs_map *pre_calculated_map_coeffs Enter the type of map to use in ligand fitting fo-fc_difference_map: Fo-Fc difference map phased on partial model (requires FOBS in your input file) fobs_map: Fo map phased on partial model (requires FOBS in your input file)
pre_calculated_map_coeffs: map calculated from FP PHIB [FOM] coefficients in input data file (or 2FOFCWT PH2FOFCWT coeffs) If you supply a map just leave this at pre_calculated_map_coeffs. If your labels choice is not available and '---' is present, you can choose '---'.
- ligand_format = *PDB SMILES Enter whether the files contain SMILES strings or PDB formatted information
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- input_filesexisting_ligand_file_list = None You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration. ligand_start = None LigandFit will attempt to put your ligand superimposing on ligand_start if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the ligand_start ligand must be present in the ligand_file ligand (and must match exactly)ncs_in = None You can supply a file with NCS information for use with
ligands_from_ncsinput_ligand_compare_file = None If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file. cif_def_file_list = None You can supply cif files for real-space refinement after fittingrefinement_file = None You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with phenix.refinefobs_labels = None Labels for Fobs SigFobs or lobs Siglobs for refinement_file... same format as for phenix.refiner_free_label = None Label for FreeR_flag in refinement_file...same format as for phenix.refinemap_in = None Map file (alternative to data file). Can be .ccp4, .map, .mrc and will be converted to map coefficients before use.

- `existing_ligand_file_list = None` You can enter a list of PDB files with ligands you have already fit. These will be used to exclude that region from consideration.
- `ligand_start = None` LigandFit will attempt to put your ligand superimposing on `ligand_start` if supplied. This must have some of the same atoms as your ligand, but does not have to have all of them. NOTE: all the atom names AND residue numbers in the `ligand_start` ligand must be present in the `ligand_file` ligand (and must match exactly)
- `ncs_in = None` You can supply a file with NCS information for use with `ligands_from_ncs`
- `input_ligand_compare_file = None` If you enter a PDB file with a ligand in it, the coordinates of the newly-built ligand will be compared with the coordinates in this file.
- `cif_def_file_list = None` You can supply cif files for real-space refinement after fitting
- `refinement_file = None` You can supply a file for full refinement containing F/I SIGF/SIGI FreeR_flag If you supply this file then after real-space refinement a round of full refinement will be carried out with `phenix.refine`
- `fobs_labels = None` Labels for Fobs SigFobs or lobs Siglobs for `refinement_file`... same format as for `phenix.refine`
- `r_free_label = None` Label for FreeR_flag in `refinement_file`...same format as for `phenix.refine`
- `map_in = None` Map file (alternative to data file). Can be `.ccp4`, `.map`, `.mrc` and will be converted to map coefficients before use.
- `search_parametersfixed_ligand = False` Use fixed ligand (no rotations of any bonds) if `setconformers = 1` Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate. `group_search = 0` Enter the ID number of the group from the ligand to use to seed the search for conformations `ligand_cc_min = 0.75` Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations `ligand_completeness_min = 1` Enter the minimum completeness of the ligand to the map to quit searching for more conformations `local_search = True` If `local_search` is True then, only the region within `search_dist` of the point in the map with the highest local rmsd will be searched in the FFT search for fragments `search_dist = 10` If `local_search` is True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments `use_cc_local = False` You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify `use_cc_local`, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$. `ligands_from_ncs = False` You can try to use ncs (from your partial model file or from your `ncs_in` file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand. `max_ligands_from_ncs = 1` You can specify how many of the ligands already found to consider using NCS (usually 1) `n_group_search = 3` Enter the number of different fragments of the ligand that will be looked for in FFT search of the map `n_indiv_tries_max = 10` If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified `n_indiv_tries_min = 5` If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified `number_of_ligands = 1` Number of copies of the ligand expected in the asymmetric unit `offsets_list = 7 53 29` You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models. `refine_ligand = True` You can carry out real-space refinement on the ligand after fitting `ligand_occupancy = 1.0` You can set the initial occupancy of the

ligand real_space_target_weight = 10. You can carry change the weight on the real-space term in real-space refinement on the ligand after fittingfittingParameters for tracing liganddelta_phi_ligand = 40 Specify the angle (degrees) between successive tries in FFT search for fragments fit_phi_inc = 20 Specify the angle (degrees) between rotations around bonds fit_phi_range = -180 180 Range of bond rotation angles to search

- fixed_ligand = False Use fixed ligand (no rotations of any bonds) if set
- conformers = 1 Enter how many conformers to create. If greater than 1, then ELBOW will always be used to generate them. If 1 then ELBOW will be used if a PDB file is not specified. These conformers are used to identify allowed torsion angles for your ligand. The alternative is to use the empirical rules in RESOLVE. ELBOW takes longer but is more accurate.
- group_search = 0 Enter the ID number of the group from the ligand to use to seed the search for conformations
- ligand_cc_min = 0.75 Enter the minimum correlation coefficient of the ligand to the map to quit searching for more conformations
- ligand_completeness_min = 1 Enter the minimum completeness of the ligand to the map to quit searching for more conformations
- local_search = True If local_search is True then, only the region within search_dist of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- search_dist = 10 If local_search is True then, only the region within this distance of the point in the map with the highest local rmsd will be searched in the FFT search for fragments
- use_cc_local = False You can specify the use of a local correlation coefficient for scoring ligand fits to the map. If you do not do this, then the region over which the ligand is scored are all points within 2.5 Å of the atoms in the ligand. If you do specify use_cc_local, then the region over which the ligand is scored are all these points, plus all the contiguous points that have density greater than $0.5 * \sigma$.
- ligands_from_ncs = False You can try to use ncs (from your partial model file or from your ncs_in file) along with any ligands already found to place additional copies of your ligand. Only applicable if there is one type of ligand.
- max_ligands_from_ncs = 1 You can specify how many of the ligands already found to consider using NCS (usually 1)
- n_group_search = 3 Enter the number of different fragments of the ligand that will be looked for in FFT search of the map
- n_indiv_tries_max = 10 If 0 is specified, all fragments are searched at once otherwise all are first searched at once then individually up to the number specified
- n_indiv_tries_min = 5 If 0 is specified, all placements of a fragment are tested at once otherwise all are first tested at once then individually up to the number specified
- number_of_ligands = 1 Number of copies of the ligand expected in the asymmetric unit
- offsets_list = 7 53 29 You can specify an offset for the orientation of the templates in searching for ligands. This is used in generating diversity in models.
- refine_ligand = True You can carry out real-space refinement on the ligand after fitting
- ligand_occupancy = 1.0 You can set the initial occupancy of the ligand
- real_space_target_weight = 10. You can carry change the weight on the real-space term in real-space refinement on the ligand after fitting

- fittingParameters for tracing ligand `delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds `fit_phi_range = -180 180` Range of bond rotation angles to search
- `delta_phi_ligand = 40` Specify the angle (degrees) between successive tries in FFT search for fragments
- `fit_phi_inc = 20` Specify the angle (degrees) between rotations around bonds
- `fit_phi_range = -180 180` Range of bond rotation angles to search
- `search_targetligand_near_chain = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. If you set `ligand_near_chain="None"` or leave it blank or do not set it, then all chains will be included. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line). `ligand_near_res = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain` The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line). `ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`. `name_of_ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`. `search_center = 0.0 0.0 0.0` Enter coordinates for center of search region (ignored if [0.,0.,0.])
- `ligand_near_chain = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain`. If you set `ligand_near_chain="None"` or leave it blank or do not set it, then all chains will be included. The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).
- `ligand_near_res = None` You can specify where to search for the ligand either with `search_center` or with `ligand_near_res` and `ligand_near_chain` The keywords `ligand_near_res` and `ligand_near_chain` refer to residue/chain in the file defined by `input_partial_model_file` (or model if running from command line).
- `ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
- `name_of_ligand_near_pdb = None` You can specify where LigandFit should look for your ligands by providing a PDB file containing one or more copies of the ligand. If you want you can provide a PDB file with ligand+ macromolecule and specify the ligand name with `name_of_ligand_near_pdb`.
- `search_center = 0.0 0.0 0.0` Enter coordinates for center of search region (ignored if [0.,0.,0.])
- `generalextennd_try_list = True` You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with `nbatch`. `ligand_id = None` You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in `input_lig_file` has. The keyword is only valid if a single copy of the ligand is to be found. `nbatch = 5` You can specify the number of processors to use (`nproc`) and the number of batches to divide the data into for parallel jobs. Normally you will set `nproc` to the number of processors available and leave `nbatch` alone. If you leave `nbatch` as `None` it will be set automatically, with a value depending on the Wizard. This is recommended. The value of `nbatch` can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the

same value of nbatch. nproc = 1 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch. If you set nproc=Auto and your machine has n processors, then it will use n-1 processors, or 1 if only 1 is available. resolve_command_list = None Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: coarse_grid resolution 200 2.0 hkl_in test.mtz . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes. coot_name = "coot" If your version of coot is called something else, then you can specify that here. i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screen. background = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end. max_wait_time = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'. wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:.... cache_resolve_libs = True Use caching of resolve libraries to speed up resolve. resolve_size = 12 Size for solve/resolve ('<small>giant</small>', '<small>huge</small>', '<small>extra_huge</small>' or a number where 12=giant 18=huge) check_run_command = False You can have the wizard check your run command at startup. run_command = "sh " When you specify nproc=nn, you can run the subprocesses as jobs in background with sh (default...

- extend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch.
- ligand_id = None You can specify an integer value for the ID of a ligand... This number will be added to whatever residue number the ligand search model in input_lig_file has. The keyword is only valid if a single copy of the ligand is to be found.
- nbatch = 5 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch.
- nproc = 1 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same

results, keep the same value of nbatch. If you set nproc=Auto and your machine has n processors, then it will use n-1 processors, or 1 if only 1 is available

- `resolve_command_list` = None Commands for resolve. One per line in the form: keyword value (put in the quotes.) value can be optional Examples: `coarse_grid resolution 200 2.0 hkl test.mtz` . NOTE: for command-line usage you need to enclose the whole set of commands in double quotes and each individual command in single quotes.
- `coot_name` = "coot" If your version of coot is called something else, then you can specify that here.
- `i_ran_seed` = 72432 Random seed (positive integer) for model-building and simulated annealing refinement
- `raise_sorry` = False You can have any failure end with a Sorry instead of simply printout to the screen
- `background` = True When you specify `nproc=nn`, you can run the jobs in background (default if `nproc` is greater than 1) or foreground (default if `nproc=1`). If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `check_wait_time` = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end
- `max_wait_time` = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time` = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....
- `cache_resolve_libs` = True Use caching of resolve libraries to speed up resolve
- `resolve_size` = 12 Size for solve/resolve ("", "_giant", "_huge", "_extra_huge" or a number where 12=giant 18=huge
- `check_run_command` = False You can have the wizard check your run command at startup
- `run_command` = "sh " When you specify `nproc=nn`, you can run the subprocesses as jobs in background with `sh` (default) or submit them to a queue with the command of your choice (i.e., `qsub`). If you have a multi-processor machine, use `sh`. If you have a cluster, use `qsub` or the equivalent command for your system. NOTE: If you set `run_command=qsub` (or otherwise submit to a batch queue), then you should set `background=False`, so that the batch queue can keep track of your runs. There is no need to use `background=True` in this case because all the runs go as controlled by your batch system. If `nproc` is greater than 1 and you use `run_command='sh '` (or similar, `sh` is default) then normally you will use `background=True` so that all the jobs run simultaneously.
- `queue_commands` = None You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: `queue_commands='#PBS -N mr_rosetta'`
`queue_commands='#PBS -j oe'` `queue_commands='#PBS -l walltime=03:00:00'`
`queue_commands='#PBS -l nodes=1:ppn=1'` NOTE: you can put in the characters '<path>' in any `queue_commands` line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated

by '_', up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1_/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1'

- `condor_universe` = vanilla The universe for condor is usually vanilla. However you might need to set it to local for your cluster
- `add_double_quotes_in_condor` = True You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.
- `condor` = None Specifies if the `group_run_command` is submitting a job to a condor cluster. Set by default to True if `group_run_command=condor_submit`, otherwise False. For condor job submission `mr_rosetta` uses a customized script with condor commands. Also uses `one_subprocess_level=True`
- `last_process_is_local` = True If true, run the last process in a group in background with `sh` as part of the job that is submitting jobs. This prevents having the job that is submitting jobs sit and wait for all the others while doing nothing
- `skip_r_factor` = False You can skip R-factor calculation if refinement is not done and `maps_only=True`
- `test_flag_value` = Auto Normally leave this at Auto (default). This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with `test_flag_value`.
- `skip_xtriage` = False You can bypass xtriage if you want. This will prevent you from applying anisotropy corrections, however.
- `base_path` = None You can specify the base path for files (default is current working directory)
- `temp_dir` = None Define a temporary directory
- `local_temp_directory` = None Write all temporary files to this directory and copy files specified by `keep_files` back to TEMP directory in working directory at the end of run. Used to speed up read/write operations. NOTE: Deletes all files except those specified by `keep_files` just like `clean_up=True`.
- `clean_up` = None At the end of the entire run the TEMP directories will be removed if `clean_up` is True. Also all files except for `overall_best_xxx` (final) files and `.eff` files In the `AutoSol_run_xx_` `AutoBuild_run_xx_` etc directory will be removed Files listed in `keep_files` will not be deleted. If you want to remove files after your run is finished use a command like `"phenix.autobuild run=1 clean_up=True"`;
- `print_citations` = True Print citations at end of run
- `solution_output_pickle_file` = None At end of run, write solutions to this file in output directory if defined
- `job_title` = None Job title in PHENIX GUI, not used on command line
- `top_output_dir` = None This is used in subprocess calls of wizards and to tell the Wizard where to look for the STOPWIZARD file.
- `wizard_directory_number` = None This is used by the GUI to define the run number for Wizards. It is the same as `desired_run_number` NOTE: this value can only be specified on the command line, as the directory number is set before parameters files are read.
- `verbose` = False Command files and other verbose output will be printed
- `extra_verbose` = False Facts and possible commands will be printed every cycle if True
- `debug` = False You can have the wizard stop with error messages about the code if you use debug. Additionally the output goes to the terminal if you specify `"debug=True"`;

- `require_nonzero = True` Require non-zero values in data columns to consider reading in.
- `remove_path_word_list = None` List of words identifying paths to remove from PATH. These can be used to shorten your PATH. For example... `cns ccp4 coot` would remove all paths containing these words except those also containing `phenix`. Capitalization is ignored.
- `fill = False` Fill in all missing reflections to resolution `res_fill`. Applies to density modified maps. See also `filled_2fofc_maps` in `autobuild`.
- `res_fill = None` Resolution for filling in missing data (default = highest resolution of any datafile). Only applies to density modified maps. Default is fill to high resolution of data. Ignored if `fill=False`
- `check_only = False` Just read in and check initial parameters. Not for general use
- `keep_files = ligandfit*.pdb` List of files that are not to be cleaned up. wildcards permitted
- `displaynumber_of_solutions_to_display = None` Number of solutions to put on screen and to write out
`solution_to_display = 1` Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- `number_of_solutions_to_display = None` Number of solutions to put on screen and to write out
- `solution_to_display = 1` Solution number of the solution to display and write out (use 0 to let the wizard display the top solution)
- `run_controlignore_blanks = None` `ignore_blanks` allows you to have a command-line keyword with a blank value like `"input_lig_file_list="`; `stop = None` You can stop the current wizard with `"stopwizard"` or `"stop"`; . If you type `"phenix.autobuild run=3 stop"`; then this will stop run 3 of `autobuild`. `display_facts = None` Set `display_facts` to `True` and optionally `run=[run-number]` to display the facts for run `run-number`. If you just say `display_facts` then the facts for the highest-numbered existing run will be shown. `display_summary = None` Set `display_summary` to `True` and optionally `run=[run-number]` to show the summary for run `run-number`. If you just say `display_summary` then the summary for the highest-numbered existing run will be shown. `carry_on = None` Set `carry_on` to `True` to carry on with highest-numbered run from where you left off. `run = None` Set run to `n` to continue with run `n` where you left off. `copy_run = None` Set `copy_run` to `n` to copy run `n` to a new run and continue where you left off. `display_runs = None` List all runs for this wizard. `delete_runs = None` List runs to delete: `1 2 3-5 9:12` `display_labels = None` `display_labels=test.mtz` will list all the labels that identify data in `test.mtz`. You can use the label strings that are produced in `AutoSol` to identify which data to use from a datafile like this: `peak.data="F+ SIGF+ F- SIGF-"`; # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in `AutoSol` and `AutoBuild` like this: `input_refinement_labels="FP SIGFP FreeR_flags"`; # each individual label counts `dry_run = False` Just read in and check parameter names `params_only = False` Just read in and return parameter defaults `display_all = False` Just read in and display parameter defaults `scoot = None` Not presently applicable for `ligandfit`
- `ignore_blanks = None` `ignore_blanks` allows you to have a command-line keyword with a blank value like `"input_lig_file_list="`;
- `stop = None` You can stop the current wizard with `"stopwizard"` or `"stop"`; . If you type `"phenix.autobuild run=3 stop"`; then this will stop run 3 of `autobuild`.
- `display_facts = None` Set `display_facts` to `True` and optionally `run=[run-number]` to display the facts for run `run-number`. If you just say `display_facts` then the facts for the highest-numbered existing run will be shown.
- `display_summary = None` Set `display_summary` to `True` and optionally `run=[run-number]` to show the summary for run `run-number`. If you just say `display_summary` then the summary for the

highest-numbered existing run will be shown.

- `carry_on = None` Set `carry_on` to `True` to carry on with highest-numbered run from where you left off.
- `run = None` Set `run` to `n` to continue with run `n` where you left off.
- `copy_run = None` Set `copy_run` to `n` to copy run `n` to a new run and continue where you left off.
- `display_runs = None` List all runs for this wizard.
- `delete_runs = None` List runs to delete: 1 2 3-5 9:12
- `display_labels = None` `display_labels=test.mtz` will list all the labels that identify data in `test.mtz`. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: `peak.data="F+ SIGF+ F- SIGF"`; # the entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol and AutoBuild like this: `input_refinement_labels="FP SIGFP FreeR_flags"`; # each individual label counts
- `dry_run = False` Just read in and check parameter names
- `params_only = False` Just read in and return parameter defaults
- `display_all = False` Just read in and display parameter defaults
- `coot = None` Not presently applicable for `ligandfit`
- `special_keywordswrite_run_directory_to_file = None` Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.
- `write_run_directory_to_file = None` Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.
- `non_user_parameters` These are obsolete parameters and parameters that the wizards use to communicate among themselves. Not normally for general use.
 - `gui_output_dir = None` Used only by the GUI
 - `sg = None` Obsolete. Use `space_group` instead
 - `get_lig_volume = False` You can ask to get the volume of the ligand and to then stop
 - `input_data_file = None` Not normally used. Use `"data"`; instead
 - `input_lig_file = None` Not normally used. Use `"ligand"`; instead.
 - `ligand_code = None` Not normally used. Use `"ligand"`; instead.
 - `input_partial_model_file = None` Not normally used. Use `"model"`; instead
 - `cif_already_generated = False` You can specify that the ligand cif file is already generated

Using the LigandFit GUI

ligandfit_gui.html

Contents

- Overview
- Configuration
- Running the program
- Output
- References

Overview

LigandFit is one of the four PHENIX "wizards" for automated structure determination. It performs flexible ligand fitting into difference maps using RESOLVE, followed by optional real-space refinement. As elsewhere, this documentation covers the use of the GUI specifically, and is not a general-purpose reference for program function. See the command-line documentation for a complete list of parameters and details on usage. If you want to see an example of how LigandFit works, we recommend reading the tutorial first.

Note that waters will be omitted when placing the ligand. However, if you are using pre-calculated map coefficients, you may need to remove waters beforehand to avoid "flattening" the difference density around the ligand. Protein atoms overlapping with the ligand density will prevent a successful placement, so some editing of the input structure is generally recommended.

Configuration

The first tab in the LigandFit window contains a field for adding input files, which can be dragged and dropped from the desktop, or picked from a file browser by clicking the "+" button.

To run, you will need the following input files:

A PDB file containing the initial ligand-free model. (It is okay for this to contain previously-placed ligands as long as the intended binding site is empty.) Water molecules will be automatically removed if necessary. An MTZ file containing either experimental amplitudes (F,SIGF) or pre-calculated difference map coefficients. A file describing your ligand. This should be either a PDB file containing a single residue, or a text file containing a SMILES string. In the latter case, eLBOW will be used to generate the starting geometry.

- A PDB file containing the initial ligand-free model. (It is okay for this to contain previously-placed ligands as long as the intended binding site is empty.) Water molecules will be automatically removed if necessary.
- An MTZ file containing either experimental amplitudes (F,SIGF) or pre-calculated difference map coefficients.
- A file describing your ligand. This should be either a PDB file containing a single residue, or a text file containing a SMILES string. In the latter case, eLBOW will be used to generate the starting geometry.

Additional optional files:

A PDB file containing ligand coordinates for comparison at the end of the program. A PDB file containing previously fit ligands (in addition to the starting model), which will be used to exclude occupied regions from fitting. CIF files defining ligand restraints for refinement. A reflections file containing F/I, SIGF/SIGI, and

FreeR_flag for running a full round of phenix.refine after fitting.

- A PDB file containing ligand coordinates for comparison at the end of the program.
- A PDB file containing previously fit ligands (in addition to the starting model), which will be used to exclude occupied regions from fitting.
- CIF files defining ligand restraints for refinement.
- A reflections file containing F/I, SIGF/SIGI, and FreeR_flag for running a full round of phenix.refine after fitting.

Ligandfit will try to guess what each input file should be used for automatically. If it does not choose the desired data type, you can change this by right-clicking the "Data type" column for that file, or selecting the file and clicking the "Modify file data type" button below the list.

MTZ column labels (and map type, if applicable) will be extracted automatically when a file is loaded.

By default, LigandFit runs multiple slightly different jobs in parallel and picks the best result from these; however, check the "Quick search" box if you want it to run a fast single job instead.

Other options:

Ligand map type tells LigandFit what type of map to calculate or expect in the input file. If you are supplying map coefficients from phenix.refine or another program, "Pre-calculated" is appropriate. If you are using raw data, "Fo-Fc" is the standard, but you may use "Fobs" instead. Minimum CC sets the correlation to density required to consider a placement successful. Real-space refinement will optimize the fit to density after placing the ligand. This is optional (and may be unnecessary if you intend to run phenix.refine immediately afterward), but is usually very fast and improves the fit. Number of conformers to create determines how the ligand is partitioned for placement of fragments. When this is set to 1, RESOLVE will try to identify rigid groups automatically. Larger numbers will instead use eLBOW to generate multiple conformations which are used to identify the rigid groups. This is somewhat slower, but often more accurate.

- Ligand map type tells LigandFit what type of map to calculate or expect in the input file. If you are supplying map coefficients from phenix.refine or another program, "Pre-calculated" is appropriate. If you are using raw data, "Fo-Fc" is the standard, but you may use "Fobs" instead.
- Minimum CC sets the correlation to density required to consider a placement successful.
- Real-space refinement will optimize the fit to density after placing the ligand. This is optional (and may be unnecessary if you intend to run phenix.refine immediately afterward), but is usually very fast and improves the fit.
- Number of conformers to create determines how the ligand is partitioned for placement of fragments. When this is set to 1, RESOLVE will try to identify rigid groups automatically. Larger numbers will instead use eLBOW to generate multiple conformations which are used to identify the rigid groups. This is somewhat slower, but often more accurate.

Running the program

Unless quick mode is enabled, LigandFit typically works by starting multiple similar jobs with slightly different parameters, to sample as many candidate ligand conformations and density fits as possible. Although this increases runtime substantially, it parallelizes well across multiple CPUs if they are available, and a queuing system may be utilized to split up tasks.

Depending on how you have configured your computer and your Phenix installation, as well as the operating system used, you may have several options for starting LigandFit:

Run locally; this is the default mode (and on Windows, the only mode), and offers the best integration with the GUI, but has the limitation that the GUI may not be closed while the program is running. Run "detached": this starts a completely separate process, which will allow you to close the GUI and resume later. (Not available on Windows.) Run on a queuing system: this is usually only available on Linux, and requires that queuing support be enabled in the GUI preferences. Currently Sun Grid Engine is the preferred system, although PBS can be made to work with some customization. If you use this option, be sure to set the number of processors to 1, or the job may interfere with other processes on the queuing system.

- Run locally; this is the default mode (and on Windows, the only mode), and offers the best integration with the GUI, but has the limitation that the GUI may not be closed while the program is running.
- Run "detached": this starts a completely separate process, which will allow you to close the GUI and resume later. (Not available on Windows.)
- Run on a queuing system: this is usually only available on Linux, and requires that queuing support be enabled in the GUI preferences. Currently Sun Grid Engine is the preferred system, although PBS can be made to work with some customization. If you use this option, be sure to set the number of processors to 1, or the job may interfere with other processes on the queuing system.

Additionally, if queuing system support is enabled, the GUI will also present an option to distribute the individual jobs across the queue. If this option is selected, either local or detached mode should be used, and the primary job will be run locally, with child processes submitted to the queue.

Output

LigandFit will generate a difference map used for fitting, and a separate PDB file for each copy placed. All of these can be opened at once in one of the graphics programs by clicking the buttons next to the file list. The button labeled "Run phenix.refine" will launch that GUI with the start model, reflections (if amplitudes were supplied), and all ligand output files already loaded.

As a general rule, a correlation coefficient (CC) above 0.7 is considered a good fit; CCs below this value usually indicate that the ligand is not placed correctly (although in rare cases the fit may be partially correct). In any case, you should always visually inspect the ligand placement in Coot, both before and after refinement. If LigandFit was unable to place the ligand correctly, there are several recommendations for troubleshooting:

If you do not obtain a good fit of your ligand to the map, there are a few things you should check:

Does your starting model have other molecules where the ligand needs to be placed? If so, then you will need to remove these before LigandFit can fit the ligand there. The LigandFit Wizard excludes all locations that are occupied by atoms in the starting model. Note that waters are the exception to this rule, and will be ignored as necessary - however, if you are fitting to an Fo-Fc difference map, the presence of waters may obscure ligand density. Have a careful look at all the output files. Work your way through the main log file (e.g., LigandFit_run_1_1.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? Have a look at the difference electron density map. Does it have clear density for the ligand? If not, you may want to try other types of maps. You can run phenix.maps to obtain a sigmaA-weighted mFo-DFc map that should optimally show the placement of your ligand. If you have atomic-resolution data, try setting the high-resolution limit slightly lower; data beyond approximately 1.8Å rarely improve the fit, and may actually make it more difficult.

- Does your starting model have other molecules where the ligand needs to be placed? If so, then you will need to remove these before LigandFit can fit the ligand there. The LigandFit Wizard excludes all locations that are occupied by atoms in the starting model. Note that waters are the exception to this rule,

and will be ignored as necessary - however, if you are fitting to an Fo-Fc difference map, the presence of waters may obscure ligand density.

- Have a careful look at all the output files. Work your way through the main log file (e.g., `LigandFit_run_1_1.log`). Is there anything strange or unusual in any of them that may give you a clue as to what to try next?
- Have a look at the difference electron density map. Does it have clear density for the ligand? If not, you may want to try other types of maps. You can run `phenix.maps` to obtain a sigmaA-weighted mFo-DFc map that should optimally show the placement of your ligand.
- If you have atomic-resolution data, try setting the high-resolution limit slightly lower; data beyond approximately 1.8Å rarely improve the fit, and may actually make it more difficult.

References

Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006). Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 63, 101-7 (2006).

- Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006).
- Ligand identification using electron-density map correlations. T.C. Terwilliger, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 63, 101-7 (2006).

Generating a polder map

polder.html

Contents

- Overview
- Running the program
- Video Tutorial
- How to interpret the output .log file
- Examples
- What if no ligand has been modeled yet?
- Other options
- Reference
- List of all available keywords

Overview

A polder map is an omit map which excludes the bulk solvent around the omitted region. This way, weak densities, which can be obscured by bulk solvent, may become visible. Typical omitted regions are:

ligands residues protruding into solvent area and with unclear side chain orientations (such as lysine, arginine, etc.) C- or N-terminal residues

- ligands
- residues protruding into solvent area and with unclear side chain orientations (such as lysine, arginine, etc.)
- C- or N-terminal residues

It is not recommended to omit large portions of the model at once (as the excluded bulk solvent volume might become too large) or to omit single atoms. Instead: several residues or one ligand are typical selections for polder maps. It should be kept in mind that phenix.polder does map calculations, no bias-removal strategies (such as refinement, annealing, etc.) are applied.

A presentation file with examples can be found [here](#).

Running the program

phenix.polder can be executed via the GUI, or the command line:

```
% phenix.polder model.pdb data.mtz selection='chain B and resseq 3'
```

This command calculates an omit map of residue 3 in chain B. Common file formats are supported (.pdb and .cif for models; .mtz, .hkl files for data). The syntax for the selections is the same as used by other phenix tools.

The output of phenix.polder consists of a single MTZ file, named polder_map_coeffs.mtz by default. This file will contain amplitudes and phases for two maps:

mFo-DFc_polder: map coefficients of polder omit map mFo-DFc_omit: map coefficients for omit map where bulk solvent floods into omitted region. It is equivalent to remove the selection from the model and to calculate

a difference map. This map can be used for comparison with the polder map.

- mFo-DFc_polder: map coefficients of polder omit map
- mFo-DFc_omit: map coefficients for omit map where bulk solvent floods into omitted region. It is equivalent to remove the selection from the model and to calculate a difference map. This map can be used for comparison with the polder map.

To load the maps manually in Coot, choose the "Open MTZ, mmCIF, fcf or phs..." option and activate the "Is a Difference Map" option in the window which opens after choosing the filename

Video Tutorial

The tutorial video is available on the Phenix YouTube channel and covers the following topics:

- basic overview
- how to run phenix.polder via the GUI
- example of a polder map for molecule MES 88 in structure 1ABA

How to interpret the output .log file

After running phenix.polder, a log file called 'polder.log' is created in the output directory. The log file can be viewed by clicking on the "Run status" tab after the job is finished. Alternatively, the log file can be opened from the command line.

The log file can be divided in five parts:

1. Summary of input parameters: As most phenix tools, the header of the log file lists the input parameters used to perform the calculations.
2. List of atoms used for the polder OMIT selection: The atoms which are used to define the region where the bulk solvent is excluded are listed here. It is recommended to check this list to see if the atom selection was correct. The list is in PDB format, therefore containing the atom name, residue number, chain, coordinates, etc.
3. Crystallographic R- and Rfree-factors: for the unmodified input model and data, for polder map computation and for 'conventional' OMIT map calculation
4. List of correlation coefficients for assessing the confidence of interpreting the polder map: Details on how these CC values are calculated are described in the paper about phenix.polder (see reference at the end of the page). If CC(1,3) is larger than CC(1,2) and CC(2,3), then the density most likely corresponds to the atomic features of the polder OMIT selection. If CC(1,2) and CC(2,3) are larger or comparable to CC(1,3), then the density in the OMIT region resembles to bulk solvent density.
5. Values for plotting the Discrepancy function $D(q)$. This table should be rarely used.

Examples

The following two figures show examples where polder maps are more informative than conventional OMIT maps.

Solvent molecule MES88 in structure 1ABA. The positive and negative mFobs-DFmodel OMIT difference density is displayed in green and red, respectively. (a) OMIT map contoured at $\pm 3\sigma$. (b) Polder map contoured at $\pm 3\sigma$. In the OMIT map, there is only some density for the O, N and S atoms. The polder map shows difference electron density for the entire molecule

Residue Gln H105 in structure 1F8T: (a) Original 2mFobs-DFmodel (blue, 1σ contour) and mFobs-DFmodel maps, (b) OMIT map and (c) polder map. The positive and negative mFobs-DFmodel difference density contoured at 3σ is displayed in green and red, respectively. In (c), the GLN side chain was real space refined in COOT. There is no density for the GLN side chain in the 2mFobs-DFmodel map (a), and the residual mFobs-DFmodel map has a negative peak at the side chain position, suggesting that the modeled orientation is incorrect. The OMIT map (b) shows no density for the glutamine side chain. The polder map shows continuous v-shaped density suggesting a different orientation for the side chain (c).

What if no ligand has been modeled yet?

To compute a polder map, the ligand has to be present in the model. To calculate a difference map excluding locally the bulk solvent around a density peak, use the `compute_box` option of polder, which has a separate documentation page.

Other options

There are several other programs in Phenix for calculating omit maps.

`phenix.composite_omit_map` can produce relatively bias-free maps by computing a number of omit maps covering specific regions, and then combining the areas around the omitted regions. Default options allow the bulk solvent mask to extend into the omitted region. `phenix.refine` can be used to manually generate omit maps for a smaller selection of the model by specifying the `omit_selection` keyword. The single-omit mode of the composite omit map program is equivalent to this. `phenix.maps` can generate simple omit maps without refinement.

- `phenix.composite_omit_map` can produce relatively bias-free maps by computing a number of omit maps covering specific regions, and then combining the areas around the omitted regions. Default options allow the bulk solvent mask to extend into the omitted region.
- `phenix.refine` can be used to manually generate omit maps for a smaller selection of the model by specifying the `omit_selection` keyword. The single-omit mode of the composite omit map program is equivalent to this.
- `phenix.maps` can generate simple omit maps without refinement.

Reference

D. Liebschner, P. V. Afonine, N. W. Moriarty, B. K. Poon, O. V. Sobolev, T. C. Terwilliger and P. D. Adams, Polder maps: Improving OMIT maps by excluding bulk-solvent, *Acta Cryst. D*, 73, 148-157

List of all available keywords

`job_title` = None Job title in PHENIX GUI, not used on command line
`solvent_exclusion_mask_selection` = None Atoms around which bulk solvent mask is set to zero
`scattering_table` = *n_gaussian wk1995 it1992 neutron electron Scattering table for structure factors calculations
`output_file_name_prefix` = None Prefix for output filename
`mask_output` = False Additional output: ccp4 maps containing the solvent mask for initial model (mask_all.ccp4), when ligand is omitted (mask_omit.ccp4) and the mask used for polder (mask_polder.ccp4)
`debug` = False Additional output: biased omit map (ligand used for mask calculation but omitted from model)
`polder_resolution_factor` = 0.25 Used to determine the grid step = resolution_factor * high resolution
`sphere_radius` = 5 Radius of sphere around atoms where solvent mask is reset to zero
`box_buffer` = None Buffer around selection box: Increase the box for resetting the mask by a buffer
`compute_box` = False Reset mask within a box (parallel to unit cell axes) defined by an atom selection
`altloc_scale` = False Apply

special scaling procedure for alternate conformations (only one residue, no mix)guiGUI-specific parameter required for output directoryoutput_dir = None data_column_label = None free_column_label = None

- job_title = None Job title in PHENIX GUI, not used on command line
- solvent_exclusion_mask_selection = None Atoms around which bulk solvent mask is set to zero
- scattering_table = *n_gaussian wk1995 it1992 neutron electron Scattering table for structure factors calculations
- output_file_name_prefix = None Prefix for output filename
- mask_output = False Additional output: ccp4 maps containing the solvent mask for initial model (mask_all.ccp4), when ligand is omitted (mask_omit.ccp4) and the mask used for polder (mask_polder.ccp4)
- debug = False Additional output: biased omit map (ligand used for mask calculation but omitted from model)
- polderresolution_factor = 0.25 Used to determine the grid step = resolution_factor * high resolution
- sphere_radius = 5 Radius of sphere around atoms where solvent mask is reset to zero
- box_buffer = None Buffer around selection box: Increase the box for resetting the mask by a buffer.
- compute_box = False Reset mask within a box (parallel to unit cell axes) defined by an atom selection
- altloc_scale = False Apply special scaling procedure for alternate conformations (only one residue, no mix)
- guiGUI-specific parameter required for output directoryoutput_dir = None data_column_label = None free_column_label = None
- output_dir = None
- data_column_label = None
- free_column_label = None

Generating a polder map with the compute_box option

polder_box.html

Contents

- Overview
- Example
- How to use the compute_box option
- Why the mask should be inspected
- Do not increase the box_buffer too much

Overview

A polder OMIT map can help visualizing weak densities by excluding bulk solvent around the omitted region. This type of OMIT map is particularly useful for ligands. However, to calculate the polder map, it is necessary to have the ligand (or a placeholder) present in the model and there might be cases where it is useful to calculate a polder map when nothing has been placed yet. Strictly speaking, this type of map is thus not an OMIT map but a difference map, with the option to define a region where the bulk solvent mask is excluded. Such a map can be calculated with the use_box option of the polder tool.

Example

The COOT screenshot below shows difference density in the outer region of a protein (model 1ABA, in which the solvent molecule MES A88 was deleted: 1aba_mod.pdb). The positive difference density (green) seems connected, but the shape is not very clear.

The next figure shows the bulk solvent mask (blue) covering the density peak. The bulk solvent density might hide additional features of the peak.

The compute_box option of the polder tool reassigns a local, user-defined, box-shaped portion of the bulk solvent mask. This way, interesting difference density peaks can be inspected without bias from the bulk solvent density (with the benefit of not having to disable the bulk solvent model for the entire structure)

How to use the compute_box option

1. Obtain the selection: Select several atoms around the peaks, approximately forming a rectangle containing the area of interest. These atoms will be used to define the region in which the bulk solvent mask will be excluded. The atoms will still contribute to structure factor calculations, so they will not be omitted. For example, the following atoms could be used (name - residue number - chain):

CG1 48 A OE1 60 A CD2 7 A CB 64 A

- CG1 48 A
- OE1 60 A
- CD2 7 A
- CB 64 A

2. "Translate" this selection to a selection string recognized by Phenix: (chain A and ((resseq 48 and name CG1) or (resseq 60 and name OE1) or (resseq 7 and name CD2) or (resseq 64 and name CB)))

3. Run phenix.polder:

Command line usage: `% phenix.polder 1aba_mod.pdb 1aba.mtz compute_box=true box_buffer=3 mask_output=True selection='(chain A and ((resseq 48 and name CG1) or (resseq 60 and name OE1) or (resseq 7 and name CD2) or (resseq 64 and name CB)))'` If the calculation is performed with the GUI, activate the following options:

Command line usage:

`% phenix.polder 1aba_mod.pdb 1aba.mtz compute_box=true box_buffer=3 mask_output=True selection='(chain A and ((resseq 48 and name CG1) or (resseq 60 and name OE1) or (resseq 7 and name CD2) or (resseq 64 and name CB)))'`

If the calculation is performed with the GUI, activate the following options:

4. Meaning of the options:

`compute_box`: uses the `compute_box` option instead of the OMIT option in the polder tool `box_buffer`: Increase the box in every direction by the value specified (in Ånstroem). See the next section why this might be useful. `mask_output`: Output the bulk solvent mask. It is recommended to inspect the mask to make sure that the peak is included in the modified mask.

- `compute_box`: uses the `compute_box` option instead of the OMIT option in the polder tool
- `box_buffer`: Increase the box in every direction by the value specified (in Ånstroem). See the next section why this might be useful.
- `mask_output`: Output the bulk solvent mask. It is recommended to inspect the mask to make sure that the peak is included in the modified mask.

5. Check the modified mask in COOT: Open the model file in coot. Click "File" --> "Open Map" --> and choose the file `mask_polder.ccp4` (it might be necessary to increase the contour level to make the map appear). Center on the peak in question.

The bulk solvent mask (pink) does not cover the difference density peak anymore.

Another view showing the rectangular shape of the modified area.

6. Inspect the polder difference map: The difference density is clearer and a MES molecule fits it well.

Why the mask should be inspected

The bulk solvent mask is excluded in the smallest rectangle which contains the selected atoms and which is parallel to unit cell axes. It can thus occur that the box excluding the mask does not cover the density peak in question. The figure below shows the bulk solvent mask when `box_buffer=0`. The mask (blue) still covers part of the density to be investigated (bottom right).

To prevent this, either the parameter `box_buffer` can be used or the selection can be changed. The latter is preferred because a large box buffer can be detrimental to the map (see next section for an example).

Do not increase the box_buffer too much

If the value for `box_buffer` is too high, large volumes of bulk solvent might not be accounted for, which can be detrimental to the detailed features of the difference map which we want to visualize. The figure below shows the resulting difference map when `box_buffer = 12`. The difference map is worse than the initial map.

Explanation: The unit cell for 1ABA is relatively small (30, 48, 61 in P212121) and using such a large value for `box_buffer` is almost equivalent to not using any bulk solvent at all and keeping the intermolecular volume

empty. The value for `box_buffer` should therefore not exceed 5 A. Also, choosing carefully the selection defining the box can prevent the need for using the `box_buffer` parameter.

Quantum Mechanical Restraints (QMR) usage

[quantum_mechanical_restraints.html](#)

Contents

- Authors
- Theory and Background
- Running an example `mmtbx.quantum_interface` `phenix.refine`
- `mmtbx.quantum_interface`
- `phenix.refine`
- Parameters
- Literature

Authors

Nigel W. Moriarty, Dorothee Liebschner

Theory and Background

In macromolecular crystallographic structure refinement, ligands present challenges for the generation of geometric restraints due to their large chemical variability, their possible novel nature and their specific interaction with the binding pocket of the protein. Quantum-mechanical approaches are useful for providing accurate ligand geometries, but can be plagued by the number of minima in flexible molecules. In an effort to avoid these issues, the Quantum Mechanical Restraints (QMR) procedure optimizes the ligand geometry in situ, thus accounting for the influence of the macromolecule on the local energy minima of the ligand. The optimized ligand geometry is used to generate target values for geometric restraints during the crystallographic refinement. As demonstrated using a sample of >2330 ligand instances in >1700 protein–ligand models, QMR restraints generally result in lower deviations from the target stereochemistry compared with conventionally generated restraints. In particular, the QMR approach provides accurate torsion restraints for ligands and other entities.

QMR is part of the Quantum Interface (QI) module that covers the interaction of Phenix with QM packages.

Running an example

There are two major ways to interact with the QMR modules in Phenix.

- Run directly in `phenix.refine` by supplying a PHIL scope to specify the ligand details. This will modify the restraints in memory.
- Run a command line program (`mmtbx.quantum_interface`) that creates the QMR PHIL scope (which also works for the `phenix.refine` option) and then using the same program run the QM step to write the restraints file for refinement.

Note: Using the python2 installer requires the setting of the `PHENIX_MOPAC` environmental variable to point to the MOPAC executable. The python3 installer is shipped with the MOPAC executables.

`mmtbx.quantum_interface`

For a number of reasons, we will cover the latter first. Using the example of PDB:4w53, download from the PDB website using:

```
~> phenix.fetch_pdb 4w53 --mtz
```

Command to produce PHIL scope for QM minimisation using mmtbx.quantum_interface is:

```
~> mmtbx.quantum_interface 4w53.pdb format=qi write_qmr_phil=True
```

which will fail because of the bulk test for hydrogen atoms.

```
Sorry: Model must have Hydrogen atoms
```

Also note that the write_qmr_phil=True option writes a PHIL file. In this example, the format of the PHIL file works for QI interface but the default works for phenix.refine. Add Hydrogen atoms to the model using:

```
~> phenix.ready_set 4w53.pdb
```

which produces a file with hydrogens atoms – 4w53.updated.pdb. It also may produce a restraints file that needs to be used next but in this case because the restraints for BNZ (benzene) are shipped with Phenix it is not necessary to supply to the program unless you want a different list of restraints (not ideal values). This file can be created cheaply but is needed as the skeleton for the QMR values to be inserted in the bond, angle and torsion restraints. So, we can repeat the quantum interface command with the new files.:

```
~> mmtbx.quantum_interface 4w53.updated.pdb format=qi write_qmr_phil=True
```

This will prompt for the selection of the ligand which in this case is “1”.

```
1 : "chain A and resid 200 and resname MBN"
2 : "chain A and resid 201 and resname EPE"
```

```
Enter selection by choosing number or typing a new one ~>
```

The result is a PHIL scope for MBN QMR restraints:

```
4w53.updated_A_200_MBN.phil.
```

Alternatively, a selection can be supplied directly.:

```
mmtbx.quantum_interface 4w53.updated.pdb format=qi write_qmr_phil=True qi.selection="chain A and resid 200 and
```

A quick tour of the PHIL scope is helpful. The complete scope for the QMR module is shown at the end of this page. A reduced scope is shown here.

```
qi.qm_restraints {
  selection = "chain A and resid 200 and resname MBN"
  run_in_macro_cycles = *first_only first_and_last all last_only test
  buffer = 3.5
  calculate = *in_situ_opt starting_energy final_energy starting_strain final_strain starting_bound final_bound
  write_files = *restraints pdb_core pdb_buffer pdb_final_core *pdb_final_buffer
  restraints_filename = Auto
  package {
    program = *mopac test
    charge = Auto
    multiplicity = Auto
    method = Auto
    basis_set = Auto
    solvent_model = None
    nproc = 1
    read_output_to_skip_opt_if_available = True
    ignore_input_differences = False
    view_output = None
  }
}
```

The key items are the selection, buffer and package items. The selection is the active part of the QM minimisation. The buffer is the radius from the selection to include in the QM environment. The package scope defines the 3rd party QM package details.

The program will also suggest the next command line for running the QM part of the restraints calculation. The skeleton restraints file may need to be included.

```
~&gt; mmtbx.quantum_interface 4w53.updated.pdb run_qmr=True 4w53.updated_A_200_MBN.phil
```

The result is a set of QMR restraints:

```
4w53.updated_A_200_MBN_3.5_C_PM6-D3H4.cif
```

Provide this to the desired refinements. The final QM calculation upon which the restraints are based are in:

```
4w53.updated_cluster_final_A_200_MBN_3.5_C_PM6-D3H4.pdb
```

The input and output files for the MOPAC calculations are in the directory for inspection.

```
mopac_4w53.updated_A_200_MBN_3.5_C_PM6-D3H4.out
mopac_4w53.updated_A_200_MBN_3.5_C_PM6-D3H4.arc
mopac_4w53.updated_A_200_MBN_3.5_C_PM6-D3H4.mop
```

phenix.refine

The restraints file 4w53.updated_A_200_MBN_3.5_C_PM6-D3H4.cif can be used in a refinement or the restraints can be calculated on the fly in the refinement run.

To get a PHIL scope for phenix.refine, remove the format=qi parameter from the write_qmr command.:

```
~&gt; mmtbx.quantum_interface 4w53.updated.pdb write_qmr_phil=True qi.selection="chain A and resid 200 and resn
```

The refinement command can then be run.:

```
~&gt; phenix.refine 4w53.updated.pdb 4w53.mtz 4w53.updated_A_200_MBN.phil
```

The major output is below including the name of the restraints file.

```
===== Update restraints from QM =====
```

QM restraints calculations for macro cycle 1 Time for calculation of "chain A and resid 200 and resname MBN" using PM6-D3H4 None: - Updating QM restraints: "chain A and resid 200 and resname MBN" Core atoms " C MBN A 200 " (2721) " C1 MBN A 200 " (2722) " C2 MBN A 200 " (2723) " C3 MBN A 200 " (2724) " C4 MBN A 200 " (2725) " C5 MBN A 200 " (2726) " C6 MBN A 200 " (2727) " H1 MBN A 200 " (2728) " H2 MBN A 200 " (2729) " H3 MBN A 200 " (2730) " H2A MBN A 200 " (2731) " H3A MBN A 200 " (2732) " H4 MBN A 200 " (2733) " H5 MBN A 200 " (2734) " H6 MBN A 200 " (2735) Buffer residues ILE A 78 LEU A 84 VAL A 87 TYR A 88 LEU A 91 ALA A 99 MET A 102 VAL A 103 VAL A 111 LEU A 118 LEU A 121 PHE A 153 MBN A 200 Starting stats: bond: 0.011 angle: 1.42 dihedral: 0.00 RMS difference in entire QM model : 0.108 Interim stats : bond: 0.009 angle: 0.59 dihedral: 0.00 Writing PDB :

```
qmr_cluster_final_01_A_200_MBN_3.5_C_PM6-D3H4.pdb Checking Transfer 29 " C1 MBN A 200 " - " C2 MBN A 200 " 1.394 ~> 1.405 30 " C1 MBN A 200 " - " C6 MBN A 200 " 1.394 ~> 1.405 33 " C4 MBN A 200 " - " C5 MBN A 200 " 1.388 ~> 1.398 45 " C3 MBN A 200 " - " C4 MBN A 200 " 1.389 ~> 1.398 47 " C MBN A 200 " - " C1 MBN A 200 " 1.502 ~> 1.493 51 " C2 MBN A 200 " - " C3 MBN A 200 " 1.388 ~> 1.396 52 " C5 MBN A 200 " - " C6 MBN A 200 " 1.389 ~> 1.397 1 " C1 MBN A 200 " - " C MBN A 200 " - " H1 MBN A 200 " 111.6 ~> 108.9 2 " H2A MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 107.3 ~> 108.4 3 " H1 MBN A 200 " - " C MBN A 200 " - " H2A MBN A 200 " 107.2 ~> 108.2 4 " C2 MBN A 200 " - " C1 MBN A 200 " - " C6 MBN A 200 " 118.3 ~> 119.2 5 " C1 MBN A 200 " - " C MBN A 200 " - " H2A MBN A 200 " 110.9 ~> 111.7 6 " C1 MBN A 200 " - " C6 MBN A 200 " - " C5 MBN A 200 " 121.0 ~> 120.2 7 " C1 MBN A 200 " - " C2 MBN A 200 " - " C3 MBN A 200 " 121.0 ~> 120.2 8 " C MBN A 200 " - " C1 MBN A 200 " - " C2 MBN A 200 " 120.8 ~> 120.1 9 " C1
```

MBN A 200 " - " C2 MBN A 200 " - " H2 MBN A 200 " 119.4 ~> 120.1 10 " C1 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A 200 " 119.4 ~> 120.0 11 " C2 MBN A 200 " - " C3 MBN A 200 " - " H3 MBN A 200 " 119.8 ~> 119.5 12 " C4 MBN A 200 " - " C5 MBN A 200 " - " H5 MBN A 200 " 120.1 ~> 119.8 13 " H1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 108.1 ~> 107.8 14 " C3 MBN A 200 " - " C4 MBN A 200 " - " C5 MBN A 200 " 119.4 ~> 119.6 15 " C MBN A 200 " - " C1 MBN A 200 " - " C6 MBN A 200 " 120.9 ~> 120.7 16 " C4 MBN A 200 " - " C3 MBN A 200 " - " H3 MBN A 200 " 120.0 ~> 120.2 17 " C5 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.1 18 " C6 MBN A 200 " - " C5 MBN A 200 " - " H5 MBN A 200 " 119.8 ~> 119.9 19 " C5 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A 200 " 119.6 ~> 119.7 20 " C2 MBN A 200 " - " C3 MBN A 200 " - " C4 MBN A 200 " 120.2 ~> 120.3 21 " C4 MBN A 200 " - " C5 MBN A 200 " - " C6 MBN A 200 " 120.2 ~> 120.3 22 " C3 MBN A 200 " - " C2 MBN A 200 " - " H2 MBN A 200 " 119.6 ~> 119.6 23 " C1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 111.6 ~> 111.6 24 " C3 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.2 1 " C2 MBN A 200 " - " C MBN A 200 " - " C1 MBN A 200 " - " H1 MBN A 200 " 31.7 ~> 87.0 Finished stats : bond: 0.000 angle: 0.00 dihedral: 0.00 planarity (n=1) mean: 0.034 min: 0.034 max: 0.034 Writing restraints : qmr_01_A_200_MBN_3.5_C_PM6-D3H4.cif

QM restraints calculations for macro cycle 1

Time for calculation of "chain A and resid 200 and resname MBN" using PM6-D3H4 None: -

Updating QM restraints: "chain A and resid 200 and resname MBN"

Core atoms " C MBN A 200 " (2721) " C1 MBN A 200 " (2722) " C2 MBN A 200 " (2723) " C3 MBN A 200 " (2724) " C4 MBN A 200 " (2725) " C5 MBN A 200 " (2726) " C6 MBN A 200 " (2727) " H1 MBN A 200 " (2728) " H2 MBN A 200 " (2729) " H3 MBN A 200 " (2730) " H2A MBN A 200 " (2731) " H3A MBN A 200 " (2732) " H4 MBN A 200 " (2733) " H5 MBN A 200 " (2734) " H6 MBN A 200 " (2735) Buffer residues ILE A 78 LEU A 84 VAL A 87 TYR A 88 LEU A 91 ALA A 99 MET A 102 VAL A 103 VAL A 111 LEU A 118 LEU A 121 PHE A 153 MBN A 200

Core atoms

" C MBN A 200 " (2721) " C1 MBN A 200 " (2722) " C2 MBN A 200 " (2723) " C3 MBN A 200 " (2724) " C4 MBN A 200 " (2725) " C5 MBN A 200 " (2726) " C6 MBN A 200 " (2727) " H1 MBN A 200 " (2728) " H2 MBN A 200 " (2729) " H3 MBN A 200 " (2730) " H2A MBN A 200 " (2731) " H3A MBN A 200 " (2732) " H4 MBN A 200 " (2733) " H5 MBN A 200 " (2734) " H6 MBN A 200 " (2735)

" C MBN A 200 " (2721)

" C1 MBN A 200 " (2722)

" C2 MBN A 200 " (2723)

" C3 MBN A 200 " (2724)

" C4 MBN A 200 " (2725)

" C5 MBN A 200 " (2726)

" C6 MBN A 200 " (2727)

" H1 MBN A 200 " (2728)

" H2 MBN A 200 " (2729)

" H3 MBN A 200 " (2730)

" H2A MBN A 200 " (2731)

" H3A MBN A 200 " (2732)

" H4 MBN A 200 " (2733)

" H5 MBN A 200 " (2734)

" H6 MBN A 200 " (2735)

Buffer residues

ILE A 78 LEU A 84 VAL A 87 TYR A 88 LEU A 91 ALA A 99 MET A 102 VAL A 103 VAL A 111 LEU A 118
LEU A 121 PHE A 153 MBN A 200

ILE A 78

LEU A 84

VAL A 87

TYR A 88

LEU A 91

ALA A 99

MET A 102

VAL A 103

VAL A 111

LEU A 118

LEU A 121

PHE A 153

MBN A 200

Starting stats: bond: 0.011 angle: 1.42 dihedral: 0.00

RMS difference in entire QM model : 0.108

Interim stats : bond: 0.009 angle: 0.59 dihedral: 0.00

Writing PDB : qmr_cluster_final_01_A_200_MBN_3.5_C_PM6-D3H4.pdb

Checking

Transfer

29 " C1 MBN A 200 " - " C2 MBN A 200 " 1.394 ~> 1.405 30 " C1 MBN A 200 " - " C6 MBN A 200 " 1.394 ~>
1.405 33 " C4 MBN A 200 " - " C5 MBN A 200 " 1.388 ~> 1.398 45 " C3 MBN A 200 " - " C4 MBN A 200 "
1.389 ~> 1.398 47 " C MBN A 200 " - " C1 MBN A 200 " 1.502 ~> 1.493 51 " C2 MBN A 200 " - " C3 MBN A
200 " 1.388 ~> 1.396 52 " C5 MBN A 200 " - " C6 MBN A 200 " 1.389 ~> 1.397 1 " C1 MBN A 200 " - " C MBN
A 200 " - " H1 MBN A 200 " 111.6 ~> 108.9 2 " H2A MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 "
107.3 ~> 108.4 3 " H1 MBN A 200 " - " C MBN A 200 " - " H2A MBN A 200 " 107.2 ~> 108.2 4 " C2 MBN A
200 " - " C1 MBN A 200 " - " C6 MBN A 200 " 118.3 ~> 119.2 5 " C1 MBN A 200 " - " C MBN A 200 " - " H2A
MBN A 200 " 110.9 ~> 111.7 6 " C1 MBN A 200 " - " C6 MBN A 200 " - " C5 MBN A 200 " 121.0 ~> 120.2 7 "
C1 MBN A 200 " - " C2 MBN A 200 " - " C3 MBN A 200 " 121.0 ~> 120.2 8 " C MBN A 200 " - " C1 MBN A 200
" - " C2 MBN A 200 " 120.8 ~> 120.1 9 " C1 MBN A 200 " - " C2 MBN A 200 " - " H2 MBN A 200 " 119.4 ~>
120.1 10 " C1 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A 200 " 119.4 ~> 120.0 11 " C2 MBN A 200 " - "
C3 MBN A 200 " - " H3 MBN A 200 " 119.8 ~> 119.5 12 " C4 MBN A 200 " - " C5 MBN A 200 " - " H5 MBN A
200 " 120.1 ~> 119.8 13 " H1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 108.1 ~> 107.8 14 " C3

MBN A 200 " - " C4 MBN A 200 " - " C5 MBN A 200 " 119.4 ~> 119.6 15 " C MBN A 200 " - " C1 MBN A 200 " -
" C6 MBN A 200 " 120.9 ~> 120.7 16 " C4 MBN A 200 " - " C3 MBN A 200 " - " H3 MBN A 200 " 120.0 ~>
120.2 17 " C5 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.1 18 " C6 MBN A 200 " - "
C5 MBN A 200 " - " H5 MBN A 200 " 119.8 ~> 119.9 19 " C5 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A
200 " 119.6 ~> 119.7 20 " C2 MBN A 200 " - " C3 MBN A 200 " - " C4 MBN A 200 " 120.2 ~> 120.3 21 " C4
MBN A 200 " - " C5 MBN A 200 " - " C6 MBN A 200 " 120.2 ~> 120.3 22 " C3 MBN A 200 " - " C2 MBN A 200 "
- " H2 MBN A 200 " 119.6 ~> 119.6 23 " C1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 111.6 ~>
111.6 24 " C3 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.2 1 " C2 MBN A 200 " - " C
MBN A 200 " - " C1 MBN A 200 " - " H1 MBN A 200 " 31.7 ~> 87.0

29 " C1 MBN A 200 " - " C2 MBN A 200 " 1.394 ~> 1.405

30 " C1 MBN A 200 " - " C6 MBN A 200 " 1.394 ~> 1.405

33 " C4 MBN A 200 " - " C5 MBN A 200 " 1.388 ~> 1.398

45 " C3 MBN A 200 " - " C4 MBN A 200 " 1.389 ~> 1.398

47 " C MBN A 200 " - " C1 MBN A 200 " 1.502 ~> 1.493

51 " C2 MBN A 200 " - " C3 MBN A 200 " 1.388 ~> 1.396

52 " C5 MBN A 200 " - " C6 MBN A 200 " 1.389 ~> 1.397

1 " C1 MBN A 200 " - " C MBN A 200 " - " H1 MBN A 200 " 111.6 ~> 108.9

2 " H2A MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 107.3 ~> 108.4

3 " H1 MBN A 200 " - " C MBN A 200 " - " H2A MBN A 200 " 107.2 ~> 108.2

4 " C2 MBN A 200 " - " C1 MBN A 200 " - " C6 MBN A 200 " 118.3 ~> 119.2

5 " C1 MBN A 200 " - " C MBN A 200 " - " H2A MBN A 200 " 110.9 ~> 111.7

6 " C1 MBN A 200 " - " C6 MBN A 200 " - " C5 MBN A 200 " 121.0 ~> 120.2

7 " C1 MBN A 200 " - " C2 MBN A 200 " - " C3 MBN A 200 " 121.0 ~> 120.2

8 " C MBN A 200 " - " C1 MBN A 200 " - " C2 MBN A 200 " 120.8 ~> 120.1

9 " C1 MBN A 200 " - " C2 MBN A 200 " - " H2 MBN A 200 " 119.4 ~> 120.1

10 " C1 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A 200 " 119.4 ~> 120.0

11 " C2 MBN A 200 " - " C3 MBN A 200 " - " H3 MBN A 200 " 119.8 ~> 119.5

12 " C4 MBN A 200 " - " C5 MBN A 200 " - " H5 MBN A 200 " 120.1 ~> 119.8

13 " H1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 108.1 ~> 107.8

14 " C3 MBN A 200 " - " C4 MBN A 200 " - " C5 MBN A 200 " 119.4 ~> 119.6

15 " C MBN A 200 " - " C1 MBN A 200 " - " C6 MBN A 200 " 120.9 ~> 120.7

16 " C4 MBN A 200 " - " C3 MBN A 200 " - " H3 MBN A 200 " 120.0 ~> 120.2

17 " C5 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.1

18 " C6 MBN A 200 " - " C5 MBN A 200 " - " H5 MBN A 200 " 119.8 ~> 119.9

19 " C5 MBN A 200 " - " C6 MBN A 200 " - " H6 MBN A 200 " 119.6 ~> 119.7

20 " C2 MBN A 200 " - " C3 MBN A 200 " - " C4 MBN A 200 " 120.2 ~> 120.3

21 " C4 MBN A 200 " - " C5 MBN A 200 " - " C6 MBN A 200 " 120.2 ~> 120.3
 22 " C3 MBN A 200 " - " C2 MBN A 200 " - " H2 MBN A 200 " 119.6 ~> 119.6
 23 " C1 MBN A 200 " - " C MBN A 200 " - " H3A MBN A 200 " 111.6 ~> 111.6
 24 " C3 MBN A 200 " - " C4 MBN A 200 " - " H4 MBN A 200 " 120.3 ~> 120.2
 1 " C2 MBN A 200 " - " C MBN A 200 " - " C1 MBN A 200 " - " H1 MBN A 200 " 31.7 ~> 87.0

Finished stats : bond: 0.000 angle: 0.00 dihedral: 0.00

planarity (n=1) mean: 0.034 min: 0.034 max: 0.034

Writing restraints : qmr_01_A_200_MBN_3.5_C_PM6-D3H4.cif

The PHIL scope can be used for all subsequent refinements to recalculate the QM restraints.

Parameters

qm_restraintsselection = None selection for core of atoms to calculate new restraints via a QM geometry
 minimisationrun_in_macro_cycles = *first_only first_and_last all last_only test the steps of the refinement that
 the restraints generation is runbuffer = 3.5 distance to include entire residues into the enviroment of the
 corecalculate = *in_situ_opt starting_energy final_energy starting_strain final_strain starting_bound
 final_bound starting_binding final_binding starting_higher_single_point final_higher_single_point Choose QM
 calculations to runwrite_files = *restraints pdb_core pdb_buffer pdb_final_core pdb_final_buffer which ligand
 or cluster files to writeprotein_optimisation_freeze = *all None main_chain main_chain_to_beta
 main_chain_to_delta torsions the parts of protein residues that are frozen when an amino acid is the main
 selectionremove_water = False restraints_filename = Auto restraints filename is based on model name if not
 specifiedcleanup = all *most None ignore_x_h_distance_protein = False skip check on transfer of proton
 during QM optimisationignore_lack_of_h_on_ligand = False skip check on protonation of ligand for entities
 such as MgF3capping_groups = True include_nearest_neighbours_in_optimisation = False include the side
 chains of protein in the QM optimisationinclude_inter_residue_restraints = False do_not_update_restraints =
 False For testing and maybe getting strain energy of standard restraintsbuffer_selection = None use this
 instead of distance from selections specific_atom_chargesatom_selection = None charge = None
 specific_atom_multiplicitiesatom_selection = None multiplicity = None freeze_specific_atomsatom_selection =
 None packageprogram = *mopac test charge = Auto multiplicity = Auto method = Auto basis_set = Auto
 solvent_model = None nproc = 1 read_output_to_skip_opt_if_available = False ignore_input_differences =
 False view_output = None

- qm_restraintsselection = None selection for core of atoms to calculate new restraints via a QM geometry minimisationrun_in_macro_cycles = *first_only first_and_last all last_only test the steps of the refinement that the restraints generation is runbuffer = 3.5 distance to include entire residues into the enviroment of the corecalculate = *in_situ_opt starting_energy final_energy starting_strain final_strain starting_bound final_bound starting_binding final_binding starting_higher_single_point final_higher_single_point Choose QM calculations to runwrite_files = *restraints pdb_core pdb_buffer pdb_final_core pdb_final_buffer which ligand or cluster files to writeprotein_optimisation_freeze = *all None main_chain main_chain_to_beta main_chain_to_delta torsions the parts of protein residues that are frozen when an amino acid is the main selectionremove_water = False restraints_filename = Auto restraints filename is based on model name if not specifiedcleanup = all *most None ignore_x_h_distance_protein = False skip check on transfer of proton during QM optimisationignore_lack_of_h_on_ligand = False skip check on protonation of ligand for entities such as MgF3capping_groups = True include_nearest_neighbours_in_optimisation = False include the side

chains of protein in the QM optimisation
 include_inter_residue_restraints = False
 do_not_update_restraints = False For testing and maybe getting strain energy of standard
 restraints
 buffer_selection = None use this instead of distance from
 selection
 specific_atom_chargesatom_selection = None charge = None
 specific_atom_multiplicitiesatom_selection = None multiplicity = None
 freeze_specific_atomsatom_selection = None
 packageprogram = *mopac test charge = Auto multiplicity
 = Auto method = Auto basis_set = Auto solvent_model = None nproc = 1
 read_output_to_skip_opt_if_available = False ignore_input_differences = False view_output = None

- selection = None selection for core of atoms to calculate new restraints via a QM geometry minimisation
- run_in_macro_cycles = *first_only first_and_last all last_only test the steps of the refinement that the restraints generation is run
- buffer = 3.5 distance to include entire residues into the environment of the core
- calculate = *in_situ_opt starting_energy final_energy starting_strain final_strain starting_bound final_bound starting_binding final_binding starting_higher_single_point final_higher_single_point Choose QM calculations to run
- write_files = *restraints pdb_core pdb_buffer pdb_final_core pdb_final_buffer which ligand or cluster files to write
- protein_optimisation_freeze = *all None main_chain main_chain_to_beta main_chain_to_delta torsions the parts of protein residues that are frozen when an amino acid is the main selection
- remove_water = False
- restraints_filename = Auto restraints filename is based on model name if not specified
- cleanup = all *most None
- ignore_x_h_distance_protein = False skip check on transfer of proton during QM optimisation
- ignore_lack_of_h_on_ligand = False skip check on protonation of ligand for entities such as MgF3
- capping_groups = True
- include_nearest_neighbours_in_optimisation = False include the side chains of protein in the QM optimisation
- include_inter_residue_restraints = False
- do_not_update_restraints = False For testing and maybe getting strain energy of standard restraints
- buffer_selection = None use this instead of distance from selection
- specific_atom_chargesatom_selection = None charge = None
- atom_selection = None
- charge = None
- specific_atom_multiplicitiesatom_selection = None multiplicity = None
- atom_selection = None
- multiplicity = None
- freeze_specific_atomsatom_selection = None
- atom_selection = None
- packageprogram = *mopac test charge = Auto multiplicity = Auto method = Auto basis_set = Auto solvent_model = None nproc = 1 read_output_to_skip_opt_if_available = False ignore_input_differences = False view_output = None

- program = *mopac test
- charge = Auto
- multiplicity = Auto
- method = Auto
- basis_set = Auto
- solvent_model = None
- nproc = 1
- read_output_to_skip_opt_if_available = False
- ignore_input_differences = False
- view_output = None

Literature

In situ ligand restraints from quantum-mechanical methods. D. Liebschner, N.W. Moriarty, B.K. Poon, and P.D. Adams. Acta Cryst. D79, 100-110 (2023).

- In situ ligand restraints from quantum-mechanical methods. D. Liebschner, N.W. Moriarty, B.K. Poon, and P.D. Adams. Acta Cryst. D79, 100-110 (2023).

ReadySet!

ready_set.html

Contents

- Author
- Purpose
- General Procedure
- Ligand hydrogen addition
- Metal coordination
- Neutron exchange addition
- List of all available keywords

Author

Nigel W. Moriarty

Purpose

ReadySet! is a program designed to prepare a PDB file for refinement as in ReadySet! Refine!!!. It will add hydrogens to the protein model using phenix.reduce and to the ligands using eLBOW. The appropriate restraints are also written to disk. Hydrogens can also be added to water molecules. Deuterium atoms can be added to facilitate dual xray-neutron refinement. Metal coordination files are also generated.

General Procedure

Ligand hydrogen addition

Including hydrogens in a refinement leads to better models. ReadySet! will add hydrogens to the ligands using eLBOW and PDB Chemical Components. The input PDB file is divided into 'standard' residues including the standard aminoacids and RNA/DNA bases. The other residues (usually ligands) are tested using the three-letter and atomic names against the PHENIX monomer library and the PDB Chemical Components database.

If the ligand is determined to be in the PHENIX monomer library then the hydrogens are added with the atom naming from the library. This is done using a SMILES string taken from the PDB Chemical Components database and the atom names from the monomer library. In this case, the hydrogens are added to the output PDB file but there is no restraints written because phenix.refine will use the library restraints.

If the ligand is determined to be in the PDB Chemical Components database, the SMILES string and the atom names are used to generate a molecule that represents the ligand. The atomic naming is determined using either the version 2 or version 3 PDB names. The restraints are written to disk.

If no match is found in the PHENIX monomer library or PDB Chemical Components database, the residue atoms are used to generate the ligand. The restraints are written to disk.

Once there is a ligand representation including hydrogens, the ligand must be included in the output. For each copy of the ligand in the model the presentation is pruned to match the number of non-hydrogen atoms and

overlayed onto the ligand orientation. Hydrogens are added in an optimised geometry for each copy of the ligand.

Covalently bound ligands are handled and two files, the CIF link restraints file and the atom selection file, are output.

Metal coordination

Any metals in the model are coordinated and the results output in an "edits" for phenix.refine. The distances and angles in the PDB file are used in the output.

Neutron exchange addition

Deuteriums are added to aminoacids that exhibit exchangeable sites. The hydrogens are placed in alternative location "A" and the corresponding deuteriums are placed in "B".

List of all available keywords

ready_setinputpdb_file_name = None PDB filename containing protein model, ligands, metal centres and water molecules
cif_file_name = None CIF restraints file to provide chemical information for
ligandscif_dir_name = None directory containing CIF restraints file to provide chemical information for
ligands
ligand_cache_directory = None output_dir = None output_file_name = None actions
hydrogens = True
histidine_nitrogen_protonation = False neutron_option = *all_h all_d hd_and_h hd_and_d all_hd
add_h_to_water = False add_d_to_water = False neutron_exchange_hydrogens = False Add deuteriums in
exchangeable sites
perdeuterate = False Add deuteriums in all possible sites
ligands = True
optimise_ligand_geometry = False Perform an AM1 geometry optimisation on each
ligand
trust_residue_code_is_chemical_components_code = False metals = False Generate edits file for
metal coordination
remove_waters = False Remove waters from model
link_to_edits = False Output edits
determined by LINK record
optimise_final_geometry_of_hydrogens = True
metal_coordination
use_default_bondlengths = True Use standard defaults for ideal bondlengths. If false use
the input geometry values.
bond_cutoff = None output_angles = False run_options
random_seed =
03056941001 run_within_refine = False

- ready_setinputpdb_file_name = None PDB filename containing protein model, ligands, metal centres and water molecules
cif_file_name = None CIF restraints file to provide chemical information for
ligandscif_dir_name = None directory containing CIF restraints file to provide chemical information for
ligands
ligand_cache_directory = None output_dir = None output_file_name = None actions
hydrogens = True
histidine_nitrogen_protonation = False neutron_option = *all_h all_d hd_and_h hd_and_d all_hd
add_h_to_water = False add_d_to_water = False neutron_exchange_hydrogens = False Add deuteriums in
exchangeable sites
perdeuterate = False Add deuteriums in all possible sites
ligands = True
optimise_ligand_geometry = False Perform an AM1 geometry optimisation on each
ligand
trust_residue_code_is_chemical_components_code = False metals = False Generate edits file for
metal coordination
remove_waters = False Remove waters from model
link_to_edits = False Output edits
determined by LINK record
optimise_final_geometry_of_hydrogens = True
metal_coordination
use_default_bondlengths = True Use standard defaults for ideal bondlengths. If false use
the input geometry values.
bond_cutoff = None output_angles = False run_options
random_seed =
03056941001 run_within_refine = False

- inputpdb_file_name = None PDB filename containing protein model, ligands, metal centres and water molecules
cif_file_name = None CIF restraints file to provide chemical information for
ligandscif_dir_name = None directory containing CIF restraints file to provide chemical information for

ligandsligand_cache_directory = None output_dir = None output_file_name = None

- pdb_file_name = None PDB filename containing protein model, ligands, metal centres and water molecules
- cif_file_name = None CIF restraints file to provide chemical information for ligands
- cif_dir_name = None directory containing CIF restraints file to provide chemical information for ligands
- ligand_cache_directory = None
- output_dir = None
- output_file_name = None
- actionshydrogens = True histidine_nitrogen_protonation = False neutron_option = *all_h all_d hd_and_h hd_and_d all_hd add_h_to_water = False add_d_to_water = False neutron_exchange_hydrogens = False Add deuteriums in exchangeable sitesperdeuterate = False Add deuteriums in all possible sitesligands = True optimise_ligand_geometry = False Perform an AM1 geometry optimisation on each ligandtrust_residue_code_is_chemical_components_code = False metals = False Generate edits file for metal coordinationremove_waters = False Remove waters from modellink_to_edits = False Output edits determined by LINK recordsoptimise_final_geometry_of_hydrogens = True
- hydrogens = True
- histidine_nitrogen_protonation = False
- neutron_option = *all_h all_d hd_and_h hd_and_d all_hd
- add_h_to_water = False
- add_d_to_water = False
- neutron_exchange_hydrogens = False Add deuteriums in exchangeable sites
- perdeuterate = False Add deuteriums in all possible sites
- ligands = True
- optimise_ligand_geometry = False Perform an AM1 geometry optimisation on each ligand
- trust_residue_code_is_chemical_components_code = False
- metals = False Generate edits file for metal coordination
- remove_waters = False Remove waters from model
- link_to_edits = False Output edits determined by LINK records
- optimise_final_geometry_of_hydrogens = True
- metal_coordinationuse_default_bondlengths = True Use standard defaults for ideal bondlengths. If false use the input geometry values.bond_cutoff = None output_angles = False
- use_default_bondlengths = True Use standard defaults for ideal bondlengths. If false use the input geometry values.
- bond_cutoff = None
- output_angles = False
- run_optionsrandom_seed = 03056941001 run_within_refine = False
- random_seed = 03056941001
- run_within_refine = False

phenix.secondary_structure_restraints

secondary_structure_restraints.html

Description

It is a tool to generate secondary structure annotation in various formats using pdb file. When HELIX/SHEET records are present in input pdb file, no searching occurs, they are used as source of information about protein secondary structure. To force automatic detection of protein secondary structure elements, one needs to remove them from input file.

There are several methods available to search protein secondary structure:

- ksdssp (default): uses KSDSSP
- dssp: slightly different method by Nat Echols, stand-alone tool is mmtbx.dssp
- from_ca: tries to find protein secondary structure from position of C-alpha atoms. Finds more SS elements than other methods, works better when the structure is distorted. (author Tom Terwilliger).
- cablam: CaBLAM uses CA-trace geometry to validate protein backbone. Since the CA trace is more reliably modeled from low-resolution density than the full backbone trace, CaBLAM can identify modeling errors and intended secondary structure elements in low-resolution models. (author Christopher Williams).

Output format:

- phenix (default): Phil parameter file suitable to use with all Phenix tools except phenix.refine
- phenix.refine: Phil parameter file suitable to use with phenix.refine
- pdb: HELIX/SHEET records in PDB format
- pymol: output pymol script which can be loaded in pymol. It will display model with hydrogen bonds:

Command line usage examples

```
% phenix.secondary_structure_restraints model.pdb % phenix.secondary_structure_restraints model.pdb  
format=phenix.refine % phenix.secondary_structure_restraints model.pdb ignore_annotation_in_file=True %  
phenix.secondary_structure_restraints model.pdb search_method=from_ca format=pdb
```

```
% phenix.secondary_structure_restraints model.pdb
```

```
% phenix.secondary_structure_restraints model.pdb format=phenix.refine
```

```
% phenix.secondary_structure_restraints model.pdb ignore_annotation_in_file=True
```

```
% phenix.secondary_structure_restraints model.pdb search_method=from_ca format=pdb
```

GUI

Graphical user interface is currently available under phenix.refine GUI, "Refinement settings" tab, select "Use secondary structure restraints" checkbox and click "Select Atoms" button.

List of all available keywords

```
show_all_params = False filter_outliers = True format = *phenix phenix_refine phenix_bonds pymol pdb  
refmac kinemage csv quiet = False verbose = -1 output_prefix = None ignore_annotation_in_file = False
```

ignore annotation that is present in input file
`file_name = None` secondary_structures_by_chain = True Only applies if `search_method = from_ca`. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with `ss_by_chain=False`. If your model is complete use `ss_by_chain=True`. If your model is many fragments, use `ss_by_chain=False`.
`from_ca_conservative = False` various parameters changed to make `from_ca` method more conservative, hopefully to closer resemble `ksdssp`.
`max_rmsd = 1` Only applies if `search_method = from_ca`. Maximum rmsd to consider two chains with identical sequences as the same for ss identification.
`use_representative_chains = True` Only applies if `search_method = from_ca`. Use a representative of all chains with the same sequence. Alternative is to examine each chain individually. Can be much slower with `use_representative_of_chain=False` if there are many symmetry copies. Ignored unless `ss_by_chain` is True.
`max_representative_chains = 100` Only applies if `search_method = from_ca`. Maximum number of representative chains.
`enabled = False` Turn on secondary structure restraints (main switch).
`protein_enabled = True` Turn on secondary structure restraints for protein.
`search_method = *ksdssp from_ca cablam` Particular method to search protein secondary structure.
`distance_ideal_n_o = 2.9` Target length for N-O hydrogen bond.
`distance_cut_n_o = 3.5` Hydrogen bond with length exceeding this value will not be established.
`remove_outliers = True` If true, h-bonds exceeding `distance_cut_n_o` length will not be established.
`restrain_hbond_angles = True` helix.
`serial_number = None` helix identifier = None.
`enabled = True` Restrict this particular helix.
`selection = None` helix type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
`sigma = 0.05` slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value.
`hbond_donor = None` acceptor = None sheet.
`enabled = True` Restrict this particular sheet.
`first_strand = None` sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value.
`strand_selection = None` sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None
`hbond_donor = None` acceptor = None nucleic_acid.
`enabled = True` Turn on secondary structure restraints for nucleic acid.
`hbond_distance_cutoff = 3.4` Hydrogen bonds with length exceeding this limit will not be established.
`scale_bonds_sigma = 1`. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints.
`base_pair_enabled = True` Restrict this particular base-pair.
`base1 = None` Selection string selecting at least one atom in the desired residue.
`base2 = None` Selection string selecting at least one atom in the desired residue.
`saenger_class = 0` Type of base-pairing.
`restrain_planarity = False` Apply planarity restraint to this base-pair.
`planarity_sigma = 0.176` restrain_hbonds = True Restrict hydrogen bonds length for this base-pair.
`restrain_hb_angles = True` Restrict angles around hydrogen bonds for this base-pair.
`restrain_parallelity = True` Apply parallelity restraint to this base-pair.
`parallelity_target = 0` parallelity_sigma = 0.0335 stacking_pair.
`enabled = True` Restrict this particular base-pair.
`base1 = None` Selection string selecting at least one atom in the desired residue.
`base2 = None` Selection string selecting at least one atom in the desired residue.
`angle = 0` sigma = 0.0...

- `show_all_params = False`
- `filter_outliers = True`
- `format = *phenix phenix_refine phenix_bonds pymol pdb refmac kinemage csv`
- `quiet = False`
- `verbose = -1`
- `output_prefix = None`
- `ignore_annotation_in_file = False` ignore annotation that is present in input file
- `file_name = None`

- `secondary_structures_by_chain = True` Only applies if `search_method = from_ca`. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with `ss_by_chain=False`. If your model is complete use `ss_by_chain=True`. If your model is many fragments, use `ss_by_chain=False`. `from_ca_conservative = False` various parameters changed to make `from_ca` method more conservative, hopefully to closer resemble `ksdssp`. `max_rmsd = 1` Only applies if `search_method = from_ca`. Maximum rmsd to consider two chains with identical sequences as the same for ss identification. `use_representative_chains = True` Only applies if `search_method = from_ca`. Use a representative of all chains with the same sequence. Alternative is to examine each chain individually. Can be much slower with `use_representative_of_chain=False` if there are many symmetry copies. Ignored unless `ss_by_chain` is `True`. `max_representative_chains = 100` Only applies if `search_method = from_ca`. Maximum number of representative chains. `enabled = False` Turn on secondary structure restraints (main switch). `protein_enabled = True` Turn on secondary structure restraints for protein. `search_method = *ksdssp from_ca cablam` Particular method to search protein secondary structure. `distance_ideal_n_o = 2.9` Target length for N-O hydrogen bond. `distance_cut_n_o = 3.5` Hydrogen bond with length exceeding this value will not be established. `remove_outliers = True` If true, h-bonds exceeding `distance_cut_n_o` length will not be established. `restrain_hbond_angles = True` helix. `serial_number = None` helix identifier = None. `enabled = True` Restrict this particular helix. `selection = None` helix type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints. `sigma = 0.05` slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10. `angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value. `hbond_donor = None` acceptor = None sheet. `enabled = True` Restrict this particular sheet. `first_strand = None` sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10. `angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value. `strand_selection = None` sense = parallel antiparallel *unknown `bond_start_current = None` `bond_start_previous = None` `hbond_donor = None` acceptor = None nucleic acid. `enabled = True` Turn on secondary structure restraints for nucleic acid. `hbond_distance_cutoff = 3.4` Hydrogen bonds with length exceeding this limit will not be established. `scale_bonds_sigma = 1`. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints. `base_pair_enabled = True` Restrict this particular base-pair. `base1 = None` Selection string selecting at least one atom in the desired residue. `base2 = None` Selection string selecting at least one atom in the desired residue. `saenger_class = 0` Type of base-pairing. `restrain_planarity = False` Apply planarity restraint to this base-pair. `planarity_sigma = 0.176` `restrain_hbonds = True` Restrict hydrogen bonds length for this base-pair. `restrain_hb_angles = True` Restrict angles around hydrogen bonds for this base-pair. `restrain_parallelity = True` Apply parallelity restraint to this base-pair. `parallelity_target = 0` `parallelity_sigma = 0.0335` `stacking_pair_enabled = True` Restrict this particular base-pair. `base1 = None` Selection string selecting at least one atom in the desired residue. `base2 = None` Selection string selecting at least one atom in the desired residue. `angle = 0` sigma = 0.027

- `ss_by_chain = True` Only applies if `search_method = from_ca`. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with `ss_by_chain=False`. If your model is complete use `ss_by_chain=True`. If your model is many fragments, use `ss_by_chain=False`.

- `from_ca_conservative = False` various parameters changed to make `from_ca` method more conservative, hopefully to closer resemble `ksdssp`.

- `max_rmsd = 1` Only applies if `search_method = from_ca`. Maximum rmsd to consider two chains with identical sequences as the same for ss identification

- `use_representative_chains = True` Only applies if `search_method = from_ca`. Use a representative of all chains with the same sequence. Alternative is to examine each chain individually. Can be much slower with `use_representative_of_chain=False` if there are many symmetry copies. Ignored unless `ss_by_chain` is `True`.
- `max_representative_chains = 100` Only applies if `search_method = from_ca`. Maximum number of representative chains
- `enabled = False` Turn on secondary structure restraints (main switch)
- `proteinenabled = True` Turn on secondary structure restraints for `proteinsearch_method = *ksdssp`
`from_ca` cablam Particular method to search protein secondary structure.
`distance_ideal_n_o = 2.9` Target length for N-O hydrogen bond
`distance_cut_n_o = 3.5` Hydrogen bond with length exceeding this value will not be established
`remove_outliers = True` If true, h-bonds exceeding `distance_cut_n_o` length will not be established
`restrain_hbond_angles = True` helix
`serial_number = None` helix_identifier = None enabled = True Restraining this particular helix
`selection = None` helix_type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
`sigma = 0.05` slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value
`hbonddonor = None` acceptor = None sheet
`enabled = True` Restraining this particular sheet
`first_strand = None` sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value
`strandselection = None` sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None
`hbonddonor = None` acceptor = None
- `enabled = True` Turn on secondary structure restraints for protein
- `search_method = *ksdssp` from_ca cablam Particular method to search protein secondary structure.
- `distance_ideal_n_o = 2.9` Target length for N-O hydrogen bond
- `distance_cut_n_o = 3.5` Hydrogen bond with length exceeding this value will not be established
- `remove_outliers = True` If true, h-bonds exceeding `distance_cut_n_o` length will not be established
- `restrain_hbond_angles = True`
- `helixserial_number = None` helix_identifier = None enabled = True Restraining this particular helix
`selection = None` helix_type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
`sigma = 0.05` slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value
`hbonddonor = None` acceptor = None
- `serial_number = None`
- `helix_identifier = None`
- `enabled = True` Restraining this particular helix
- `selection = None`
- `helix_type = *alpha pi 3_10 unknown` Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
- `sigma = 0.05`
- `slack = 0`
- `top_out = False`

- angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
- angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular value
- hbonddonor = None acceptor = None
- donor = None
- acceptor = None
- sheetenabled = True Restrains this particular sheetfirst_strand = None sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular valuestrandselection = None sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None hbonddonor = None acceptor = None
- enabled = True Restrains this particular sheet
- first_strand = None
- sheet_id = None
- sigma = 0.05
- slack = 0
- top_out = False
- angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
- angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular value
- strandselection = None sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None
- selection = None
- sense = parallel antiparallel *unknown
- bond_start_current = None
- bond_start_previous = None
- hbonddonor = None acceptor = None
- donor = None
- acceptor = None
- nucleic_acidenabled = True Turn on secondary structure restraints for nucleic acidshbond_distance_cutoff = 3.4 Hydrogen bonds with length exceeding this limit will not be establishedscale_bonds_sigma = 1. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints.base_pairenable = True Restrains this particular base-pairbase1 = None Selection string selecting at least one atom in the desired residuebase2 = None Selection string selecting at least one atom in the desired residueaenger_class = 0 Type of base-pairingrestrain_planarity = False Apply planarity restraint to this base-pairplanarity_sigma = 0.176 restrain_hbonds = True Restrains hydrogen bonds length for this base-pairrestrain_hb_angles = True Restrains angles around hydrogen bonds for this base-pairrestrain_parallelity = True Apply parallelity restraint to this base-pairparallelity_target = 0 parallelity_sigma = 0.0335 stacking_pairenable = True Restrains this particular base-pairbase1 = None Selection string selecting at least one atom in the desired residuebase2 = None Selection string selecting at least one atom in the desired residueangle = 0 sigma = 0.027

- enabled = True Turn on secondary structure restraints for nucleic acids
- hbond_distance_cutoff = 3.4 Hydrogen bonds with length exceeding this limit will not be established
- scale_bonds_sigma = 1. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints.
- base_pairenabled = True Restraint this particular base-pair
 - base1 = None Selection string selecting at least one atom in the desired residue
 - base2 = None Selection string selecting at least one atom in the desired residue
 - saenger_class = 0 Type of base-pairing
 - restrain_planarity = False Apply planarity restraint to this base-pair
 - planarity_sigma = 0.176
 - restrain_hbonds = True Restrain hydrogen bonds length for this base-pair
 - restrain_hb_angles = True Restrain angles around hydrogen bonds for this base-pair
 - restrain_parallelity = True Apply parallelity restraint to this base-pair
 - parallelity_target = 0
 - parallelity_sigma = 0.0335
- enabled = True Restraint this particular base-pair
 - base1 = None Selection string selecting at least one atom in the desired residue
 - base2 = None Selection string selecting at least one atom in the desired residue
 - saenger_class = 0 Type of base-pairing
 - restrain_planarity = False Apply planarity restraint to this base-pair
 - planarity_sigma = 0.176
 - restrain_hbonds = True Restrain hydrogen bonds length for this base-pair
 - restrain_hb_angles = True Restrain angles around hydrogen bonds for this base-pair
 - restrain_parallelity = True Apply parallelity restraint to this base-pair
 - parallelity_target = 0
 - parallelity_sigma = 0.0335
- stacking_pairenabled = True Restraint this particular base-pair
 - base1 = None Selection string selecting at least one atom in the desired residue
 - base2 = None Selection string selecting at least one atom in the desired residue
 - angle = 0
 - sigma = 0.027